

ПОНЯТИЕ РЕЛЯЦИОННОЙ МОДЕЛИ ДАННЫХ

Реляционная модель данных основывается на математических принципах, которые вытекают из теории множеств и математической логики (раздел исчисления предикатов) [1]. Эти принципы впервые были применены в области моделирования данных в конце 1960-х гг. доктором Е. Ф. Коддом, а впервые опубликованы — в 1970 г. [2], [3]. Тогда же появились первые прототипы реляционных систем управления базами данных (СУБД). Долгое время считалось невозможным добиться эффективной реализации таких систем. Постепенное накопление методов и алгоритмов организации реляционных баз данных и управления ими привело к тому, что уже в середине 1980-х гг. реляционные системы практически вытеснили с мирового рынка ранние СУБД (иерархические и сетевые) [4].

Техническая статья «Реляционная модель данных для больших разделяемых банков данных» доктора Е. Ф. Кодда, опубликованная в 1970 г., является родоначальницей современной теории реляционных БД [3]. Доктор Кодд определил 12 правил реляционной модели (которые называют 12 правилами Кодда) [2].

12 правил Кодда

Реляционная СУБД должна быть способна полностью управлять базой данных через ее реляционные возможности:

1. Информационное правило — вся информация в РБД (включая имена таблиц и столбцов) должна определяться строго как значения в таблицах.
2. Гарантированный доступ — любое значение в РБД должно быть гарантированно доступно для использования через комбинацию имени таблицы, значения первичного ключа и имени столбца.
3. Поддержка пустых значений (`null value`) — СУБД должна уметь работать с пустыми значениями (неизвестными или неиспользованными).

ными значениями), в отличие от значений по умолчанию, и независимо для любой предметной области.

4. Онлайн-реляционный каталог — описание РБД и ее содержания должны быть представлены на логическом уровне как таблицы, к которым можно применять запросы, используя язык базы данных.
5. Исчерпывающий язык управления данными — по крайней мере один из поддерживаемых языков должен иметь четко определенный синтаксис и быть всеобъемлющим. Он должен поддерживать описание структуры данных и манипулирование ими, правила целостности, авторизацию и транзакции.
6. Правило обновления представлений (views) — все представления, теоретически обновляемые, могут быть обновлены через систему.
7. Вставка, обновление и удаление — СУБД поддерживает не только запрос на отбор данных, но и вставку, обновление и удаление.
8. Физическая независимость данных — на программы-приложения и специальные программы логически не влияют изменения физических методов доступа к данным и структур хранилищ данных.
9. Логическая независимость данных — на программы-приложения и специальные программы логически не влияют, в пределах разумного, изменения структур таблиц.
10. Независимость целостности — язык РБД должен быть способен определять правила целостности. Они должны сохраняться в онлайн-справочнике, и не должно существовать способа их обойти.
11. Независимость распределения — на программы-приложения и специальные программы логически не влияет, первый раз используются данные или повторно.
12. Согласование языковых уровней — невозможность обойти правила целостности, определенные через язык базы данных, использованием языков низкого уровня.

Кодд предложил применение реляционной алгебры к системе управления реляционной базой данных (СУРБД) для разбиения данных в связанные наборы. Он организовал свою СУБД вокруг теории, основанной на наборах данных.

Самая распространенная трактовка реляционной модели данных принадлежит известному последователю идей Кодда Кристоферу Дейту [1]. Согласно трактовке Дейта, реляционная модель состоит из трех частей, описывающих разные аспекты реляционного подхода: структурной части, манипуляционной части и целостной части.

В структурной части модели фиксируется, что единственной родовой структурой данных, используемой в РБД, является нормализованное n -арное отношение (кортеж). Определяются понятия domain атрибутов, кортежей, заголовка, тела и переменной отношения [5].

В манипуляционной части модели определяются два фундаментальных механизма манипулирования РБД — реляционная алгебра и реляционное исчисление. Первый механизм базируется на классической теории множеств, а второй — на классическом логическом аппарате исчисления предикатов первого порядка. Основной функцией манипуляционной части реляционной модели является обеспечение меры реляционности любого конкретного языка РБД: язык называется реляционным, если он обладает не меньшей выразительностью и мощностью, чем реляционная алгебра [6] или реляционное исчисление [7].

Ограничения целостности

Три типа **ограничений целостности** являются неотъемлемой частью реляционной модели данных: целостность объекта, ссылочная целостность и целостность домена.

Целостность сущности

Первое требование называется требованием **целостности сущности** (entity integrity). Объекту или сущности реального мира в РБД соответствуют кортежи отношений. Конкретно требование состоит в том, что любой

кортеж любого значения-отношения любой переменной отношения должен быть отличим от любого другого кортежа этого значения отношения по составным значениям заранее определенного множества атрибутов переменной отношения, т. е. **любая переменная отношения должна обладать первичным ключом**. Это требование автоматически удовлетворяется, если в системе не нарушаются базовые свойства отношений.

Целостность ссылок

Ссылочная целостность (eferential integrity) касается концепции внешнего ключа (foreign key). Связи между данными, хранимыми в разных отношениях, в реляционной БД устанавливаются с помощью использования внешних ключей — для установления связи между кортежем из отношения А с определённым кортежем отношения В в предусмотренные для этого атрибуты кортежа отношения А записывается значение первичного ключа (а в общем случае значение потенциального ключа) целевого кортежа отношения В. Таким образом, всегда имеется возможность выполнить две операции [8]:

- определить, с каким кортежем в отношении В связан определённый кортеж отношения А;
- найти все кортежи отношения А, имеющие связи с определённым кортежем отношения В.

Благодаря наличию связей в реляционной БД можно хранить факты без избыточного дублирования, т. е. в нормализованном виде. Ссылочная целостность может быть проиллюстрирована следующим образом:

Дана пара отношений А и В, связанных внешним ключом. Первичный ключ отношения В — атрибут В. key. Внешний ключ отношения А, ссылающийся на В, — атрибут А. b. Ссылочная целостность для пары отношений А и В имеет место тогда, когда выполняется условие: для каждого кортежа отношения А существует соответствующий кортеж отношения В, т. е. кортеж, у которого (В. key = А. b).

База данных обладает свойством ссылочной целостности, когда для любой пары связанных внешним ключом отношений в ней выполняется условие ссылочной целостности.

Если вышеприведённое условие не выполняется, говорят, что в базе данных нарушена ссылочная целостность. Такая БД не может нормально эксплуатироваться, так как в ней разорваны логические связи между зависимыми друг от друга фактами. Непосредственным результатом нарушения ссылочной целостности становится то, что корректным запросом не всегда удаётся получить корректный результат.

Целостность домена

Домен (domain) — это набор всех допустимых значений, которые может содержать атрибут. Понятие «домен» часто путают с понятием «тип данных». Необходимо различать эти два понятия. Тип данных — это физическая концепция, а домен — логическая. Например, «целое число» — это тип данных, а «возраст» — это домен, ограничение, накладываемое предметной областью на данный атрибут.

Ограничение домена — ограничения на значения атрибутов из множества значений домена. Если значение атрибута неизвестно, то значение атрибута — NULL.

Модель данных

Модель данных, или концептуальное описание предметной области — начальный уровень проектирования баз данных.

С точки зрения теории РБД основные принципы реляционной модели на концептуальном уровне можно сформулировать следующим образом [1], [2], [3]:

- все данные представляются в виде структуры, определенной в виде строк и столбцов и называемой **отношением**;
- все значения являются скалярами. Это означает, что для любой строки и столбца любого отношения существует одно и только одно значение;

- все операции выполняются над целым отношением, и результатом их выполнения также является целое отношение. Этот принцип называется замыканием.

Формулируя принципы реляционной модели, доктор Кодд [2] выбрал термин «отношение» (relation), так как он считал, что этот термин однозначен (в то время как, например, термин «таблица» имеет множество различных видов — таблица в тексте, электронная таблица и т. д.). Весьма распространено следующее заблуждение: реляционная модель названа так потому, что она определяет связи между таблицами. На самом деле название этой модели происходит от отношений (таблиц базы данных), лежащих в ее основе.

Каждая строка, содержащая данные, называется **кортежем**, каждый столбец отношения называется **атрибутом** (на уровне практической работы с современными РБД используются термины «запись» и «поле»).

Элементами описания реляционной модели данных на концептуальном уровне являются сущности, атрибуты, домены (domain) и связи.

Сущность

Сущность — некоторый обособленный объект или событие, информацию о котором необходимо сохранять в БД; сущность имеет определенный набор свойств — атрибутов. Сущности могут быть как физические (реально существующие объекты: например, СТУДЕНТ, атрибуты: № зачетной книжки, фамилия, факультет, специальность, № группы и т. д.), так и абстрактные (например, ЭКЗАМЕН, атрибуты: дисциплина, дата, преподаватель, аудитория и пр.). Для сущности различают ее тип и экземпляр. **Тип** характеризуется именем и списком свойств, а **экземпляр** — конкретными значениями свойств.

Атрибут

Атрибуты сущности бывают следующие.

Идентифицирующие и описательные. Идентифицирующие атрибуты имеют уникальное значение для сущностей данного типа и являются **потенциальными** ключами. Они позволяют однозначно распознавать экземпляры сущ-

ности. Из потенциальных ключей выбирается один первичный ключ (ПК). В качестве ПК обычно выбирается потенциальный ключ, по которому чаще происходит обращение к экземплярам записи. ПК должен включать в свой состав минимально необходимое для идентификации количество атрибутов. Остальные атрибуты называются описательными.

Простые и составные. Простой атрибут состоит из одного компонента, его значение неделимо. Составной атрибут является комбинацией нескольких компонентов, возможно, принадлежащих разным типам данных (например, адрес). Решение о том, использовать составной атрибут или разбивать его на компоненты, зависит от особенностей процессов его использования и может быть связано с обеспечением высокой скорости работы с большими базами данных.

Однозначные и многозначные — могут иметь соответственно одно или много значений для каждого экземпляра сущности.

Основные и производные. Значение основного атрибута не зависит от других атрибутов. Значение производного атрибута вычисляется на основе значений других атрибутов (например, возраст человека вычисляется на основе даты его рождения и текущей даты).

Спецификация атрибута состоит из его названия, указания типа данных и описания ограничений целостности — множества значений (или домена), которые может принимать данный атрибут.

Связи

Связи на концептуальном уровне представляют собой простые ассоциации между сущностями. Например, утверждение «Покупатели приобретают продукты» указывает, что между сущностями «Покупатели» и «Продукты» существует связь, и такие сущности называются участниками этой связи.

Существует несколько типов связей между двумя сущностями: это связи один-к-одному, один-ко-многим и многие-ко-многим.

Каждая связь в реляционной модели характеризуется именем, обязательностью, типом и степенью. Различают *факультативные и обязательные* связи.

Если сущность одного типа оказывается связанной с сущностью другого типа, то между этими типами объектов существует *обязательная* связь. Иначе связь является *факультативной*.

Степень связи определяется количеством сущностей, которые охвачены данной связью. Пример бинарной связи — связь между отделом и сотрудниками, которые в нем работают.

Диаграмма «сущность — связь»

Диаграмма «сущность — связь» (Entity-Relationship diagrams, или ER-diagram) служит для описания схемы базы на концептуальном уровне проектирования. Метод был предложен Питером Пин-Шань Ченом (Peter Pin-Shan Chen) в 1976 г. [9, 10].

В дальнейшем многими авторами были разработаны свои варианты подобных моделей (нотация Мартина [11], нотация IDEF1X [11], нотация Баркера [12] и др.). Кроме того, различные программные средства, реализующие одну и ту же нотацию, могут отличаться своими возможностями. По сути, все варианты диаграмм «сущность — связь» исходят из одной идеи — рисунок всегда нагляднее текстового описания. Все такие диаграммы используют графическое изображение сущностей предметной области, их свойств (атрибутов) и взаимосвязей между ними.

Рассмотрим последовательность действий, которые необходимо произвести, чтобы построить ER-диаграмму в нотации Баркера.

Необходимо *правильно и полно* описать предметную область будущей БД. В виде последовательного списка выписать все функции и запросы, которые будет выполнять БД.

В выписанном списке требований к БД подчеркнуть все существительные. Данное подмножество будет содержать *потенциальные* сущности предметной области.

Для выделенных сущностей необходимо выписать атрибуты. Атрибуты помещают в прямоугольник с заголовком сущности, к которой атрибуты относятся.

Выбирают для каждой сущности ключевые атрибуты (первичный ключ). Их обозначают подчеркиванием или выделяют жирным шрифтом. Вторичные ключи выделяют курсивом.

Определяют связи, которые существуют между атрибутами. Связи обозначаются линиями. Конец линии, который подходит к сущности, связанной отношением «многие», разделяется на три части.

Полученная диаграмма называется ER-диаграммой. После написания ER-диаграммы требуется ее анализ с последующим уточнением, если потребуется. Если на ER-диаграмме присутствуют связи многие-ко-многим, то такую связь нужно разделить на две один-ко-многим с помощью дополнительной сущности.

Если необходимо, строится уточненная ER-диаграмма, в которой отсутствуют связи многие-ко-многим. Полученная схема является прототипом будущей РБД: сущности являются таблицами, атрибуты — полями. Все связи можно реализовать стандартными средствами СУБД.

Проектирование схемы БД должно решать задачи минимизации дублирования данных, упрощения и ускорения процедур их обработки и обновления. При неправильно спроектированной схеме РБД могут возникнуть аномалии модификации данных. Для решения подобных проблем проводится нормализация отношений.

Однако в технологии работы с хранилищами данных может использоваться обратный прием — денормализация отношений с целью увеличения скорости выполнения запросов к очень большим объемам архивных данных.

В рамках реляционной модели данных Э. Ф. Коддом были разработаны принципы нормализации отношений [2], [3].

Нормализация

Нормализация — это формальный метод анализа отношений на основе их первичного ключа и существующих связей. Ее задача — это замена одной схемы (или совокупности отношений) БД другой схемой, в которой отношения имеют более простую и регулярную структуру.

Первая нормальная форма (1НФ) связана с понятиями простого и сложного атрибутов. Простой атрибут — это атрибут, значения которого атомарны (т. е. неделимы). Сложный атрибут может иметь значение, представляющее собой объединение нескольких значений одного или разных доменов. В первой нормальной форме устраняются повторяющиеся атрибуты или группы атрибутов, т. е. производится выявление неявных сущностей, «замаскированных» под атрибуты.

Отношение приведено к 1НФ, если все его атрибуты — простые, т. е. значение атрибута не должно быть множеством или повторяющейся группой. Для приведения таблиц к 1НФ необходимо разбить сложные атрибуты на простые, а многозначные атрибуты вынести в отдельные отношения [1].

Вторая нормальная форма (2НФ) применяется к отношениям с составными ключами (состоящими из двух и более атрибутов) и связана с понятиями функциональной зависимости. Если в любой момент времени каждому значению атрибута А соответствует единственное значение атрибута В, то В функционально зависит от А ($A \rightarrow B$). Во второй нормальной форме устраняются атрибуты, зависящие только от части уникального ключа. Эта часть уникального ключа определяет отдельную сущность. Отношение находится во 2НФ, если оно приведено к 1НФ и каждый неключевой атрибут функционально полно зависит от составного первичного ключа (ПК) [1].

Третья нормальная форма (3НФ) связана с понятием транзитивной зависимости. Пусть А, В, С — атрибуты некоторого отношения. При этом $A \rightarrow B$ и $B \rightarrow C$, но обратное соответствие отсутствует, т. е. С не зависит от В или В не зависит от А. Тогда говорят, что С транзитивно зависит от А ($A \rightarrow C$). В треть-

ей нормальной форме устраняются атрибуты, которые зависят от атрибутов, не входящих в уникальный ключ. Эти атрибуты являются основой отдельной сущности. Отношение находится в 3НФ, если оно находится во 2НФ и не имеет атрибутов, не входящих в ПК и находящихся в транзитивной зависимости от ПК [1].

Существуют также нормальная форма Бойса — Кодда (НФБК), 4НФ, 5НФ и т. д.

НФБК стоит особняком ото всех НФ. Для того чтобы отношение находилось в НФБК, не нужно, чтобы оно было в 3НФ. Если в отношении имеется несколько потенциальных ключей, то оно не подходит под формулировку 3НФ, так как не все атрибуты зависят только от первичного ключа, они могут зависеть и от потенциальных ключей. Для таких отношений и была задумана НФБК, которая гласит: отношение находится в НФБК в том случае, если все атрибуты находятся в функциональной зависимости от потенциального ключа. Например, в отношении определены следующие атрибуты: № п/п, № зачетной книжки, Ф. И. О. студента, оценка. В данном отношении определен ПК — № п/п. Атрибуты Ф. И. О. и оценка зависят как от ПК, так и от № зачетной книжки. Следовательно, данное отношение не находится в 3НФ, оно находится в НФБК, так как атрибут № зачетной книжки может быть выбран в качестве ПК, т. е. является потенциальным ключом.

Нормализацию отношений выше НФБК используют очень редко. Чем выше степень нормализации, тем больше таблиц будет получено, а это усложнит работу с БД.

Моделирование структуры базы данных при помощи алгоритма нормализации имеет серьезные недостатки: методика нормализации предполагает первоначальное размещение всех атрибутов проектируемой предметной области в одном отношении, что является очень неестественной операцией. Интуитивно разработчик сразу проектирует несколько отношений в соответствии с обнаруженными сущностями. Даже если совершить насилие над собой и создать одно

или несколько отношений, включив в них все предполагаемые атрибуты, то совершенно неясен смысл полученного отношения. Невозможно сразу определить полный список атрибутов. Пользователи имеют привычку называть разными именами одни и те же вещи или, наоборот, называть одними именами разные вещи. Для проведения процедуры нормализации необходимо выделить зависимости атрибутов, что тоже очень нелегко.

В реальном проектировании структуры базы данных применяется другой метод — так называемое *семантическое моделирование* [4]. Семантическое моделирование представляет собой моделирование структуры данных, опирающееся на смысл этих данных. В качестве инструмента семантического моделирования используются различные варианты диаграмм «сущность — связь (ER)» с построением концептуальной модели базы данных. Обычно проектирование с помощью ER-диаграмм позволяет построить РБД, нормализованную до 3НФ или НФБК. Нормализацию удобно применять для проверки ER-модели.

КРАТКИЕ СВЕДЕНИЯ ОБ ОПЕРАТОРАХ SQL

Structured Query Language (SQL) — язык структурированных запросов, можно разделить на несколько подразделов [13]:

Data Definition Language (DDL) — язык определения данных;

Data Manipulation Language (DML) — язык манипулирования данными;

Structured Query Language (SQL) — язык запросов.

Начнем изучение операторов SQL с последнего подраздела.

Выбор полей

Любой запрос начинается с оператора `SELECT`¹ (выбрать), после которого через запятую следует перечислить список полей (выражений), значения которых требуется выбирать. Если имя выражения или поля является зарезервированным в MS SQL Server значением, то его следует экранировать, заключив в квадратные скобки.

```
SELECT
    [выражение1]
    , [выражение2]
    , ...;
```

В предложении `SELECT` можно записать следующее выражение:

```
SELECT 1;
```

В результате выполнения запроса будет выведено «1» (рис. 1).

¹ SQL — регистронезависимый язык. Однако следует придерживаться определенных соглашений. Предикаты языка удобно записывать с использованием символов верхнего регистра, а имена таблиц и полей — в `PascalCase` (или, иначе, `UpperCamelCase`).

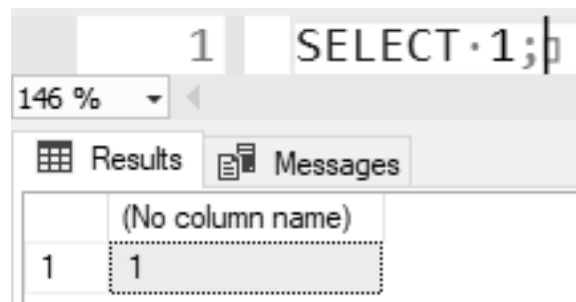


Рис. 1. Результат выполнения запроса

Следующим оператором является FROM (из), после которого следует название источника (таблицы), поля которой были указаны в предложении SELECT. Запрос заканчивается оператором «точка с запятой»²:

```
SELECT
    TargetTable.Field1
    , TargetTable.Field2
    , ...
    , TargetTable.FieldN
FROM TargetTable;
```

Для сокращения записи можно использовать вместо имени таблицы её псевдоним, имя которого может быть любым соответствующим правилу именования полей, т. е. начинаться с буквы и не иметь в своем составе других символом, кроме букв и цифр:

```
SELECT
    T.Field1
    , T.Field2
    ..., T.FieldN
FROM TargetTable AS3 T;
```

Так как все поля принадлежат только таблице TargetTable, то в предложении SELECT имя таблицы или псевдоним можно не указывать:

² В SQL Server Management Studio оператор «;» является необязательным.

³ Предикат AS в SQL Server Management Studio можно опустить.

```

SELECT
    Field1
    , Field2
    ,
    ..., FieldN
FROM TargetTable;

```

Если требуется выбирать все имеющиеся в таблице поля, то используется предикат «звездочка»:

```

SELECT *
FROM TargetTable;

```

Пример 1

Показать все поля из таблицы `Person.AddressType`⁴.

На рисунке 2 показаны все поля таблицы `Person.AddressType`.

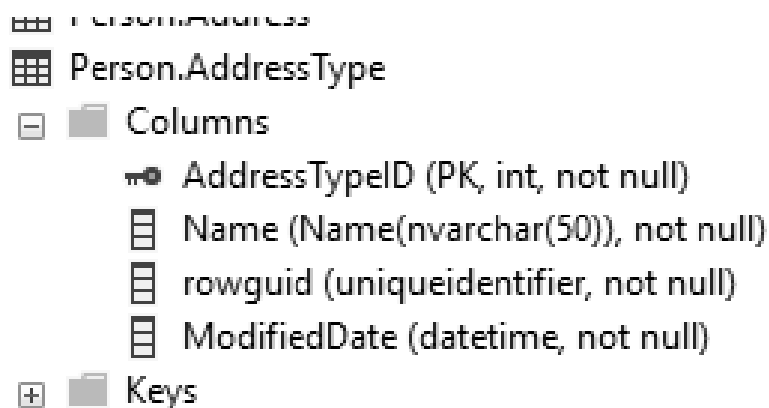


Рис. 2. Структура таблицы `Person.AddressType`

⁴ Для изучения операторов SQL необходимо иметь уже кем-то созданную и заполненную базу данных (БД). Корпорация Microsoft создала и заполнила учебную БД AdventureWorks2019, которую можно скачать с официального сайта и восстановить в SQL Server Management Studio (SSMS). Подробное описание того, как восстановить БД, приведено на официальном сайте.

Напишем скрипт запроса:

```
SELECT *  
FROM Person.AddressType;
```

Выполните запрос, нажав на кнопку «Выполнить» (Execute). Результат выполнения данного запроса представлен на рисунке 3. Как видно, на рисунке отображены все имеющиеся у таблицы поля.

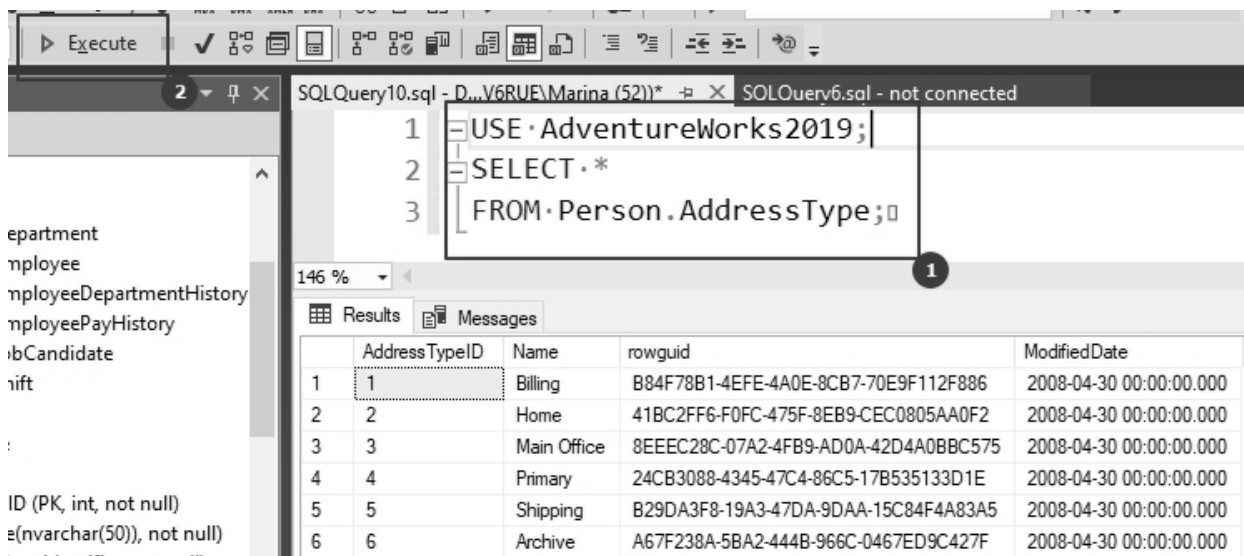


Рис. 3. Результат выполнения запроса (пример 1)

Пример 2

Из таблицы *Production.Product* БД *AdventureWorks* показать наименование продукта, его цвет и размер (выбрать значения полей *Name*, *Color* и *Size*).

Создайте новый запрос. Выберите БД, к которой собираетесь обращаться (рис. 3). Наберите текст запроса:

```
SELECT  
    Name  
    , Color  
    , Size  
FROM Production.Product;
```

Результат выполнения запроса представлен на рисунке 4.

Results		Messages	
	Name	Color	Size
281	Mountain-100 Black...	Black	42
282	Mountain-100 Black...	Black	44
283	Mountain-100 Black...	Black	48
284	Mountain-200 Silver,...	Silver	38
285	Mountain-200 Silver,...	Silver	42
286	Mountain-200 Silver,...	Silver	46

Рис. 4. Результат выполнения запроса (пример 2)

Фильтрация данных

Для фильтрации данных из выбранных полей используется оператор WHERE (где), после которого следует множество предикатов. Все записи из таблицы, значения полей которых при подстановке в предикат дают значение «истина», попадут в результирующий набор данных.

```

SELECT
    Field1
    , Field2
    , ...
    , FieldN
FROM TargetTable
WHERE <Условие 1> AND (OR) <Условие 2> ...;

```

Условий может быть несколько, сочетаться они могут при помощи различных комбинаций булевых операторов AND, OR, NOT⁵ (И, ИЛИ, НЕ соответственно).

⁵ Старайтесь строить запрос, не используя условие отрицания, так как на его отработку уходит больше времени. Если нельзя переписать условие на положительный исход, то для записи условия можно использовать как союз NOT, так и знак «<>» или сочетание «!=».

Пример 3

Выбрать из таблицы *Production.Product* товары, размер которых 44 или 46 (размер имеет символьный тип).

```
SELECT
    [Name]
    , Color
    , Size
FROM Production.Product
WHERE (Size = '44' OR Size = '46');
```

Результат выполнения запроса представлен на рисунке 5. Как видно, размер имеет только значение 44 или 46.

	Name	Color	Size
1	HL Road Frame - Red, 44	Red	44
2	LL Road Frame - Red, 44	Red	44
3	ML Road Frame - Red, 44	Red	44
4	LL Road Frame - Black, 44	Black	44
5	HL Mountain Frame - Silver, 44	Silver	44
6	HL Mountain Frame - Silver, 46	Silver	46
7	HL Mountain Frame - Black, 44	Black	44
8	HL Mountain Frame - Black, 46	Black	46
9	Road-150 Red, 44	Red	44
10	Road-450 Red, 44	Red	44
11	Road-650 Red, 44	Red	44
12	Road-150 Silver, 44	Silver	44

Рис. 5. Результат выполнения запроса (пример 3)

Условий может быть несколько, сочетаться они могут при помощи различных комбинаций булевых операторов AND, OR, NOT (И, ИЛИ, НЕ соответственно).

Дополним пример 3 ещё одним условием.

Пример 4

Выбрать из таблицы Production.Product товары чёрного цвета, размер которых 44 или 46 (размер имеет символьный тип).

```
SELECT
    [Name]
    , Color
    , Size
FROM Production.Product
WHERE (Size = '44' OR Size = '46') AND Color = 'Black';
```

Результат запроса представлен на рисунке 6.

Results		Messages	
	Name	Color	Size
1	LL Road Frame - Black, 44	Black	44
2	HL Mountain Frame - Black, 44	Black	44
3	HL Mountain Frame - Black, 46	Black	46
4	Road-650 Black, 44	Black	44
5	Mountain-100 Black, 44	Black	44
6	Mountain-200 Black, 46	Black	46
7	Mountain-300 Black, 44	Black	44
8	Road-250 Black, 44	Black	44
9	ML Mountain Frame - Black, 44	Black	44
10	HL Road Frame - Black, 44	Black	44

Рис. 6. Результат выполнения запроса (пример 4)

Как видно из рисунка 6, в результирующем множестве представлены только товары чёрного цвета (Black).

Предикат LIKE

Для фильтрации строковых данных на полное совпадение можно использовать знак равенства. Однако обычно для строковых переменных поиск по полному совпадению является частным случаем. Чаше используется частичное совпадение. Для таких целей применяется предикат LIKE. Формат данного предиката такой:

```
SELECT
    Fields1
    , Fields2
    , ...
    , FieldsN
FROM TableName
WHERE FieldK LIKE 'example';
-- Последняя строка эквивалентна
-- WHERE FieldK = 'example';
```

Предикат LIKE может принимать в качестве аргумента не только строку, но и шаблон. Например, для частичного совпадения можно использовать управляющие последовательности (escape-sequences):

% (знак процента) — заменяет любое количество любых символов;

_ (нижнее подчеркивание) — заменяет один символ;

[множество_символов] — любой одиночный символ, содержащийся в перечисленном множестве символов;

[^множество_символов] — любой одиночный символ, НЕ содержащийся в перечисленном множестве символов.

Пример 5

Выбрать из таблицы *Person.Person* (полные) имена всех сотрудников, фамилия которых начинается на буквосочетание 'Smi'.

```
SELECT
    LastName
  , Firstname
  , MiddleName
FROM Person.Person
WHERE LastName LIKE 'Smi%';
```

Как видно из рисунка 7, в результирующий набор данных попали сотрудники с фамилией Smith и Smith-Bates.

	LastName	Firstname	MiddleName
95	Smith	Terrance	I
96	Smith	Thomas	NULL
97	Smith	Tina	K
98	Smith	Tyler	J
99	Smith	Victoria	NULL
100	Smith	William	A
101	Smith	Wyatt	P
102	Smith	Xavier	M
103	Smith	Zachary	W
104	Smith-Bates	Lorin	NULL
105	Smith-Bates	Lorin	G.

**Рис. 7. Результат применения предиката LIKE
и управляющей последовательности «%»**

Пример 6

Выбрать из таблицы Person.Person (полные) имена всех сотрудников, фамилия которых состоит из 7 символов.

```
SELECT
    LastName
    , Firstname
    , MiddleName
FROM Person.Person
WHERE LastName LIKE '_____' ; --7 символов «подчеркивание»
```

На рисунке 8 представлен результирующий набор данных. Фамилии сотрудников состоят только из 7 букв.

	LastName	Firstname	MiddleName
1	Acevedo	Humberto	NULL
2	Aguilar	James	T.
3	Alameda	Lili	J.
4	Alberts	Amy	NULL
5	Alberts	Amy	E
6	Alberts	Amy	E.
7	Allison	Cecil	J.
8	Alvarez	Adrienne	L
9	Alvarez	Albert	R
10	Alvarez	Alberto	NULL
11	Alvarez	Andy	NULL

**Рис. 8. Результат применения предиката LIKE
и управляющей последовательности «_»**

Пример 7

Выбрать из таблицы Person.Person (полные) имена всех сотрудников, фамилия которых начинается на один из символов «А», «С», «Е», «I», «Z».

```
SELECT
    LastName
    , Firstname
    , MiddleName
FROM Person.Person
WHERE LastName LIKE '[A, C, E, I, Z]%' ;
```

Неполный результат выполнения данного запроса представлен на рисунке 9. Как видно из рисунка, после фамилии, начинающейся на символ «А», следует фамилия, которая начинается на символ «С».

	LastName	Firstname	MiddleName
906	Ashton	Chris	NULL
907	Ashton	Chris	NULL
908	Atkinson	Teresa	NULL
909	Ault	John	P.
910	Avalos	Robert	A.
911	Ayers	Stephen	M.
912	Cabello	Albert	S
913	Cai	Alan	NULL
914	Cai	Alejandro	J
915	Cai	Alisha	NULL

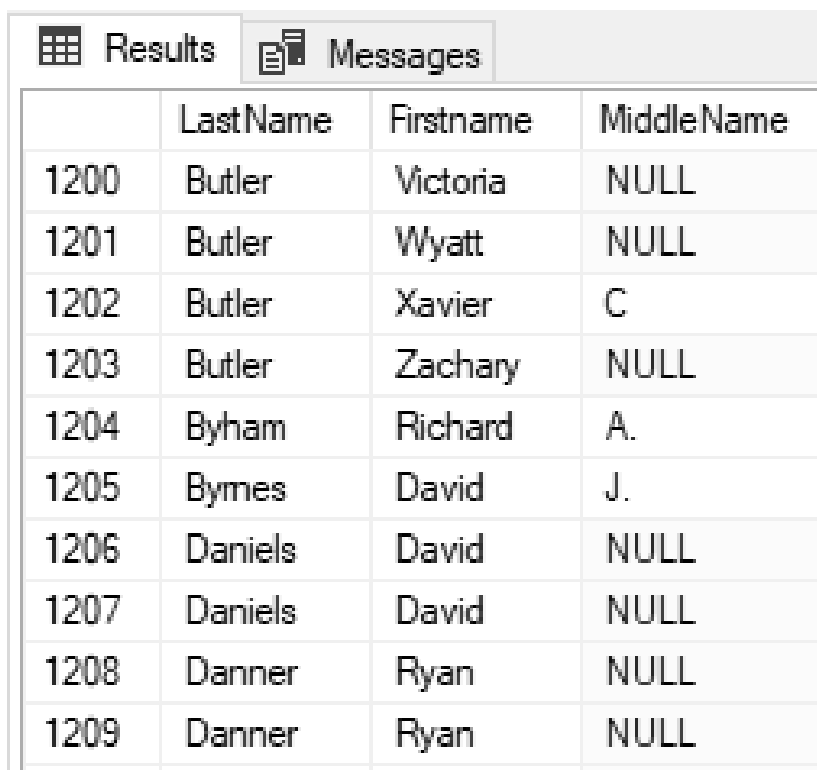
Рис. 9. Результат выполнения запроса (пример 7)

Пример 8

Выбрать из таблицы *Person.Person* (полные) имена всех сотрудников, фамилия которых не начинается на перечисленные символы «A», «C», «E», «I», «Z».

```
SELECT
    LastName
    , Firstname
    , MiddleName
FROM Person.Person
WHERE LastName LIKE '[^A, ^C, ^E, ^I, ^Z]%' ;
```

Результат выполнения скрипта представлен на рисунке 10. Из рисунка видно, что в результирующем наборе представлены фамилии, которые начинаются на символ «B», а затем на символ «D». Фамилии, которые начинаются на символ «E», пропущены.



	LastName	Firstname	MiddleName
1200	Butler	Victoria	NULL
1201	Butler	Wyatt	NULL
1202	Butler	Xavier	C
1203	Butler	Zachary	NULL
1204	Byham	Richard	A.
1205	Bymes	David	J.
1206	Daniels	David	NULL
1207	Daniels	David	NULL
1208	Danner	Ryan	NULL
1209	Danner	Ryan	NULL

Рис. 10. Результат выполнения запроса (пример 8)

Если необходимо найти записи, которые содержат специальные символы (escape-sequences), то можно воспользоваться одним из двух способов. На странице Помощи⁶ скопируйте текст создания, заполнения и выборки из таблицы данных:

Пример 9

Необходимо выбрать товары со скидкой «10–15%».

```
USE tempdb;
GO
IF EXISTS(
    SELECT TABLE_NAME
    FROM INFORMATION_SCHEMA.TABLES
    WHERE TABLE_NAME = 'mytbl2')
    DROP TABLE mytbl2;
GO
USE tempdb;
GO
CREATE TABLE mytbl2
(
    c1 sysname
);
GO
INSERT mytbl2 VALUES ('Discount is 10-15% off'), ('Discount is
.10-.15 off');
GO
```

Решение 1:

```
SELECT c1
FROM mytbl2
WHERE c1 LIKE '%10-15!% off%' ESCAPE '!';
```

⁶ <https://learn.microsoft.com/en-us/sql/t-sql/language-elements/like-transact-sql?source=recommendations&view=sql-server-ver16#c-using-the-escape-clause>.

Последняя выделенная строка в тексте запроса означает, что после символа «!» символ «%» не является специальным символом. Результат выполнения данного скрипта представлен на рисунке 11.

Results		Messages
	c1	
1	Discount is 10-15% off	

Рис 11. Результат выполнения скрипта (пример 9)

Такого же результата можно добиться, переписав текст запроса на выборку:

Решение 2:

```
SELECT c1
FROM mytbl2
WHERE c1 LIKE '%10-15[%] off%';
```

Предикат IN

Если необходимо выбрать записи, значения которых принадлежат некоторому множеству, то запрос выглядит следующим образом:

```
SELECT
    Field1
    , ...
FROM TargetTable
WHERE Field1 = <Значение1>
    OR Field1 = <Значение2>
    OR ...
    OR Field1 = <ЗначениеN>;
```

последнее предложение можно заменить, используя предикат IN:

```
WHERE Field1 IN (<Значение1>, <Значение2>, ..., <ЗначениеN>);
```

Пример 10

В таблице `Production.Product` найти товары, идентификационные номера которых равны 1, 316 или 329.

Решение 1:

```
USE AdventureWorks2019;  
SELECT ProductId, Name, Color, Size  
FROM Production.Product  
WHERE ProductID = 1 OR ProductId = 316 OR ProductID = 329;
```

Решение 2. Использование предиката IN:

```
USE AdventureWorks2019;  
SELECT ProductId, Name, Color, Size  
FROM Production.Product  
WHERE ProductID IN (1, 316, 329);
```

При выполнении и решения 1, и решения 2 получим одинаковые результаты (рис. 12).

Results		Messages		
	ProductId	Name	Color	Size
1	1	Adjustable Race	NULL	NULL
2	316	Blade	NULL	NULL
3	329	Road End Caps	NULL	NULL

Рис. 12. Результат выполнения запроса (пример 10)

Пример 11

Показать код и название валюты (`CurrencyCode` и `Name`) из таблицы `Sales.Currency`. Показать только валюты из списка ('Kroon', 'Armenian Dram', 'US Dollar').

```
USE AdventureWorks2019;
SELECT
    CurrencyCode
    , [Name]
FROM Sales.Currency
WHERE [Name] IN ('Kroon', 'Armenian Dram', 'US Dollar');
```

Результат выполнения запроса представлен на рисунке 13.

Results		Messages
	CurrencyCode	Name
1	AMD	Armenian Dram
2	EEK	Kroon
3	USD	US Dollar

Рис. 13. Результат выполнения скрипта (пример 11)

Предикат BETWEEN

Если требуется выбрать записи, входящие в некоторый диапазон значений, то запрос выглядит следующим образом:

```
SELECT Field1, ...
FROM TargetTable
WHERE Field1 >= <Значение1>
    AND Field1 <= <Значение2>;
```

последнее предложение можно заменить, используя предикат BETWEEN :

```
WHERE Field1 BETWEEN <Значение1> AND <Значение2>;
```

Пример 12

В таблице Production.Product найти товары, дата продажи которых находится в диапазоне от 2002-06-01 до 2007-06-01.

Решение 1:

```
SELECT ProductId, Name, Color, Size, SellEndDate
FROM Production.Product
WHERE SellEndDate >= '2012-05-01'
AND SellEndDate <= '2013-04-01';
```

Решение 2. Использование предиката BETWEEN:

```
SELECT ProductId, Name, Color, Size, SellEndDate
FROM Production.Product
WHERE SellEndDate BETWEEN '2012-05-01' AND '2013-04-01';
```

Результат выполнения скриптов приведен на рисунке 14.



	ProductId	Name	Color	Size	SellEndDate
1	709	Mountain Bike Socks, M	White	M	2012-05-29 00:00:00.000
2	710	Mountain Bike Socks, L	White	L	2012-05-29 00:00:00.000
3	731	ML Road Frame - Red, 44	Red	44	2012-05-29 00:00:00.000
4	732	ML Road Frame - Red, 48	Red	48	2012-05-29 00:00:00.000
5	733	ML Road Frame - Red, 52	Red	52	2012-05-29 00:00:00.000
6	734	ML Road Frame - Red, 58	Red	58	2012-05-29 00:00:00.000
7	735	ML Road Frame - Red, 60	Red	60	2012-05-29 00:00:00.000
8	740	HL Mountain Frame - Silver, 44	Silver	44	2012-05-29 00:00:00.000
9	741	HL Mountain Frame - Silver, 48	Silver	48	2012-05-29 00:00:00.000

Рис. 14. Результат использования предиката BETWEEN

Определение переменных

MS SQL Server поддерживает язык программирования Transact-SQL (T-SQL).

Определение переменных некоторых типов

Определение переменных начинается с инструкции DECLARE, после следует имя переменной, которая начинается с символа @⁷. Затем записывается

⁷ Имена локальных переменных в MS SQL Server всегда начинаются с символа @.

тип данных. Установить значение можно сразу, используя оператор присвоения (=) или оператор SET. Посмотреть содержимое переменной можно, применяя операторы SELECT или PRINT.

Пример объявления переменных целого типа:

```
DECLARE @I1 INT = 10;  
DECLARE @I2 INT;  
SET @I2 = 20;  
SELECT @I1, @I2
```

Для объявления переменных символьного типа используются типы CHAR (max⁸), VARCHAR (max) и NVARCHAR (max).

Пример объявления переменных символьного типа CHAR (10).

```
DECLARE @S1 CHAR(10) = '10';  
PRINT @S1;  
DECLARE @S2 CHAR(10) = '20';  
PRINT @S1 + @S2;
```

Результат выполнения данного запроса представлен на рисунке 15.

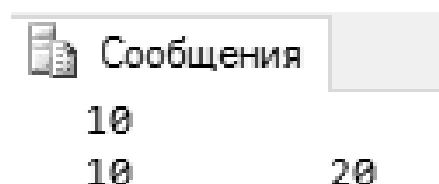


Рис. 15. Использование символьного типа данных CHAR (max)

На рисунке 15 показано, что на каждую переменную отведено 10 символов. Те символы, которые не заняты информацией, всё равно занимают количество символов, которое было выделено при инициализации переменной.

Для экономии места⁹, отводимого на хранение значения, используют символьный тип данных VARCHAR (max).

⁸ Максимальное количество символов.

```

DECLARE @S3 VARCHAR(10) = '10';
DECLARE @S4 VARCHAR(10) = '20';
PRINT @S3 + @S4;

```



**Рис. 16. Использование
символьного типа данных VARCHAR (max)**

Как видно из рисунка 16, размер «подстроился» под содержимое переменной (переменная занимает 4 символа, нет пробелов).

Перечисленные выше символьные переменные не поддерживают символы Unicode. Если необходимо хранить данные в символах Unicode, то используйте типы данных NCHAR (max) или NVARCHAR (max). Для правильного отображения символьных данных перед значением необходимо поставить символ «N».

```

DECLARE @S5 NVARCHAR(10) = N'Привет!';
PRINT @S5;

```

Результат выполнения данного скрипта представлен на рисунке 17.

⁹ CHAR (10) занимает 10 бит, VARCHAR (10) — 12 бит, из которых 10 бит на информацию, **2 бита — служебные**, которые будут использоваться при любой максимально отведенной длине на информацию. Если ваши данные будут всегда полностью заполнены, или максимальная длина не превышает **4 символов, то используйте тип CHAR.**

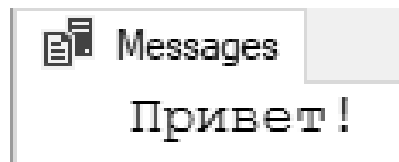


Рис. 17. Пример правильного задания значения символьной переменной

Если вы выполните тот же скрипт, но не поставите символ «N», то в зависимости от настроек среды результат может быть и правильным, и неверным. Если у SSMS настройка Server Collation кириллическая (рис. 18), то отображение кириллицы будет правильным, даже в случае отсутствия символа «N». Однако это «правильное» отображение будет только на вашем компьютере.

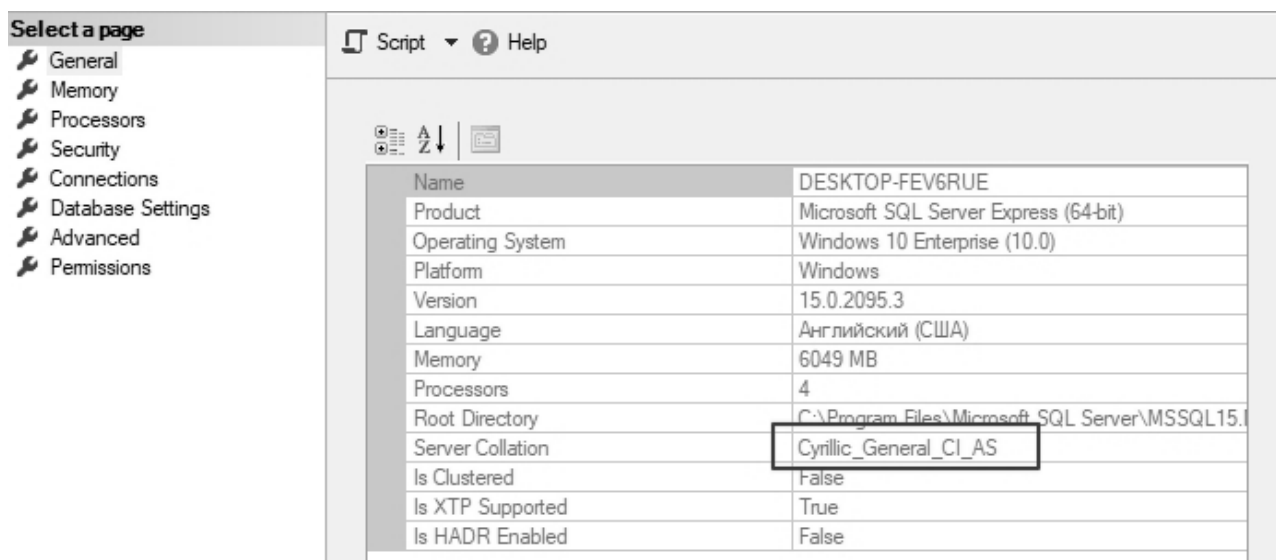


Рис. 18. Настройка сервера SSMS

Для того чтобы доказать необходимость установки символа «N», установим значение для введенной переменной @S5 на другом языке (не кириллице), но также в символах Unicode.

Воспользуемся переводчиком, переведем слово «Привет!», например на китайский язык, и скопируем его, но не будем использовать символ «N».

```
DECLARE @S5 NVARCHAR(10) = N'你好!';  
PRINT @S5;
```

На рисунке 19 представлен результат выполнения данного скрипта. Что и требовалось доказать: символы отображаются некорректно.

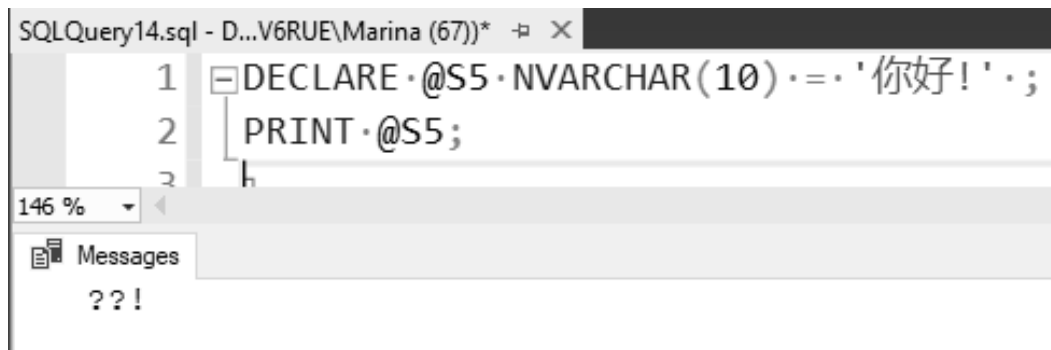


Рис. 19. Неправильное отображение символов Unicode

После добавления необходимого символа «N» и выполнения скрипта получаем правильный результат, представленный на рисунке 20.

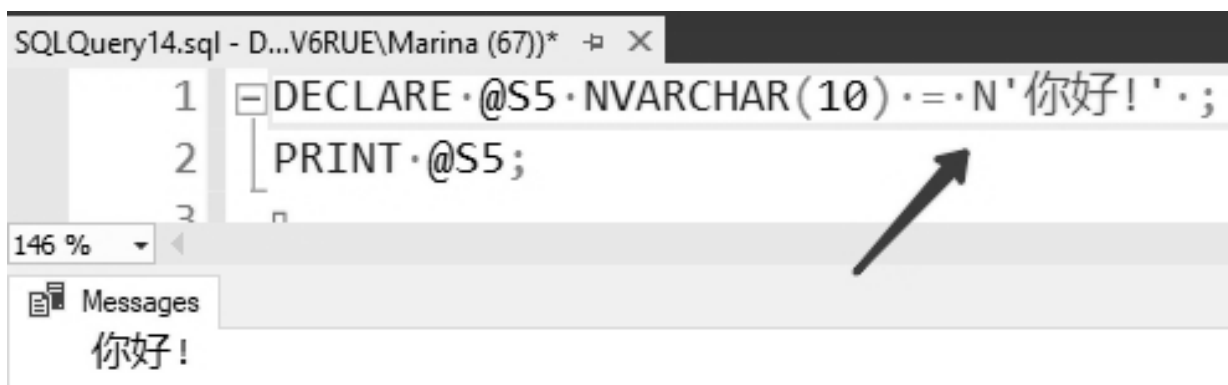


Рис 20. Правильное отображение символов Unicode

Сделаем вывод: при использовании символов Unicode обязательно (!) необходимо использовать символ «N».

Объявим переменную типа **дата** и зададим ей значение текущей даты:

```
DECLARE @T DATETIME;  
SET @T = GETDATE10();  
PRINT @T;
```

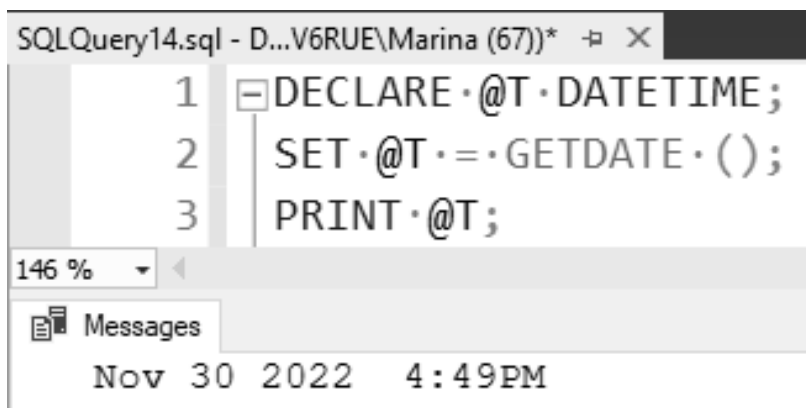


Рис. 21. Результат выполнения скрипта

Преобразование типов данных

Неявное преобразование типов

Т-SQL поддерживает явное и неявное преобразование типов данных. Тип данных **целое число**, **число с плавающей точкой** и тип **дата** можно неявно преобразовать к символьному типу.

```
DECLARE @I1 INT = 10;  
DECLARE @I2 INT;  
SET @I2 = 20;  
SELECT @I1 AS I1, @I2 AS I2
```

```
DECLARE @CHAR1 CHAR(2) = @I1;  
SELECT @CHAR1 AS CHAR1;
```

¹⁰ Функция возвращает текущее значение даты и времени.

Результаты		
	I1	I2
1	10	20

	CHAR1
1	10

Рис. 22. Неявное преобразование типов (INT в CHAR)

Символьный тип данных также может быть неявно преобразован к числовому типу данных, если значение содержит допустимые для преобразуемого типа символы (только цифры для целого типа данных и знак¹¹ отделения целой части от дробной — для числа с плавающей точкой).

```
DECLARE @INT1 INT = @S3;
DECLARE @INT2 INT = @S4;
SELECT @INT1 + @INT2 AS 'INT = ';
```

Результаты	
	INT =
1	30

Рис. 23. Неявное преобразование типов (CHAR в INT)

Если значение @S3 или @S4 из предыдущего примера будет содержать нецифровые символы, то при попытке выполнить данный запрос будет выведено сообщение об ошибке преобразования.

Явное преобразование типов

Для явного преобразования типов используются функции CONVERT() и CAST(). При невозможности преобразования будет выведено сообщение об ошибке.

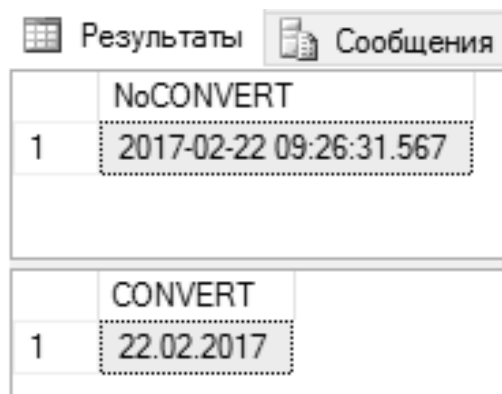
¹¹ Точка или запятая. Зависит от языковой настройки операционной системы.

CONVERT(тип данных, в который нужно преобразовать, преобразуемое выражение, стиль).

Для типа данных DATETIME значение типа позволяет выводить дату в определенном формате.

```
DECLARE @T DATETIME;  
SET @T = GETDATE();  
SELECT @T AS NoCONVERT;
```

```
DECLARE @S3 VARCHAR(10);  
SET @S3 = CONVERT(VARCHAR(10), @T, 10412);  
SELECT @S3 AS [CONVERT];
```



	NoCONVERT
1	2017-02-22 09:26:31.567

	CONVERT
1	22.02.2017

Рис. 24. Применение функции CONVERT () к типу DATETIME

Для типа данных FLOAT (REAL) или MONEY (SMALLMONEY) стиль позволяет показывать определенное количество разрядов (по умолчанию значение 0, 1 или 2 — всегда экспоненциальное представление чисел, фиксируется количество разрядов).

```
DECLARE @F FLOAT = 0.123456789;  
DECLARE @S3 VARCHAR(20);  
SET @S3 = CONVERT(VARCHAR(20), @F, 1);  
SELECT @F AS [FLOAT], @S3 AS [CONVERT];
```

¹² Принятый на территории РФ формат даты.

Результаты		Сообщения
	FLOAT	CONVERT
1	0,123456789	1.2345679e-001

Рис. 25. Применение функции CONVERT () к типу FLOAT

CAST (преобразуемое выражение AS, тип данных, в который нужно преобразовать) ;

```
DECLARE @S1 CHAR(10) = '10';
DECLARE @INT1 INT;
SET @INT1 = CAST(@S1 AS INT);
PRINT @INT1;
```

В результате выполнения данного сценария переменная символьного типа будет преобразована к типу целое число.

Работа со значениями NULL

СУБД поддерживают значение пусто (NULL) для полей, значение которых неизвестно (UNKNOWN). К таким значениям применяется трехзначная логика (табл. 1–3).

Таблица 1

AND (A, B)		A		
		TRUE	FALSE	UNKNOWN
B	TRUE	TRUE	FALSE	UNKNOWN
	FALSE	FALSE	FALSE	UNKNOWN
	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Таблица 2

OR (A, B)		A		
		TRUE	FALSE	UNKNOWN
B	TRUE	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	TRUE	UNKNOWN	UNKNOWN

Таблица 3

NOT (A)	A		
	TRUE	FALSE	UNKNOWN
	FALSE	TRUE	UNKNOWN

При необходимости показывать только известные значения полей в предложении WHERE применяется предикат IS NOT NULL (и наоборот, для выборки всех неизвестных значений применяется предикат IS NULL).

```

SELECT
    Field1
    , Field2
    , ...
FROM TargetTable
WHERE Field1 IS NOT NULL;

```

Пример 13

Выбрать наименование продукта и его цену, для которого известно значение в линейке продуктов (ProductLine):

```

SELECT
    [Name]

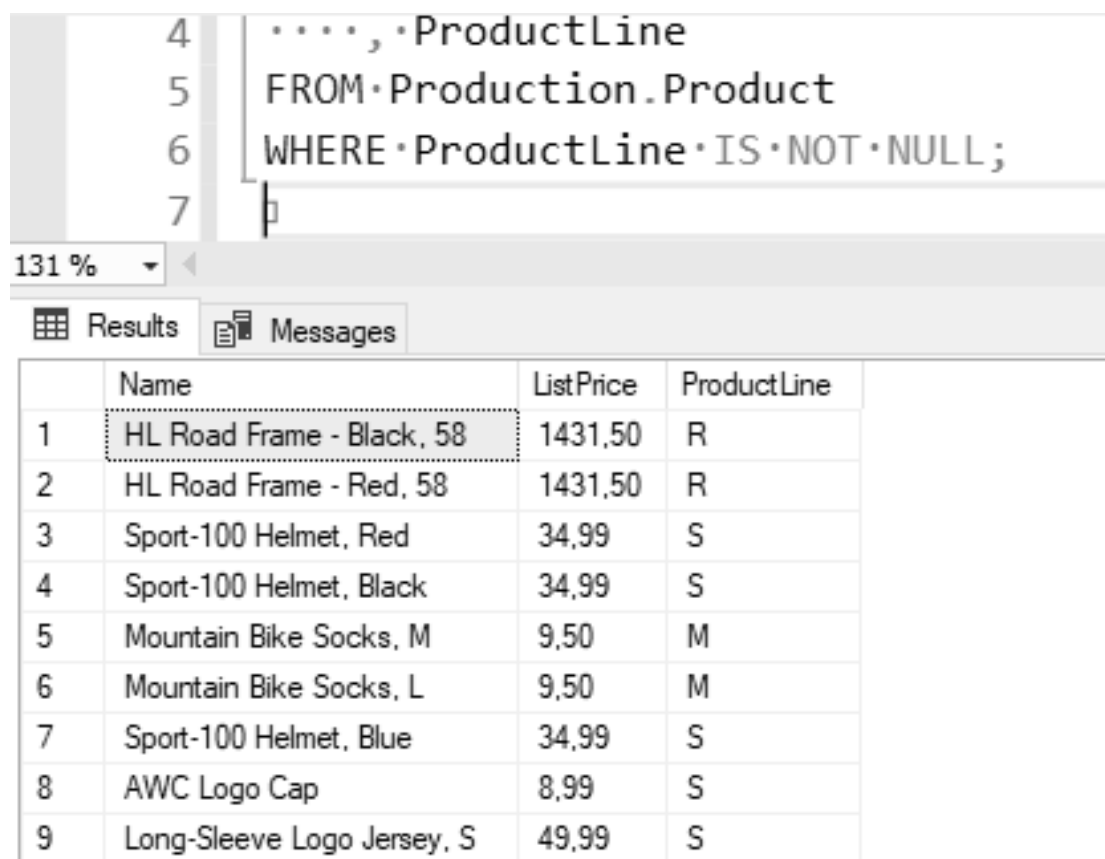
```

```

, ListPrice
, ProductLine
FROM Production.Product
WHERE ProductLine IS NOT NULL;

```

Результат выполнения скрипта представлен на рисунке 26.



```

4      ...., ProductLine
5      FROM Production.Product
6      WHERE ProductLine IS NOT NULL;
7

```

131 %

Results Messages

	Name	ListPrice	ProductLine
1	HL Road Frame - Black, 58	1431,50	R
2	HL Road Frame - Red, 58	1431,50	R
3	Sport-100 Helmet, Red	34,99	S
4	Sport-100 Helmet, Black	34,99	S
5	Mountain Bike Socks, M	9,50	M
6	Mountain Bike Socks, L	9,50	M
7	Sport-100 Helmet, Blue	34,99	S
8	AWC Logo Cap	8,99	S
9	Long-Sleeve Logo Jersey, S	49,99	S

Рис. 26. Фильтрация пустых значений в поле ProductLine

Для работы со значениями NULL в MS SQL Server предусмотрены несколько функций.

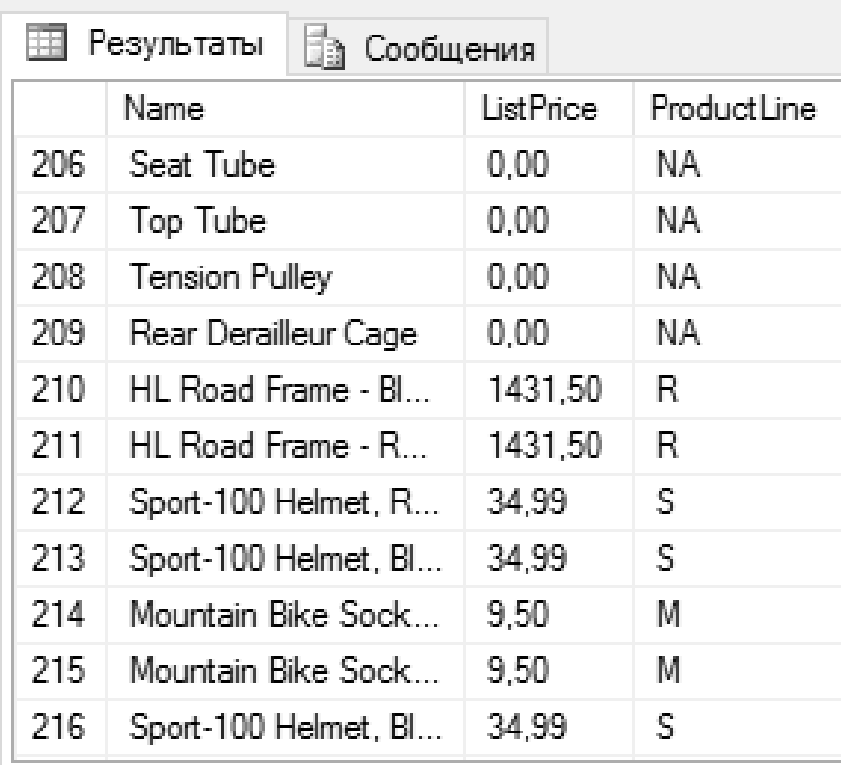
ISNULL(выражение, значение в случае, если выражение = NULL) .

Функция ISNULL() применяется в том случае, если значение NULL необходимо заменить другим значением.

Пример 14

Выбрать наименование продукта, его цену и значение в линейке продуктов (ProductLine). Если значение в линейке продуктов отсутствует, заменить его на «неизвестно» (NA).

```
SELECT  
    [Name]  
    , ListPrice  
    , ISNULL(ProductLine, 'NA') AS ProductLine  
FROM Production.Product;
```



	Name	ListPrice	ProductLine
206	Seat Tube	0,00	NA
207	Top Tube	0,00	NA
208	Tension Pulley	0,00	NA
209	Rear Derailleur Cage	0,00	NA
210	HL Road Frame - Bl...	1431,50	R
211	HL Road Frame - R...	1431,50	R
212	Sport-100 Helmet, R...	34,99	S
213	Sport-100 Helmet, Bl...	34,99	S
214	Mountain Bike Sock...	9,50	M
215	Mountain Bike Sock...	9,50	M
216	Sport-100 Helmet, Bl...	34,99	S

Рис. 27. Результат выполнения функции ISNULL ()

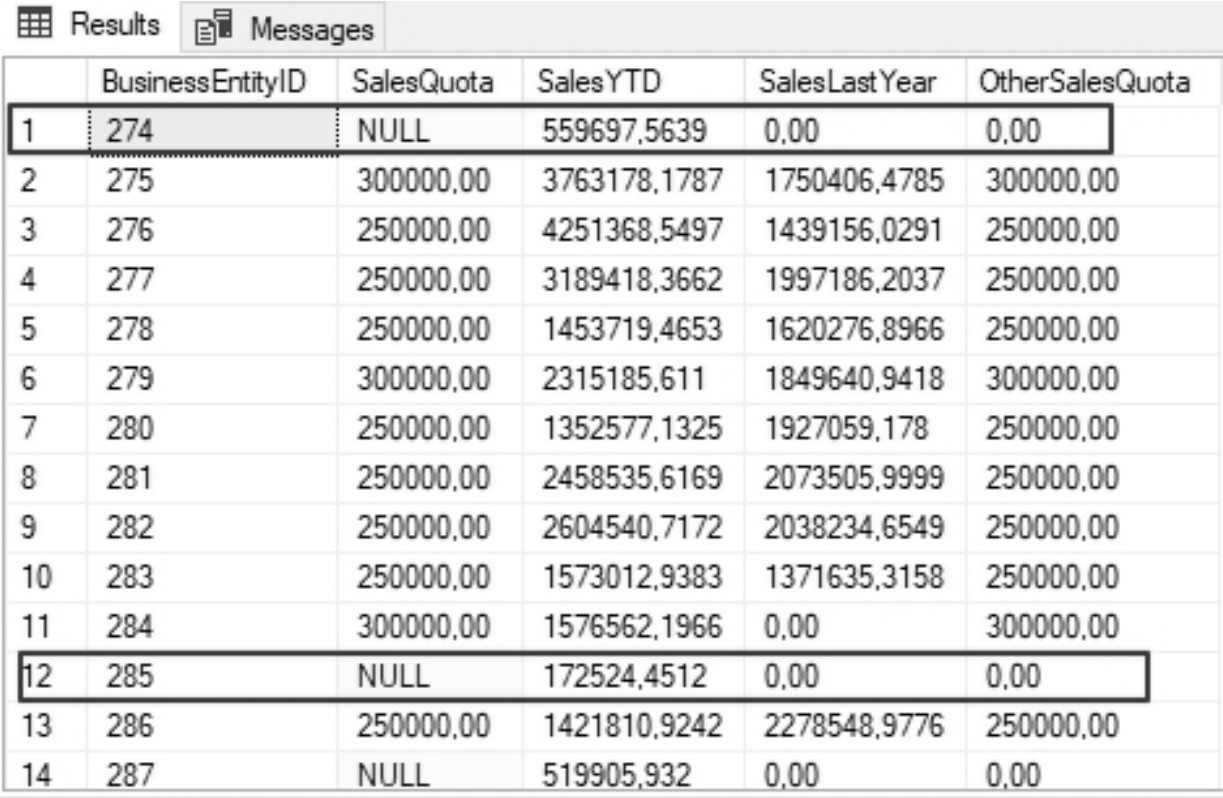
Пример 15

Показать ID сотрудников (BusinessEntityID), доли продаж (SalesQuota), продажи в этом году (SalesYTD) и продажи в прошлом году (SalesLastYear) из таблицы Sales.SalesPerson. Неизвестные доли

продаж заменить на 0 и вынести в отдельное поле с названием *OtherSalesQuota*.

```
USE AdventureWorks2019;
SELECT
    BusinessEntityID
    , SalesQuota
    , SalesYTD
    , SalesLastYear
    , ISNULL(SalesQuota, 0) AS OtherSalesQuota
FROM Sales.SalesPerson;
```

Результат выполнения данного скрипта представлен на рисунке 28. Как видно, в результирующем наборе данных значения поля *SalesQuota* в строках 1, 12, 14 содержат значение NULL, которое в поле *OtherSalesQuota* заменено на 0.



	BusinessEntityID	SalesQuota	SalesYTD	SalesLastYear	OtherSalesQuota
1	274	NULL	559697,5639	0,00	0,00
2	275	300000,00	3763178,1787	1750406,4785	300000,00
3	276	250000,00	4251368,5497	1439156,0291	250000,00
4	277	250000,00	3189418,3662	1997186,2037	250000,00
5	278	250000,00	1453719,4653	1620276,8966	250000,00
6	279	300000,00	2315185,611	1849640,9418	300000,00
7	280	250000,00	1352577,1325	1927059,178	250000,00
8	281	250000,00	2458535,6169	2073505,9999	250000,00
9	282	250000,00	2604540,7172	2038234,6549	250000,00
10	283	250000,00	1573012,9383	1371635,3158	250000,00
11	284	300000,00	1576562,1966	0,00	300000,00
12	285	NULL	172524,4512	0,00	0,00
13	286	250000,00	1421810,9242	2278548,9776	250000,00
14	287	NULL	519905,932	0,00	0,00

Рис. 28. Результат выполнения запроса (пример 15)

NULLIF (выражение1, выражение2)

Функция NULLIF () используется для сравнения двух выражений. В случае их равенства функция возвращает значение NULL, иначе — значение первого выражения.

Пример 16

Показать наименование продукта, его цену и значение в линейке продуктов (ProductLine). Показать только те значения, которые известны, а значения, которые равны «R», заменить на NULL.

```
SELECT
    [Name]
    , ListPrice
    , NULLIF(ProductLine, 'R') AS NullIfR
FROM Production.Product
WHERE ProductLine IS NOT NULL;
```

Результат выполнения данного скрипта представлен на рисунке 29. Из результирующего набора видно, что отфильтрованные данные в поле ProductLine содержат значение NULL (замененные значения «R»).

Результаты		Сообщения	
	Name	ListPrice	NullIfR
1	HL Road Frame - Black, 58	1431,50	NULL
2	HL Road Frame - Red, 58	1431,50	NULL
3	Sport-100 Helmet, Red	34,99	S
4	Sport-100 Helmet, Black	34,99	S

Рис. 29. Результат выполнения функции NULLIF ()

Пример 17

Из таблицы *Production.Document* показать поля *DocumentNode*, *Title* и *FileExtension*. Все пустые значения поля *FileExtension* заменить на *NULL*, а затем все *NULL* этого же поля заменить на *'.txt'*. Названия полей оставить без изменений.

```
SELECT
    DocumentNode
  , Title
  , ISNULL(NULLIF(FileExtension,''),'.txt') FileExtension
FROM Production.Document;
```

Результат выполнения данного скрипта представлен на рисунке 30.

	DocumentNode	Title	FileExtension
1	0x	Documents	.txt
2	0x58	Overview	.txt
3	0x5AC0	Introduction 1	.doc
4	0x5B40	Repair and Service Guidelines	.doc
5	0x68	Maintenance	.txt
6	0x6AC0	Crank Arm and Tire Maintenance	.doc
7	0x6B40	Lubrication Maintenance	.doc
8	0x78	Assembly	.txt
9	0x7AC0	Front Reflector Bracket and Reflector Assembly 3	.doc
10	0x7B40	Front Reflector Bracket Installation	.doc
11	0x7BC0	Installing Replacement Pedals	.doc
12	0x7C20	Seat Assembly	.doc
13	0x7C60	Training Wheels 2	.doc

Рис. 30. Результат выполнения запроса (пример 17)

COALESCE(выражение1, выражение2, ..., выражение по умолчанию)

Функция COALESCE() применяется в том случае, когда необходимо показать первое известное значение. Сначала проверяется выражение1, если его значение неизвестно, то проверяется значение2, и т. д. Значение по умолчанию будет выведено в том случае, если все значения в предыдущих выражениях были неизвестны. Все аргументы функции должны быть одного типа. Если это не так, то необходимо применить функции преобразования типов.

Пример 18

Показать наименование продукта, его цену и одно из известных значений из набора цвет или размер продукта. В случае если ни один из параметров продукта неизвестен, вывести значение «неизвестно» (UNKNOWN) в отдельное поле с названием «измерение» (Measurement). Показать товар с ценой, большей нуля.

```
USE AdventureWorks2019;
SELECT
    [Name]
    , ListPrice
    , COALESCE(Color, Size, 'UNKNOWN') AS Measurement
FROM Production.Product
WHERE ListPrice > 0;
```

Результат выполнения текста запроса представлен на рисунке 31. Из результирующего набора видно, что те значения, которые содержат NULL в полях, заменены на UNKNOWN.

Results		Messages	
	Name	ListPrice	Measurements
294	Mountain-500 Black, 40	539,99	Black
295	Mountain-500 Black, 42	539,99	Black
296	Mountain-500 Black, 44	539,99	Black
297	Mountain-500 Black, 48	539,99	Black
298	Mountain-500 Black, 52	539,99	Black
299	LL Bottom Bracket	53,99	UNKNOWN
300	ML Bottom Bracket	101,24	UNKNOWN
301	HL Bottom Bracket	121,49	UNKNOWN
302	Road-750 Black, 44	539,99	Black
303	Road-750 Black, 48	539,99	Black
304	Road-750 Black, 52	539,99	Black

Рис. 31. Результат выполнения запроса (пример 18)

Пример 19

Показать наименование продукта, его цену и одно из известных значений из набора цвет, или размер, или вес продукта. В случае если ни один из параметров продукта неизвестен, вывести значение «позвонить» (CALL). Показать товар с ценой больше нуля.

В отличие от предыдущего примера 18, необходимо значения в поле Weight привести к типу VARCHAR(MAX). Это можно сделать с помощью функции CAST([Weight] AS VARCHAR(10)).

```

SELECT
    [Name]
    , ListPrice
    , COALESCE(
        Color
        , Size
        , CAST([Weight] AS VARCHAR(10))
        , 'CALL') AS Parameter
FROM Production.Product
WHERE ListPrice > 0;

```

Результат выполнения данного скрипта представлен на рисунке 32.

Результаты		Сообщения	
	Name	ListPrice	Parameter
4	LL Road Seat Assembly	133,34	CALL
5	ML Road Seat Assembly	147,14	CALL
6	HL Road Seat Assembly	196,92	CALL
7	LL Touring Seat Assembly	133,34	CALL
8	ML Touring Seat Assembly	147,14	CALL
9	HL Touring Seat Assembly	196,92	CALL
10	HL Road Frame - Black, 58	1431,50	Black
11	HL Road Frame - Red, 58	1431,50	Red
12	Sport-100 Helmet, Red	34,99	Red
13	Sport-100 Helmet, Black	34,99	Black
14	Mountain Bike Socks, M	9,50	White

Рис. 32. Результат выполнения функции COALESCE ()

Предложение DISTINCT

Для выборки множества значений применяется предложение DISTINCT.

Сравним результаты выполнения двух запросов:

Показать значения столбца Field1 из таблицы TargetTable и уникальные значения столбца Field1 из таблицы TargetTable.

Таблица 4

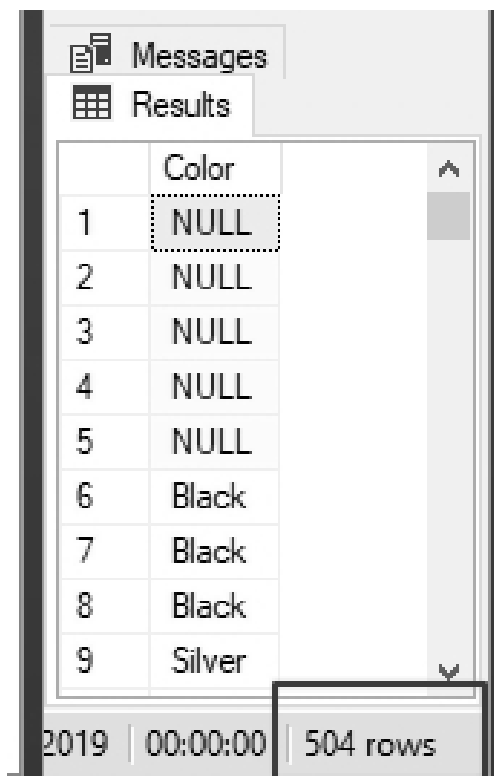
SELECT Field1 FROM TargetTable;	SELECT DISTINCT Field1 FROM TargetTable;
Результат выполнения запроса — все записи	Результат выполнения запроса — множе- ство значений

Пример 20

Показать значения столбца цвет.

```
SELECT Color  
FROM Production.Product;
```

В результате выполнения данного запроса выведено 504 строки (**все строки**) (рис. 33).



The screenshot shows a window with two tabs: 'Messages' and 'Results'. The 'Results' tab is active, displaying a table with two columns: an index and 'Color'. The first five rows have 'NULL' in the 'Color' column, rows 6-8 have 'Black', and row 9 has 'Silver'. A status bar at the bottom indicates '2019 | 00:00:00 | 504 rows'.

	Color
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	Black
7	Black
8	Black
9	Silver

2019 | 00:00:00 | 504 rows

Рис. 33. Все записи. Результат выполнения запроса

Пример 21

Показать все имеющиеся цвета товара.

```
SELECT DISTINCT Color  
FROM Production.Product
```

В результате показано 10 строк (рис. 34).

Messages	
Results	
	Color
1	NULL
2	Black
3	Blue
4	Grey
5	Multi
6	Red
7	Silver
8	Silver/Black
9	White
10	Yellow

2019 | 00:00:00 | 10 rows

Рис. 34. Результат применения предложения **DISTINCT**

Сортировка результирующего набора данных

Если необходимо результат выполнения запроса отсортировать по какому-либо полю, то применяется предложение **ORDER BY**. По умолчанию (ASC), сортировка происходит по возрастанию значений в указанном поле.

```
SELECT
    Field1
    , Field2
    , ...
    ,FieldN
FROM TargetTable
ORDER BY Field1;
```

В случае обратного порядка применяется указатель DESC.

```
SELECT
    Field1
  , Field2
  , ...
  , FieldN
FROM TargetTable
ORDER BY Field1 DESC;
```

Не всегда известно имя поля (например, в случае применения одной из вышеперечисленных функций работы со значениями NULL столбец не имеет имени). В этом случае можно не указывать имя поля, а указать его номер в операторе SELECT:

```
SELECT
    Field1
  , Field2
  , ...
  , FieldN
FROM TargetTable
ORDER BY 1;
```

Пример 22

*Показать наименование товара, его цену и цвет. Отсортировать по **возрастанию** цены (показать только товар со значением цены, отличной от нуля)*

```
SELECT
    [Name]
  , ListPrice
  , Color
FROM Production.Product
WHERE ListPrice > 0
ORDER BY ListPrice;
```

	Name	ListPrice	Color
1	Patch Kit/8 Patches	2,29	NULL
2	Road Tire Tube	3,99	NULL
3	Touring Tire Tube	4,99	NULL
4	Mountain Tire Tube	4,99	NULL
5	Water Bottle - 30 oz.	4,99	NULL
6	Bike Wash - Dissolver	7,95	NULL
7	Road Bottle Cage	8,99	NULL
8	Racing Socks, M	8,99	White
9	Racing Socks, L	8,99	White
10	AWC Logo Cap	8,99	Multi
11	Mountain Bike Socks, M	9,50	White

**Рис. 35. Результат запроса
с применением предиката сортировки**

Пример 23

Показать наименование товара, его цену и цвет. Отсортировать по убыванию цены (показать только товар, со значением цены, отличной от нуля)

```
SELECT
    [Name]
    , ListPrice
    , Color
FROM Production.Product
WHERE ListPrice > 0
ORDER BY ListPrice DESC;
```

Результат выполнения скрипта представлен на рисунке 36.

Results		Messages	
	Name	ListPrice	Color
1	Road-150 Red, 62	3578,27	Red
2	Road-150 Red, 44	3578,27	Red
3	Road-150 Red, 48	3578,27	Red
4	Road-150 Red, 52	3578,27	Red
5	Road-150 Red, 56	3578,27	Red
6	Mountain-100 Silver, 38	3399,99	Silver
7	Mountain-100 Silver, 42	3399,99	Silver
8	Mountain-100 Silver, 44	3399,99	Silver
9	Mountain-100 Silver, 48	3399,99	Silver
10	Mountain-100 Black, 38	3374,99	Black
11	Mountain-100 Black, 42	3374,99	Black
12	Mountain-100 Black, 44	3374,99	Black
13	Mountain-100 Black, 48	3374,99	Black

Рис. 36. Результат выполнения запроса (пример 23)

АГРЕГАТНЫЕ ФУНКЦИИ MS SQL SERVER

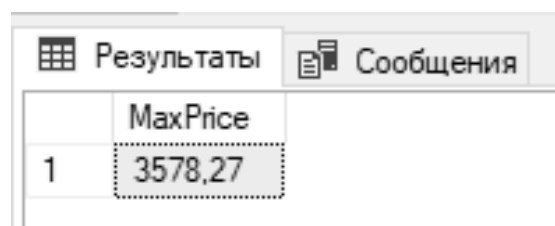
К агрегатным функциям относятся функции поиска минимального (MIN) и максимального (MAX) значений, вычисления среднего (AVG) и суммарного (SUM) значений, нахождения количества элемента (COUNT).

- ! Агрегатная функция выбирается в предложении *SELECT*.
В данном предложении возможно различное сочетание **только**
- агрегатных функций.

Пример 24

Найти максимальную цену товара.

```
SELECT MAX(ListPrice) AS MaxPrice  
FROM Production.Product;
```



The screenshot shows a SQL Server query results window with two tabs: 'Результаты' (Results) and 'Сообщения' (Messages). The 'Результаты' tab is active, displaying a single row of data. The column header is 'MaxPrice' and the value is '3578,27'.

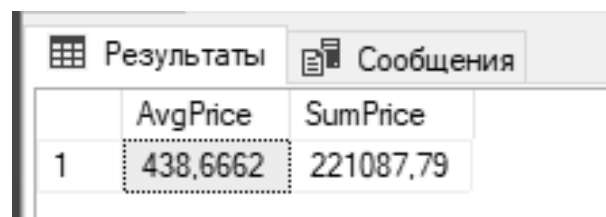
	MaxPrice
1	3578,27

Рис. 37. Результат выполнения запроса (пример 24)

Пример 25

Найти среднюю и суммарную цену всех товаров.

```
SELECT AVG(ListPrice) AS AvgPrice  
      , SUM(ListPrice) AS SumPrice  
FROM Production.Product;
```



The screenshot shows a SQL Server query results window with two tabs: 'Результаты' (Results) and 'Сообщения' (Messages). The 'Результаты' tab is active, displaying a single row of data. The column headers are 'AvgPrice' and 'SumPrice'. The values are '438,6662' and '221087,79' respectively.

	AvgPrice	SumPrice
1	438,6662	221087,79

Рис. 38. Результат выполнения запроса (пример 25)

Ограничение на тип входных данных — численное значение — имеется для функций расчета средней (AVG) и суммарной (SUM) величины. Для остальных функций можно использовать любые типы данных.

Для фильтрации исходных данных, к которым не будут применены агрегатные функции, используется предложение WHERE. Порядок выполнения в данном случае такой: сначала выполняется предложение FROM, затем фильтруются данные в предложении WHERE, и только в конце выполняются агрегатные функции из предложения SELECT.

Пример 26

Найти минимальную цену товара. При этом исключить из рассмотрения товар с ценой, равной нулю.

```
SELECT MIN(ListPrice) AS MinPrice  
FROM Production.Product  
WHERE ListPrice > 0;
```



Результаты	
	MinPrice
1	2.29

Рис. 39. Результат выполнения запроса (пример 26)

Пример 27

Показать количество разнообразных цветов товара (ассортимент цветов).

```
SELECT COUNT(DISTINCT Color) AS CountColors  
FROM Production.Product;
```

Результаты	
	CountColors
1	9

Рис. 40. Результат выполнения запроса (пример 27)

Пример 28

Показать возраст самого пожилого сотрудника.

Для выполнения данного запроса можно воспользоваться функцией `DATEDIFF(Interval, StartDate, EndDate)`, которая возвращает интервал в днях, месяцах или годах соответственно (`DAY`, `MONTH`, `YEAR`) между начальной и конечной датами. Функция `GETDATE()` возвращает текущую дату.

```
SELECT DATEDIFF(YEAR, MIN (BirthDate),GETDATE()) AS Oldest
FROM HumanResources.Employee;
```

Результаты	
	Oldest
1	79

Рис. 41. Результат выполнения запроса (пример 28)

Пример 29

Найти среднюю цену товара черного цвета.

```
SELECT AVG (ListPrice) AS AvgPrice
FROM Production.Product
WHERE ListPrice > 0 AND Color LIKE 'Black';
```

Результаты	
	AvgPrice
1	757.7107

Рис. 42. Результат выполнения запроса (пример 29)



В предложении WHERE нельзя использовать фильтрацию (ограничение) на значения агрегатных функций. Если необходимо ограничить значения агрегатных функций, используется предложение HAVING, о котором будет рассказано ниже.

Использование агрегатных функций со значениями NULL

Большинство агрегатных функций игнорируют значения NULL. Значения NULL могут приводить к неожиданным или некорректным результатам. Чтобы избежать ошибок, используйте функцию ISNULL() к тем полям таблицы, которые могут иметь значения NULL.

Пример 30

Найти средний вес товара (не учитывать пустые значения). Для решения данного примера можно использовать два подхода.

Решение 1. Использовать функцию ISNULL() для преобразования значений в нули. Подразумевается, что товаров с нулевым весом (Weight = 0) в таблице Production.Product нет.

```
SELECT AVG(ISNULL([Weight], 0)) AS AvgWeight
FROM Production.Product
WHERE [Weight] > 0;
```

Решение 2. Использовать ограничение IS NOT NULL.

```
SELECT AVG([Weight]) AS AvgWeight
FROM Production.Product
WHERE [Weight] IS NOT NULL;
```

Результаты	
	AvgWeight
1	74.069219

Рис. 43. Результат выполнения запроса (пример 30)

Для подсчета количества строк в таблице используется функция `COUNT (*)`. Для подсчета количества строк с игнорированием значений `NULL` используется функция `COUNT (Field)`.

Пример 31

Посчитать количество наименований товара.

```
SELECT COUNT(*) AS [Product Count]
FROM Production.Product;
```

Результаты	
	Product Count
1	504

Рис. 44. Результат выполнения запроса (пример 31)

Пример 32

Посчитать количество товара, имеющего атрибут «цвет».

Решение 1. Использовать функцию `COUNT (Color)` (игнорирует значения `NULL`).

```
SELECT COUNT(Color) AS [Product Count]
FROM Production.Product;
```

Решение 2. Использовать функцию `COUNT (*)`, затем в предложении `WHERE` отфильтровать значения `NULL` в поле `Color`.

```
SELECT COUNT(*) AS [Product Count]
FROM Production.Product
WHERE Color IS NOT NULL;
```



	Product Count
1	256

Рис. 45. Результат выполнения запроса (пример 32)

Итоги для сгруппированных данных

Использование предложения GROUP BY

Предложение GROUP BY используется для группировки строк в результирующий набор. Если нам необходимо применить агрегатную функцию к определенному набору данных, то в предложение SELECT необходимо включить поле, по которому происходит группировка данных. В прошлом разделе было сказано, что при использовании агрегатных функций в предложении SELECT не должно быть ничего кроме агрегатных функций. Если мы указываем поле, по которому будет происходить группировка, необходимо предложение GROUP BY, которое должно располагаться до предложения ORDER BY (если таковое имеется).

```
SELECT агрегатная_функция(Field1)
      , FieldN13
FROM TargetTable
WHERE Field2 предикат1 --необязательное предложение
GROUP BY FieldN
ORDER BY; --необязательное предложение
```

Порядок выполнения операторов следующий: из таблицы TargetTable выбираем данные из поля Field2, подходящие по критерию отбора предикат1.

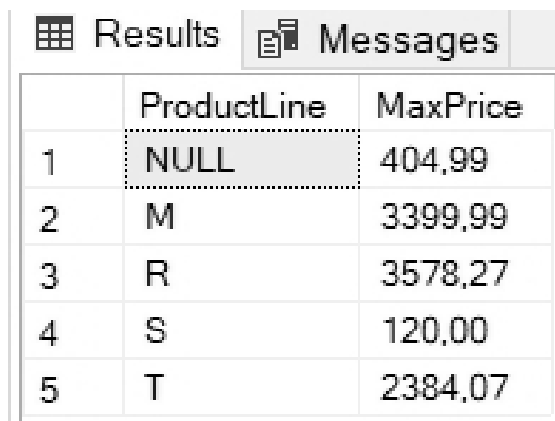
¹³ FieldN — столбец выражение (column-expression) либо является столбцом таблицы, либо является вычисляемым столбцом, где вычисления не являются агрегатными функциями.

кат1. Далее, отфильтрованные данные распределяются по всем значениям FieldN — формируются группы. К каждой полученной группе будет применена агрегатная_функция(Field1). Результат выполнения будет возвращен в виде набора данных в предложении SELECT.

Пример 33

Показать максимальную цену товара в каждой линейке продуктов.

```
SELECT
    ProductLine
    , MAX(ListPrice) As MaxPrice
FROM Production.Product
GROUP BY ProductLine;
```



	ProductLine	MaxPrice
1	NULL	404,99
2	M	3399,99
3	R	3578,27
4	S	120,00
5	T	2384,07

Рис. 46. Результат выполнения запроса (пример 33)

Пример 34

Изменить запрос так, чтобы продукты, у которых не определена линейка продуктов, не участвовали в отборе.

```
SELECT MAX(ListPrice) As MaxPrice
    , ProductLine
FROM Production.Product
WHERE ProductLine IS NOT NULL
GROUP BY ProductLine;
```

Результаты		Сообщения
	MaxPrice	ProductLine
1	3399,99	M
2	3578,27	R
3	120,00	S
4	2384,07	T

Рис. 47. Результат выполнения запроса (пример 34)

Ранее (пример 34) в предложении WHERE были отфильтрованы все значения NULL. В результате в результирующем наборе (рис. 47) отсутствует строка со значением NULL в столбце ProductLine и рассчитанная для этой группы максимальная цена.

Фильтрация сгруппированных данных, используя предложение HAVING

Предложение HAVING является опциональным и используется в том случае, когда необходимо сузить результирующий набор с помощью установки критерия на агрегатное значение. В данном предложении фильтрации подвергаются строки *после* примененной агрегации, но *перед* возвращаемым результатом. В данное предложение могут быть включены только агрегатные функции и поля, по которым происходит группировка.

```
SELECT агрегатная_функция(Field1)
      , FieldN
FROM TargetTable
WHERE Field2 предикат1 ... --необязательное предложение
GROUP BY FieldN
HAVING агрегатная_функция(Field1) предикат2
ORDER BY; --необязательное предложение
```

Пример 35

Показать максимальную цену товара в каждой линейке продуктов, значение которой не превосходит значение \$3000.

```
SELECT
    MAX(ListPrice) AS MaxPrice
    , ProductLine
FROM Production.Product
WHERE ProductLine IS NOT NULL
GROUP BY ProductLine
HAVING MAX(ListPrice) <= 3000;
```

Результаты		Сообщения
	MaxPrice	ProductLine
1	120,00	S
2	2384,07	T

Рис. 48. Применение ограничения на максимальную цену товара.

Результат выполнения запроса (пример 35)

Как видно из сравнения результатов выполнения (пример 34) и (пример 35) на рисунках (рис. 47 и 48), товар, максимальная цена которого оказалась выше \$3000, не вошёл в результирующий набор.

Изучение операторов ROLLUP и CUBE

Операторы ROLLUP и CUBE генерируют суммарную информацию в запросе.

ROLLUP выдает результирующий набор, показывая статистические вычисления для иерархии значений выбранных полей [14].

```

SELECT Field1
      , Field2
      , Field3
      , SUM(<expression>)
FROM TargetTable
GROUP BY ROLLUP (Field1, Field2, Field3);

```

Оператор ROLLUP создает группу для каждой комбинации столбцов выражений. В дополнении оператор сводит результаты в промежуточные подытоги и общие итоги. Прodelывая это, оператор ROLLUP двигается справа налево, уменьшая количество столбцов выражений, по которым он создает группы и статистические вычисления.

Порядок столбцов вносит эффект на результат выполнения оператора ROLLUP и может повлиять на количество строк в результирующем наборе.

Например, GROUP BY ROLLUP (col1, col2, col3, col4) создает группы для каждой комбинации в столбцах выражений в следующих списках [14]:

```

col1,col2,col3,col4
col1,col2,col3,null
col1,col2,null,null
col1,null,null,null
null,null,null,null — Это общий итог

```

Сравним результаты применения операторов GROUP BY (Field1, Field2) и GROUP BY ROLLUP (Field1, Field2) на следующем примере:

Пример 36

Рассчитать максимальную цену товара в каждой линейке товара и для каждого цвета.

```

SELECT
    ProductLine
    ,Color
    ,MAX(ListPrice) AS MaxPrice
FROM Production.Product
WHERE
    Color IS NOT NULL
    AND ProductLine IS NOT NULL
GROUP BY Color,ProductLine;

```

	Results	Messages	
	ProductLine	Color	MaxPrice
5	R	Black	2443,35
6	R	Red	3578,27
7	R	Silver/Black	80,99
8	R	White	8,99
9	R	Yellow	1700,99
10	S	Black	74,99
11	S	Blue	63,50
12	S	Multi	89,99
13	S	Red	34,99
14	S	Silver	54,99
15	S	Yellow	53,99
16	T	Black	245,01
17	T	Blue	2384,07
18	T	Grey	125,00
19	T	Silver/Black	80,99
20	T	Yellow	2384,07

Рис. 49. Результат выполнения запроса (пример 36)

Пример 37

Рассчитать максимальную цену товара в каждой линейке товара и для каждого цвета, а также подытоги для каждой получившейся группы.

```

SELECT
    ProductLine
    , Color
    , MAX(ListPrice) AS MaxPrice
FROM Production.Product
WHERE
    Color IS NOT NULL
    AND ProductLine IS NOT NULL
GROUP BY ROLLUP(Color,ProductLine);

```

Результатом запроса являются те же статистические вычисления, что и в простом предложении GROUP BY без ROLLUP. Кроме того, здесь создаются промежуточные итоги для каждого значения в столбце ProductLine (значения NULL в строках 15, 18, 22, 25, 29). Наконец, выводится общий итог для всех строк (значения NULL в столбцах ProductLine и Color). Результат имеет следующий вид (рис. 50).

Results Messages			
	ProductLine	Color	MaxPrice
15	NULL	Red	3578,27
16	M	Silver	3399,99
17	S	Silver	54,99
18	NULL	Silver	3399,99
19	M	Silv...	80,99
20	R	Silv...	80,99
21	T	Silv...	80,99
22	NULL	Silv...	80,99
23	M	White	9,50
24	R	White	8,99
25	NULL	White	9,50
26	R	Yell...	1700,99
27	S	Yell...	53,99
28	T	Yell...	2384,07
29	NULL	Yell...	2384,07
30	NULL	NU...	3578,27

Рис. 50. Результат выполнения запроса (пример 37)

CUBE создает группы для всех возможных сочетаний столбцов.

```
SELECT Field1
      , Field2
      , Field3
      , SUM(<expression>)
FROM TargetTable
GROUP BY CUBE (Field1, Field2, Field3);
```

Для GROUP BY CUBE (a, b) результатами являются группы для уникальных значений:

```
a, b
NULL, b
a, NULL
NULL, NULL — Это общий итог
```

Сравним результаты применения операторов GROUP BY ROLLUP (Field1, Field2) и GROUP BY CUBE (Field1, Field2) на следующем примере.

Пример 38

Показать итоговую сумму для каждого цвета товара в каждой линейке продуктов, используя функцию CUBE.

```
SELECT
      Color
      , ProductLine
      , SUM(ListPrice) AS [sum]
FROM Production.Product
WHERE
      Color IS NOT NULL
      AND ProductLine IS NOT NULL
GROUP BY CUBE(Color, ProductLine);
```

Результаты		Сообщения	
	Color	ProductLine	sum
1	Black	M	38341,74
2	Silver	M	36061,95
3	Silver/Black	M	183,57
4	White	M	19,00
5	NULL	M	74606,26
6	Black	R	27221,14
7	Red	R	53239,11
8	Silver/Black	R	183,57
9	White	R	17,98
10	Yellow	R	15380,56
11	NULL	R	96042,36
12	Black	S	573,39

Рис. 51. Результат выполнения функции CUBE

Результат выполнения запроса (пример 38)

Пример 39

Показать итоговую сумму для каждого цвета товара в каждой линейке продукта, используя функцию ROLLUP.

```

SELECT
    Color
    , ProductLine
    , SUM(ListPrice) AS [sum]
FROM Production.Product
WHERE
    Color IS NOT NULL
    AND ProductLine IS NOT NULL
GROUP BY ROLLUP(Color, ProductLine);

```

Результаты		Сообщения	
	Color	ProductLine	sum
1	Black	M	38341,74
2	Black	R	27221,14
3	Black	S	573,39
4	Black	T	463,02
5	Black	NULL	66599,29
6	Blue	S	225,49
7	Blue	T	23790,17
8	Blue	NULL	24015,66
9	Grey	T	125,00
10	Grey	NULL	125,00
11	Multi	S	478,92
12	Multi	NULL	478,92

Рис. 52. Результат выполнения функции ROLLUP.

Результат выполнения запроса (пример 39)

В результате выполнения вышеприведенных запросов для примеров 38 и 39 были получены результирующие наборы данных. При выполнении запроса с применением функции CUBE результирующий набор данных имеет на четыре строки больше, чем при выполнении запроса с применением функции ROLLUP. Результирующие наборы данных отличаются «лишним» подытогом, который можно получить при выполнении следующего запроса:

```
SELECT
    SUM(ListPrice) AS [SUM]
    , ProductLine
FROM Production.Product
WHERE
    IS NOT NULL
    AND ProductLine IS NOT NULL
GROUP BY ProductLine;
```

Результаты		Сообщения
	SUM	ProductLine
1	74606,26	M
2	96042,36	R
3	1583,74	S
4	43389,95	T

Рис. 53. Разница в результатах выполнения операторов CUBE и ROLLUP

Использование GROUPING SETS

Параметр GROUPING SETS позволяет объединять несколько предложений GROUP BY в одно предложение GROUP BY. Результаты эквивалентны тем, которые формируются с применением конструкции UNION ALL¹⁴ к указанным группам [14].

Например, GROUP BY ROLLUP (Field1, Field2) и GROUP BY GROUPING SETS (ROLLUP (Field1, Field2)) возвращают одинаковые результаты.

Если параметр GROUPING SETS имеет два или более элементов, результатом будет объединение элементов.

Пример 40

Рассчитать максимальную цену товара в каждой линейке товара и для каждого цвета, используя GROUPING SETS.

¹⁴ Объединение результатов без удаления повторяющихся строк. Подробнее описано далее.

```

SELECT
    ProductLine
    , Color
    , MAX(ListPrice) AS MaxPrice
FROM Production.Product
WHERE
    Color IS NOT NULL
    AND ProductLine IS NOT NULL
GROUP BY GROUPING SETS(
    (Color)
    , (ProductLine)
);

```

	ProductLine	Color	MaxPrice
1	M	NULL	3399,99
2	R	NULL	3578,27
3	S	NULL	89,99
4	T	NULL	2384,07
5	NULL	Black	3374,99
6	NULL	Blue	2384,07
7	NULL	Grey	125,00
8	NULL	Multi	89,99
9	NULL	Red	3578,27
10	NULL	Silver	3399,99
11	NULL	Silver/Black	80,99
12	NULL	White	9,50
13	NULL	Yellow	2384,07

Рис. 54. Результат выполнения оператора GROUPING SETS.

Результат выполнения запроса (пример 40)

GROUP BY ()

GROUP BY () Указывает пустую группу, что приводит к созданию общего итога. Он полезен в качестве одного из элементов GROUPING SET.

Пример 41

Рассчитать максимальную цену товара в каждой линейке товара и для каждого цвета. Результат дополнить подытогом максимальной цены по всем столбцам.

```
SELECT
    ProductLine
    , Color
    , MAX(ListPrice) AS MaxPrice
FROM Production.Product
WHERE
    Color IS NOT NULL AND
    ProductLine IS NOT NULL
GROUP BY GROUPING SETS(
    (Color)
    , (ProductLine)
    , ()
);
```

	ProductLine	Color	MaxPrice
3	S	NULL	89,99
4	T	NULL	2384,07
5	NULL	NULL	3578,27
6	NULL	Black	3374,99
7	NULL	Blue	2384,07
8	NULL	Grey	125,00
9	NULL	Multi	89,99
10	NULL	Red	3578,27
11	NULL	Silver	3399,99
12	NULL	Silver/Black	80,99
13	NULL	White	9,50
14	NULL	Yellow	2384,07

**Рис. 55. Результат выполнения оператора GROUPING SETS
с общим подытогом (пример 41)**

Результаты двух запросов отличаются дополнительными скобками для примера 41 в предложении `GROUPING SETS`. В результате на рисунке 55 пятая строка содержит максимальную цену и среди всех цветов, и во всех вариантах линейки товара.

GROUPING

`GROUPING` используется, чтобы различать значения `NULL`, возвращаемые операторами `ROLLUP`, `CUBE` или `GROUPING SETS`, и стандартные значения `NULL`. Возвращение `NULL` в качестве результата операции `ROLLUP`, `CUBE` или `GROUPING SETS` является особым случаем использования `NULL`. Значение служит заполнителем столбца в результирующем наборе и означает «все».

В результирующем наборе функция `GROUPING` возвращает 1 (агрегатное выражение) или нуль (неагрегатное выражение). В качестве аргумента функции `GROUPING ( )` должен быть столбец или выражение, которое содержит столбец в предложении `GROUP BY`.

Пример 42

Создать запрос, который показывает квоты продаж (`SalesQuota`) и просуммированные продажи (`SalesYTD`) из таблицы `Sales.SalesPerson`, используя оператор `CUBE`. Отделить строки, созданные с помощью агрегатных функций от строк из фактической таблицы.

```
SELECT
    SalesQuota
    , SUM(SalesYTD) AS TotalSalesYTD
    , GROUPING(SalesQuota) AS Grouping
FROM Sales.SalesPerson
GROUP BY CUBE(SalesQuota);
```

Results		Messages	
	SalesQuota	TotalSalesYTD	Grouping
1	NULL	1252127,9471	0
2	250000,00	27370537,97	0
3	300000,00	7654925,9863	0
4	NULL	36277591,9034	1

Рис. 56. Результат выполнения запроса (пример 42)

На рисунке 56 первая строка в столбце SalesQuota содержит значение Null — пустая квота продаж. Значение в последнем столбце содержит значение нуль, что означает, что к этой строке не применялась агрегатная функция. Однако в последней строке значение NULL показывает, что значение во втором столбце соответствует сумме всех строк. Об этом сигнализирует значение 1 в столбце Grouping — к этой строке применялась агрегатная функция.

Пример 43

Создать итоговый отчет, который показывает общее количество (Quantity) каждого товара на каждой полке (Shelf) из таблицы Production.ProductInventory, с использованием операторов CUBE и GROUPING.

```

SELECT
    ProductID
    , Shelf
    , SUM(Quantity) AS TotalQuantity
    , GROUPING(Shelf) AS ShelGrouping
FROM Production.ProductInventory
GROUP BY CUBE(ProductID, Shelf);

```

Results		Messages		
	ProductID	Shelf	TotalQuantity	ShelGrouping
907	935	W	156	0
908	936	W	155	0
909	937	W	153	0
910	938	W	152	0
911	939	W	150	0
912	940	W	323	0
913	941	W	388	0
914	NULL	W	2908	0
915	523	Y	209	0
916	524	Y	228	0
917	NULL	Y	437	0
918	NULL	NU...	335974	1
919	1	NU...	1085	1
920	2	NU...	1109	1
921	3	NU...	1352	1

Рис. 57. Результат выполнения запроса (пример 43)

В строке 918 на рисунке 57 значения NULL в первом и втором столбце означают, что значение третьего столбца — общее количество всех товаров на всех полках. Значение NULL в строке 914 показывает общее количество товара на полке W. Функция GROUPING применялась к полю Shelf (полка), поэтому все строки, начиная с 919, показывают общее количество соответствующего товара (ProductID) на всех полках (значение 1 в последнем столбце).

Ранжирование

Иногда удобно иметь результирующий набор данных, ранжируемый по некоторому критерию.

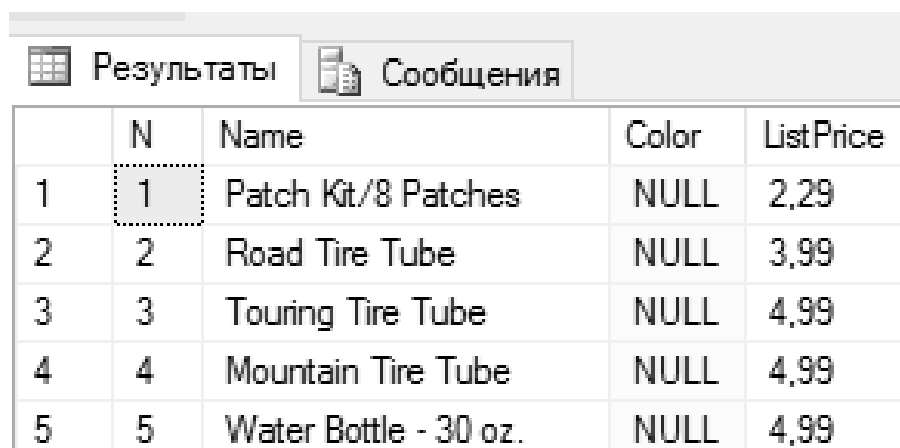
ROW_NUMBER()

Для добавления столбца, содержащего номер строки, используется функция `ROW_NUMBER() OVER (ORDER BY Field1)`. Оператор `OVER` определяет столбец, по которому будет проведено ранжирование, в круглых скобках после оператора `OVER` следует оператор `ORDER BY`¹⁵, в котором явно указывается столбец `Field1` и порядок сортировки для ранжирования `ASC/DESC`.

Пример 44

Показать наименование товара, цвет и его цену. Добавить столбец с номером строки, определяя его порядок в зависимости от цены.

```
SELECT
    ROW_NUMBER() OVER (ORDER BY ListPrice) AS N
    , Name
    , Color
    , ListPrice
FROM Production.Product
WHERE ListPrice > 0;
```



	N	Name	Color	ListPrice
1	1	Patch Kit/8 Patches	NULL	2,29
2	2	Road Tire Tube	NULL	3,99
3	3	Touring Tire Tube	NULL	4,99
4	4	Mountain Tire Tube	NULL	4,99
5	5	Water Bottle - 30 oz.	NULL	4,99

Рис. 58. Применение функции `ROW_NUMBER()`

Результат выполнения запроса (пример 44)

¹⁵ Результирующий набор можно отсортировать по другому столбцу, используя оператор `ORDER BY Field2`, который указывается самым последним в запросе. В этом случае столбец, полученный с помощью функции `ROW_NUMBER()`, не будет отсортирован по значению.

Пример 45

Показать наименование товара, цвет и его цену. Добавить столбец с номером строки, определяя его порядок в зависимости от цены. Отсортировать результирующий набор по наименованию.

```
SELECT
    ROW_NUMBER() OVER (ORDER BY ListPrice) AS N
    , Name
    , Color
    , ListPrice
FROM Production.Product
WHERE ListPrice > 0
ORDER BY Name;
```



	N	Name	Color	ListPrice
1	110	All-Purpose Bike Stand	NULL	159,00
2	10	AWC Logo Cap	Multi	8,99
3	6	Bike Wash - Dissolver	NULL	7,95
4	25	Cable Lock	NULL	25,00
5	16	Chain	Silver	20,24
6	74	Classic Vest, L	Blue	63,50
7	73	Classic Vest, M	Blue	63,50
8	72	Classic Vest, S	Blue	63,50
9	18	Fender Set - Mountain	NULL	21,98
10	93	Front Brakes	Silver	106,50
11	89	Front Derailleur	Silver	91,49

Рис. 59. Применение функции ROW_NUMBER() с сортировкой по Name.

Результат выполнения запроса (пример 45)

На рисунке 59 видно, что значения в столбце N не упорядочены, а в столбце Name расположены в лексикографическом порядке. Для сравнения (рис. 58) значения в столбце N упорядочены по возрастанию.

RANK()

Для удобства можно ранжировать результирующий набор по значению определенного столбца. Одинаковые значения будут иметь одинаковый номер ранга.

Пример 46

Ранжировать товар в Production.Product по цене. Вывести наименование и цену продукта.

```
SELECT  
    RANK() OVER(ORDER BY ListPrice) AS N  
    , Name  
    , ListPrice  
FROM Production.Product  
WHERE ListPrice > 0;
```

	N	Name	ListPrice
1	1	Patch Kit/8 Patches	2,29
2	2	Road Tire Tube	3,99
3	3	Touring Tire Tube	4,99
4	3	Mountain Tire Tube	4,99
5	3	Water Bottle - 30 oz.	4,99
6	6	Bike Wash - Dissolver	7,95
7	7	Road Bottle Cage	8,99
8	7	Racing Socks, M	8,99
9	7	Racing Socks, L	8,99
10	7	AWC Logo Cap	8,99
11	11	Mountain Bike Socks, M	9,50

Рис. 60. Применение функции RANK ()

Результат выполнения запроса (пример 46)

Пример 47

Ранжировать товар в Production.Product по цене. Вывести наименование, цену продукта и цвет.

```
SELECT
    RANK() OVER(ORDER BY ListPrice) AS N
    , Name
    , ListPrice
    , Color
FROM Production.Product
WHERE ListPrice > 0 AND Color IS NOT NULL;
```

	N	Name	ListPrice	Color
1	1	AWC Logo Cap	8,99	Multi
2	1	Racing Socks, M	8,99	White
3	1	Racing Socks, L	8,99	White
4	4	Mountain Bike Socks, M	9,50	White
5	4	Mountain Bike Socks, L	9,50	White
6	6	Chain	20,24	Silver
7	7	Half-Finger Gloves, S	24,49	Black
8	7	Half-Finger Gloves, M	24,49	Black
9	7	Half-Finger Gloves, L	24,49	Black
10	10	Sport-100 Helmet, Blue	34,99	Blue
11	10	Sport-100 Helmet, Red	34,99	Red

Рис. 61. Применение функции RANK().

Результат выполнения запроса (пример 47)

Пример 48

Ранжировать товар в Production.Product по цене, разбивая значения внутри одного цвета. Вывести наименование, цену продукта и цвет.

```

SELECT
    RANK() OVER(
        PARTITION BY Color
        ORDER BY ListPrice) AS N
    , Name
    , ListPrice
    , Color
FROM Production.Product
WHERE ListPrice > 0 AND Color IS NOT NULL;

```

	N	Name	ListPrice	Color
86	86	Mountain-100 Black, 38	3374,99	Black
87	86	Mountain-100 Black, 42	3374,99	Black
88	86	Mountain-100 Black, 44	3374,99	Black
89	86	Mountain-100 Black, 48	3374,99	Black
90	1	Sport-100 Helmet, Blue	34,99	Blue
91	2	Classic Vest, S	63,50	Blue
92	2	Classic Vest, M	63,50	Blue
93	2	Classic Vest, L	63,50	Blue
94	5	LL Touring Frame - Blu...	333,42	Blue
95	5	LL Touring Frame - Blu...	333,42	Blue
96	5	LL Touring Frame - Blu...	333,42	Blue

Рис. 62. Применение функции RANK ()

с использованием PARTITION.

Результат выполнения запроса (пример 48)

DENSE_RANK()

На рисунке 62 показано, что номера в полученном столбце N имеют «скачки». Значения в столбце N меняются с шагом, равным количеству строк с одинаковым рангом. Для того чтобы избежать неравномерности (чтобы значения шли подряд, независимо от того, сколько строк имело одинаковое значение), используется следующая функция DENSE_RANK () .

Работа с функцией `DENSE_RANK()` полностью аналогична работе с функцией `RANK()`.

Пример 49

Ранжировать товар в `Production.Product` по цене, разбивая значения внутри одного цвета. Вывести наименование, цену продукта и цвет.

```
SELECT
    DENSE_RANK() OVER
        (
            PARTITION BY Color
            ORDER BY ListPrice
        ) AS N
    , Name
    , ListPrice
    , Color
FROM Production.Product
WHERE ListPrice > 0 AND Color IS NOT NULL;
```

Результаты		Сообщения		
	N	Name	ListPrice	Color
1	1	Half-Finger Gloves, S	24,49	Black
2	1	Half-Finger Gloves, M	24,49	Black
3	1	Half-Finger Gloves, L	24,49	Black
4	2	Sport-100 Helmet, Black	34,99	Black
5	3	Full-Finger Gloves, S	37,99	Black
6	3	Full-Finger Gloves, M	37,99	Black
7	3	Full-Finger Gloves, L	37,99	Black
8	4	Men's Sports Shorts, M	59,99	Black
9	4	Men's Sports Shorts, L	59,99	Black
10	4	Men's Sports Shorts, XL	59,99	Black
11	4	Men's Sports Shorts, S	59,99	Black

Рис. 63. Применение функции `DENSE_RANK()`

с использованием `PARTITION()`.

Результат выполнения запроса (пример 49)

NTILE()

Для разбиения результирующего набора на части применяется функция `NTILE (parts)`, в скобках указывается количество частей.

Пример 50

Показать наименование товара, цену и цвет. Разбить ответ на пять равных частей, упорядочив по возрастанию цены.

```
SELECT
    NTILE(5) OVER (ORDER BY ListPrice)
    , Name
    , Color
    , ListPrice
FROM Production.Product
WHERE ListPrice > 0;
```

	(Отсутствует имя столбца)	Name	Color	ListPrice
55	1	HL Road Seat/Saddle	NULL	52,64
56	1	HL Touring Seat/Saddle	NULL	52,64
57	1	Short-Sleeve Classic J...	Yell...	53,99
58	1	Short-Sleeve Classic J...	Yell...	53,99
59	1	Short-Sleeve Classic J...	Yell...	53,99
60	1	Short-Sleeve Classic J...	Yell...	53,99
61	1	LL Bottom Bracket	NULL	53,99
62	2	Hydration Pack - 70 oz.	Silver	54,99
63	2	Men's Sports Shorts, M	Black	59,99
64	2	Men's Sports Shorts, L	Black	59,99
65	2	Men's Sports Shorts, XL	Black	59,99

Рис. 64. Применение функции `NTILE()`

Результат выполнения запроса (пример 50)

В результирующем наборе получилось 304 строки. Мы попросили разбить его на пять частей, т. е. в каждой части будет 61 строка, а в последней — 60 строк.

СОЕДИНЕНИЕ ДАННЫХ ИЗ МНОЖЕСТВА ТАБЛИЦ

Оператор `INNER JOIN`

При разработке структуры БД необходимо следовать правилу, что данные должны храниться только один раз. Такое правило позволяет не допустить аномалий добавления, удаления и обновления данных. То есть вся информация должна быть нормализована. Однако для конечного пользователя такой вид информации в таблицах будет нечитаем, так как числовые значения ключей ни о чем не говорят человеку.

Для удобочитаемости информации необходимо запрашивать данные из родительских и дочерних таблиц, соединенных по ключевому полю. В реляционной алгебре таким оператором является оператор *естественное соединение*, в стандарте SQL — оператор `INNER JOIN ... ON`:

```
SELECT
    A.Field1
  , A.FieldN
  , B.Field1
  , B.FieldN
FROM TableA AS A
     INNER JOIN TableB AS B
       ON A.ID = B.ID_TableA16
```

Обратите внимание на то, что при работе с несколькими таблицами в предложении `SELECT` необходимо указывать полное имя столбца, включая имя таблицы или ее псевдоним (`alias`).

В литературе можно встретить другой способ выборки значений из нескольких таблиц простым перечислением их имен в предложении `FROM`, а затем

¹⁶ Когда вы проектируете БД, следует придерживаться правил именования первичных и вторичных ключей. В данном примере первичный ключ именуется `ID`, вторичный ключ — `ID_ИмяРодительскойТаблицы`.

наложением ограничения в предложении WHERE — равенство первичного ключа родительской таблицы и вторичного ключа дочерней таблицы:

```
SELECT
    A.Field1
  , A.Field2
  , B.Field1
  , B.Field2
  , ...
FROM TableA AS A, TableB AS B
WHERE A.ID = B.ID_TableA
```

Такой способ лучше не использовать, так как предложение WHERE является необязательным в данном запросе и про него легко забыть. Ошибки в синтаксисе не будет, и запрос выполнится. Однако результат запроса не будет правильным. Но не это является самым страшным последствием данной ошибки, а количество результирующих данных и время выполнения запроса. Дело в том, что в результате запроса каждая строка из родительской таблицы соединится с каждой строкой дочерней таблицы, в итоге количество строк результирующего набора будет равно произведению строк первой и второй таблиц. Обычно в таблицах БД хранят тысячи строк, следовательно, в результате получится набор из миллионов строк.

В любом случае, даже если вы не забыли предложение WHERE, в котором ограничивается результирующий набор, сначала СУБД соединит и получит миллионы строк, и только после этого ограничит его для пользователя и покажет только те, ключи которых совпадают по значению. Такой подход будет сложнее и дороже для СУБД, чем применение оператора INNER¹⁷ JOIN ... ON.

В MS SQL Server существует специальный оператор *Декартово произведение* — CROSS JOIN, работа которого аналогична выполнению запроса на выборку без фильтрации:

¹⁷ В MS SQL Server оператор INNER для внутреннего соединения можно опустить.

```

SELECT
    A.Field1
    , A.Field2
    , B.Field1
    , B.Field2
    , ...
FROM TableA AS A, TableB AS B

```

Воспользуемся примером из Википедии¹⁸. В схеме данных всего две таблицы: Города (City) и Жители (Citizen), которые живут в этих городах. В таблице Citizen имеется поле ID_City, которое является внешним ключом к таблице City.

Пример 51

Показать, в каком городе живёт каждый житель.

```

SELECT
    City.City
    , Citizen.FirstName
FROM City
    INNER JOIN Citizen
    ON City.ID = Citizen.ID_City;

```

Результаты		Сообщения
	City	FirstName
1	Москва	Андрей
2	Москва	Сергей
3	Санкт-Петербург	Леонид

Рис. 65. Результат выполнения оператора INNER JOIN
Результат выполнения запроса (пример 51)

¹⁸ [https://ru.wikipedia.org/wiki/Join_\(SQL\)](https://ru.wikipedia.org/wiki/Join_(SQL)).

Пример 52

Показать пол, имя, фамилию, возраст и должность работника (БД AdventureWorks).

Пол (Gender), должность (JobTitle) и дата рождения (BirthDate) находятся в таблице Работник (Employee). Имя (FirstName) и фамилия (LastName) находятся в таблице Персоны (Person). Все персоны имеют имя, однако не все они являются работниками данного предприятия. Запись о сотруднике хранится в БД только один раз, в таблице Работник (Employee) имеется ссылка в виде вторичного ключа на таблицу Персона (Person) — BusinessEntityID.

В таблице хранится информация о дате рождения. Возраст можно рассчитать как разницу между текущей датой и датой рождения. В MS SQL Server имеется множество функций для работы с типом DateTime. В данном примере для расчета возраста можно применить функцию DATEDIFF(Интервал, НачальнаяДата, КонечнаяДата).

```
SELECT
    E.Gender
    , DATEDIFF(YEAR, E.BirthDate, GETDATE()) AS Age
    , E.JobTitle
    , P.FirstName
    , P.LastName
FROM HumanResources.Employee AS E
    INNER JOIN Person.Person AS P
    ON P.BusinessEntityID = E.BusinessEntityID;
```

Результаты		Сообщения			
	Gender	Age	JobTitle	FirstName	LastName
1	M	40	Vice President of Production	James	Hamilton
2	M	41	Production Control Manager	Peter	Krebs
3	F	67	Production Supervisor - WC60	Jo	Brown
4	M	41	Production Technician - WC60	Guy	Gilbert
5	F	44	Marketing Specialist	Jill	Williams
6	F	61	Marketing Specialist	Mary	Gibson
7	M	36	Marketing Specialist	Sariva	Hammadounosatava

Рис. 66. Результат выполнения оператора INNER JOIN

Результат выполнения запроса (пример 52)

Результирующий набор данных можно ограничить в предложении WHERE.

Пример 53

Показать имя, фамилию, должность для мужчин старше 35 лет (БД AdventureWorks).

```

SELECT
    DATEDIFF(YEAR, E.BirthDate, GETDATE()) AS Age
    , E.JobTitle
    , P.FirstName
    , P.LastName
FROM HumanResources.Employee AS E
    INNER JOIN Person.Person AS P
        ON P.BusinessEntityID = E.BusinessEntityID
WHERE E.Gender = 'M'
    AND DATEDIFF(YEAR, E.BirthDate, GETDATE()) < 35;

```

Результаты Сообщения				
	Age	JobTitle	FirstName	LastName
1	33	Production Technician - WC60	Simon	Rapier
2	32	Production Technician - WC60	Steve	Masters
3	34	Production Supervisor - WC30	Michael	Ray
4	34	Production Technician - WC30	Bjom	Rettig
5	34	Production Supervisor - WC50	Jinghao	Liu
6	33	Production Technician - WC50	Chad	Niswonger
7	34	Production Technician - WC50	Michael	Entin
8	33	Production Technician - WC40	Baris	Cetinok
9	32	Production Technician - WC40	Michael	Rothkugel
10	33	Production Technician - WC20	Paul	Singh
11	34	Production Technician - WC30	Jason	Watters
12	34	Production Technician - WC20	Fadi	Fakhouri
13	33	Production Technician - WC45	Alex	Nayberg

Рис. 67. Результат выполнения оператора `INNER JOIN` с предложением `WHERE` (пример 53)

Ограничений на количество таблиц в операторе *Соединение* нет.

Оператор `OUTER JOIN`

LEFT OUTER JOIN

Часто возникает необходимость соединить таблицы по неключевому полю. В случае совпадения в результирующем наборе останется строка с данными, принадлежащими обоим таблицам. Если нет совпадения по выбранному полю, то в результирующем наборе остается строка, содержащая данные из левой таблицы (той, к которой происходит соединение), и значения `NULL` для всех полей из правой таблицы. Покажем это с помощью диаграммы Венна.

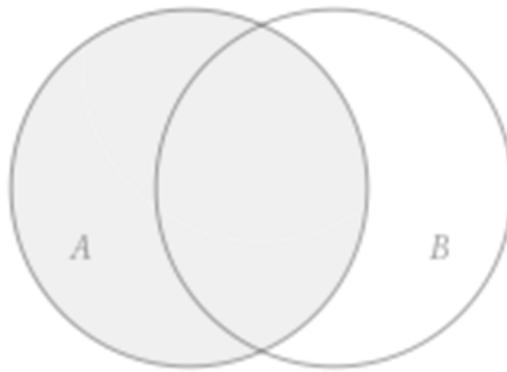


Рис. 68. Изображение LEFT OUTER JOIN с помощью диаграммы Венна

Пример 54

Показать жителей всех городов.

```
SELECT
    City.City
    , Citizen.FirstName
FROM City
LEFT OUTER JOIN Citizen
    ON City.ID = Citizen.ID_City
```

Результаты		Сообщения
	City	First Name
1	Москва	Андрей
2	Москва	Сергей
3	Санкт-Петербург	Леонид
4	Казань	NULL

Рис. 69. Результат выполнения оператора LEFT OUTER JOIN
Результат выполнения запроса (пример 54)

Как видно из рисунка 69, в результирующий набор данных попали все записи из таблицы City, которая указана первой, и соответствующие записи из

таблицы Citizen. Так как в городе Казань никто не живет, то значение в поле FirstName отсутствует.

Пример 55

Показать города, в которых проживают все жители.

```
SELECT
    City.City
    , Citizen.FirstName
FROM Citizen
LEFT OUTER JOIN City
    ON City.ID = Citizen.ID_City;
```

 Результаты  Сообщения		
	City	First Name
1	Москва	Андрей
2	Москва	Сергей
3	Санкт-Петербург	Леонид
4	NULL	Григорий

Рис. 70. Результат запроса LEFT OUTER JOIN

Результат выполнения запроса (пример 55)

Результирующий набор (рис. 70) содержит четыре строки, так как таблица, с которой происходит соединение, имеет четыре строки. Последняя строка не содержит значение в поле FirstName. С точки зрения теории реляционных баз данных значение внешнего ключа не должно быть пустым, однако в жизни может встречаться такая ситуация, как промежуточный результат (при удалении записи из главной таблицы все ссылки в дочерних таблицах принимают значение NULL).

Пример 56

Показать стандартную цену для каждого продукта.

```
SELECT
    P.Name
    , P.ModifiedDate
    , P.ListPrice
    , C.StandardCost
FROM Production.Product P
    LEFT OUTER19 JOIN Production.ProductCostHistory C
    ON P.ProductID = C.ProductID;
```

	Name	ModifiedDate	ListPrice	StandardCost
206	Seat Tube	2008-03-11 10:01:36.827	0,00	NULL
207	Top Tube	2008-03-11 10:01:36.827	0,00	NULL
208	Tension Pulley	2008-03-11 10:01:36.827	0,00	NULL
209	Rear Derailleur Cage	2008-03-11 10:01:36.827	0,00	NULL
210	HL Road Frame - Bl...	2008-03-11 10:01:36.827	1431,50	NULL
211	HL Road Frame - R...	2008-03-11 10:01:36.827	1431,50	NULL
212	Sport-100 Helmet, R...	2008-03-11 10:01:36.827	34,99	12,0278
213	Sport-100 Helmet, R...	2008-03-11 10:01:36.827	34,99	13,8782
214	Sport-100 Helmet, R...	2008-03-11 10:01:36.827	34,99	13,0863
215	Sport-100 Helmet, Bl...	2008-03-11 10:01:36.827	34,99	12,0278
216	Sport-100 Helmet, Bl...	2008-03-11 10:01:36.827	34,99	13,8782

Рис. 71. Результат запроса LEFT OUTER JOIN

Как видно из результатов запроса (рис. 71), в результирующем наборе присутствуют все строки из таблицы `Production.Product`. Строки из таблицы `Production.ProductCostHistory`, значения столбца `ProductID` в которых не совпадают со значениями одноименного столбца из таблицы `Production.Product`, присутствуют, но имеют значение `NULL`.

¹⁹ В MS SQL Server оператор `OUTER` можно опустить. При наличии операторов `LEFT`, `RIGHT`, `FULL` СУБД «знает», что применяется оператор внешнего соединения.

RIGHT OUTER JOIN

Правое внешнее соединение отличается от левого внешнего соединения направлением ассоциации. Покажем это с помощью диаграммы Венна.

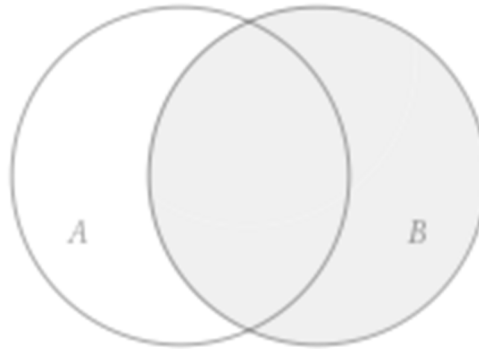


Рис. 72. Изображение RIGHT OUTER JOIN с помощью диаграммы Венна

Следующие два запроса покажут одинаковый результат:

Запрос с применением LEFT OUTER JOIN:

```
SELECT
    T1.Field1
    , ...
    , T1.FieldN
    , T2.Field1
    , ...
    ,FieldM
FROM Table1 T1
LEFT JOIN Table2 T2
ON T1.FieldK = T2.FieldK
```

Запрос с применением RIGHT OUTER JOIN:

```

SELECT
    T1.Field1
    , ...
    , T1.FieldN
    , T2.Field1
    , ...
    , FieldM
FROM Table2 T2
    RIGHT JOIN Table1 T1
        ON T2.FieldK = T1.FieldK

```

Пример 57

Используя RIGHT OUTER JOIN, выполнить задание (пример 54).

```

SELECT
    City.City
    , Citizen.FirstName
FROM Citizen
    RIGHT OUTER JOIN City
        ON City.ID = Citizen.ID_City;

```

Результаты			Сообщения		
	City	FirstName			
1	Москва	Андрей			
2	Москва	Сергей			
3	Санкт-Петербург	Леонид			
4	Казань	NULL			

Рис. 73. Результат выполнения оператора RIGHT OUTER JOIN

Результат выполнения запроса (пример 54)

Пример 58

Используя RIGHT OUTER JOIN, выполнить задание примера 55.

```

SELECT
    City.City
    , Citizen.FirstName
FROM City;
RIGHT OUTER JOIN Citizen
    ON City.ID = Citizen.ID_City

```

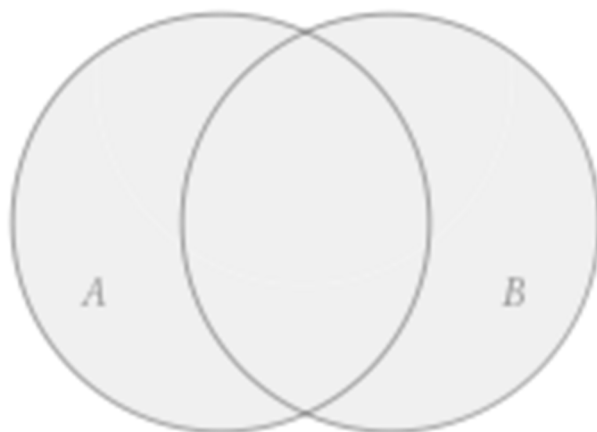
Результаты		Сообщения
	City	FirstName
1	Москва	Андрей
2	Москва	Сергей
3	Санкт-Петербург	Леонид
4	NULL	Григорий

Рис. 74. Результат запроса RIGHT OUTER JOIN
Результат выполнения запроса (пример 55)

Сравнивая результаты выполнения скриптов (рис. 69, 73 и 70, 74), мы видим их идентичность.

FULL OUTER JOIN

Оператор *полное внешнее соединение* применяется, когда нужно соединить таблицы по неключевому полю. В итоге в результирующем наборе будут присутствовать строки из левой таблицы, строки, которые совпадают по полю соединения с правой таблицей, и строки из правой таблицы. Если нет совпадения по полю соединения, то значение в этой строке будет NULL. Покажем это с помощью диаграммы Венна.



**Рис. 75. Изображение FULL JOIN
с помощью диаграммы Венна**

Пример 59

Выполнить FULL OUTER JOIN для таблицы Город (City) и Жители (Citizen).

```
SELECT
    City.City
    , Citizen.FirstName
FROM City
FULL OUTER JOIN Citizen
    ON City.ID = Citizen.ID_City;
```

Результаты		Сообщения
	City	FirstName
1	Москва	Андрей
2	Москва	Сергей
3	Санкт-Петербург	Леонид
4	Казань	NULL
5	NULL	Григорий

Рис. 76. Результат выполнения оператора FULL OUTER JOIN

СОЗДАНИЕ СВОЕЙ БАЗЫ ДАННЫХ

Для лучшего понимания операторов соединения создадим свою базу данных. Пусть имеется магазин, который завозит товары (Product) со склада (Store) и продаёт (Purchase) их. Также имеется таблица поставщиков (Vendor), так как один вид товара может поставляться разными поставщиками. Время, цена покупки и продажи записываются соответственно в таблицы Store и Purchase. После анализа предметной области получим следующую ER-диаграмму.

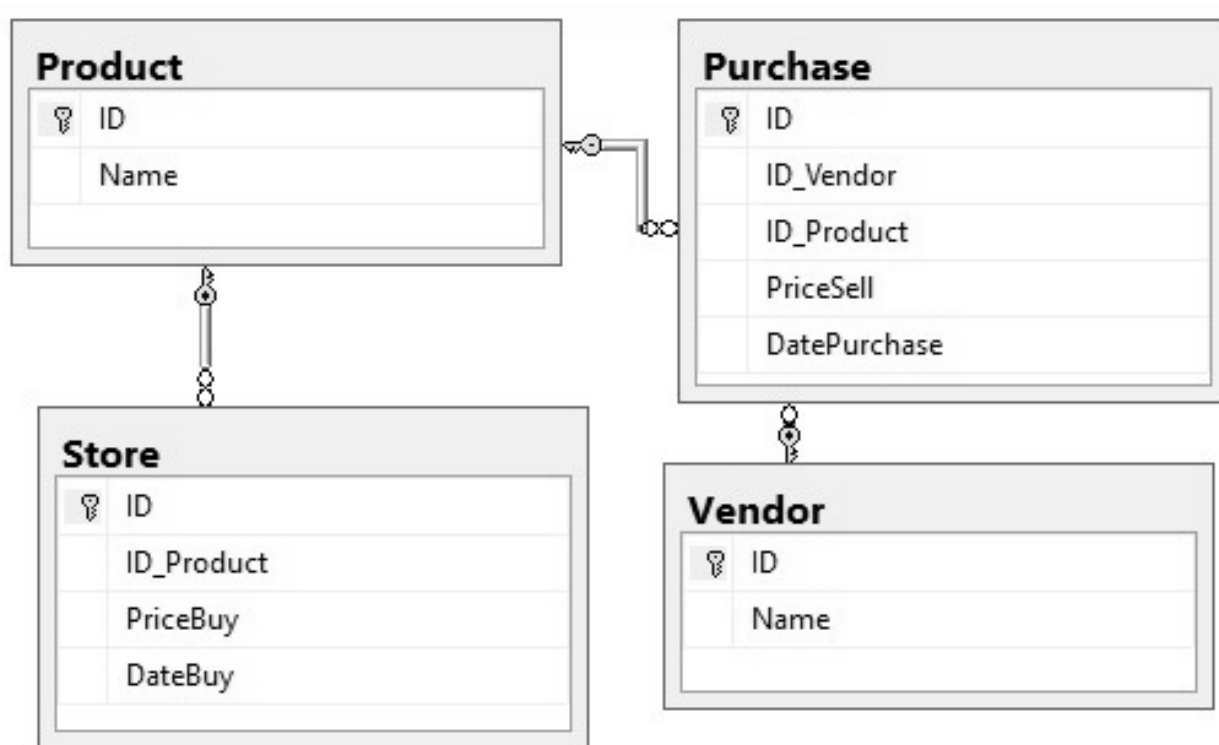


Рис. 77. ER-диаграмма проектируемой БД

Создадим новую базу данных Shop (Магазин). Для создания БД используется оператор CREATE DATABASE:

```
CREATE DATABASE DataBaseName;
```

Для создания таблицы используется оператор CREATE TABLE:

```
CREATE TABLE TableName(  
    FieldName1 <FieldType>  
    , ...  
    , FieldNameN <FieldType>  
);
```

Данный оператор может быть дополнен ограничениями, такими как первичный ключ, вторичный ключ, уникальное поле, поле имеет ограничение на значение и т. д. Возможные варианты будут рассмотрены в отдельной работе. Здесь удобно воспользоваться возможностью установить первичные и вторичные ключи, ограничения уникальности поля и непустого значения.

Скрипт на создание БД:

```
CREATE DATABASE Shop;
```

Скрипты на создание спроектированных таблиц представлены ниже:

```
CREATE TABLE Product (  
    ID INT PRIMARY KEY  
    , Name NVARCHAR(25) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Vendor (  
    ID INT PRIMARY KEY  
    , Name NVARCHAR(25) NOT NULL  
);
```

```
CREATE TABLE Purchase (  
    ID INT PRIMARY KEY  
    , ID_Vendor INT FOREIGN KEY REFERENCES Vendor(ID)  
    , ID_Product INT FOREIGN KEY REFERENCES Product(ID)  
    , PriceSell MONEY NOT NULL  
    , DatePurchase DATETIME NOT NULL  
);
```

```
CREATE TABLE Store (  
    ID INT PRIMARY KEY  
    , ID_Product INT FOREIGN KEY REFERENCES Product(ID)  
    , PriceBuy MONEY NOT NULL
```

```
), DateBuy DATETIME NOT NULL  
);
```

Заполним созданные таблицы. Для этого познакомимся с *операторами манипулирования данными*.

Заполнение таблиц данными

Операторы манипулирования данными относятся к разделу DML (Data Manipulation Language). Для вставки новых строк используется оператор INSERT.

```
INSERT INTO20 TableName(Field1, ...,FieldN)21  
VALUES22  
    (Value1ForField1, ..., ValueNForFieldN) --1-я строка  
    , (...), -- очередная строка  
    (ValueMForField1, ..., ValueKForFieldN);
```

Заполним сначала все главные таблицы (Product и Vendor):

```
INSERT INTO Product VALUES  
    (1, 'Томаты')  
    , (2, 'Бананы')  
    , (3, 'Молоко')  
    , (4, 'Манго')  
    , (5, 'Киви');
```

```
INSERT INTO Vendor VALUES  
    (1, 'Иванов')  
    , (2, 'Петров');
```

Затем заполним дочерние таблицы (Store) и (Purchase), соблюдая правило непротиворечивости данных: значение внешнего ключа должно указывать на существующий первичный ключ.

²⁰ Предлог INTO в MS SQL Server можно опустить.

²¹ Поля в круглых скобках можно не перечислять, а сами скобки не писать, если вставка данных происходит во все поля таблицы.

²² Скобки ставить обязательно. Все значения необходимо перечислять в том же порядке, в каком указаны поля таблицы. Если вставка происходит по всем полям, то значения перечисляются в порядке следования полей при проектировании таблицы.

```

INSERT INTO Store VALUES
  (1, 1, 70, '01.01.2021')
, (2, 2, 40, '01.01.2021')
, (3, 3, 50, '01.01.2021')
, (4, 4, 100, '01.01.2021')
, (5, 5, 50, '01.01.2021')
, (6, 1, 70, '01.02.2021')
, (7, 2, 40, '02.02.2021')
, (8, 3, 50, '03.02.2021');

```

```

INSERT INTO Purchase VALUES
  (1, 1, 1, 70, '01.02.2021')
, (2, 1, 2, 100, '02.02.2021')
, (3, 2, 3, 150, '03.02.2021')
, (4, 2, 4, 110, '01.02.2021')
, (5, 2, 5, 120, '03.02.2021')
, (6, 2, 1, 250, '01.02.2021')
, (7, 1, 5, 205, '02.02.2021')
, (8, 2, 5, 100, '02.02.2021');

```

Обратите внимание, что скрипты на создание (удаление) таблиц, ключей, базы данных и тому подобного можно выполнять только один раз. После выполнения скрипта таблица (или другой объект) будет создана (удалена), и создать (удалить) ее еще раз нельзя.

Скрипты на добавление данных, представленные выше, тоже нельзя выполнять дважды, но по другой причине: в таблицах имеется поле ID, помеченное как первичный ключ, что накладывает ограничение уникальности значений в данном поле.

Пример 60

Показать, какие товары какими поставщиками поставляются. База данных Shop (Магазин).

```

SELECT
  V.Name AS Vendor
, Pr.Name AS Product
FROM Purchase AS P
  INNER JOIN Vendor AS V

```

```

ON V.ID = P.ID_Vendor
  INNER JOIN Product AS Pr
    ON Pr.ID = P.ID_Product
ORDER BY 1;

```

Результаты		Сообщения	
	Vendor	Product	
1	Иванов	Томаты	
2	Иванов	Бананы	
3	Иванов	Киви	
4	Петров	Молоко	
5	Петров	Манго	
6	Петров	Киви	
7	Петров	Томаты	

Рис. 78. Результат выполнения скрипта (пример 60)

Пример 61

Показать продажи продуктов. База данных Shop (Магазин).

```

SELECT P.Name
  , S.PriceBuy AS Price
  , CONVERT(
      NVARCHAR(15)
    , S.DateBuy
    , 104) AS [Date]
FROM Store S
  INNER JOIN Product P
    ON P.ID = S.ID_Product;

```

Результаты		Сообщения	
	Name	Price	Date
1	Томаты	70,00	01.01.2021
2	Бананы	40,00	01.01.2021
3	Молоко	50,00	01.01.2021
4	Манго	100,00	01.01.2021
5	Киви	50,00	01.01.2021
6	Томаты	70,00	02.01.2021
7	Бананы	40,00	02.02.2021
8	Молоко	50,00	02.03.2021

Рис. 79. Результат выполнения скрипта (пример 61)

Пример 62

Показать прибыль по каждому дню продаж. База данных Shop (Магазин).

```
SELECT
    CONVERT(
        NVARCHAR(15)
        , P.DatePurchase
        , 104) AS [DatePurchase]
    , CONVERT(
        NVARCHAR(15)
        , S.DateBuy
        , 104) AS [DateBuy]
    , Pr.Name
```

```

, ISNULL(
    S.PriceBuy
, 0) AS PriceBuy
, ISNULL(
    P.PriceSell
, 0) AS PriceSell
, ISNULL(
    P.PriceSell
, 0)
    -ISNULL(
        S.PriceBuy
, 0) AS Profit
FROM Purchase P
LEFT OUTER JOIN Store S
    ON P.DatePurchase = S.DateBuy
INNER JOIN Product Pr
    ON Pr.ID = S.ID_Product;

```

Результаты		Сообщения				
	DatePurchase	DateBuy	Name	PriceBuy	PriceSell	Profit
1	02.01.2021	02.01.2021	Томаты	70,00	70,00	0,00
2	02.02.2021	02.02.2021	Бананы	40,00	100,00	60,00
3	02.03.2021	02.03.2021	Молоко	50,00	150,00	100,00
4	02.01.2021	02.01.2021	Томаты	70,00	110,00	40,00
5	02.03.2021	02.03.2021	Молоко	50,00	120,00	70,00
6	02.01.2021	02.01.2021	Томаты	70,00	250,00	180,00
7	02.02.2021	02.02.2021	Бананы	40,00	205,00	165,00
8	02.02.2021	02.02.2021	Бананы	40,00	100,00	60,00

Рис. 80. Результат выполнения скрипта (пример 62)

В данном примере в каждый день продажи осуществлялась покупка, поэтому в результате выполнения скрипта не было получено пустых значений (NULL), которые с помощью функции ISNULL были бы преобразованы в числовое значение — ноль.

Пример 63

Показать прибыль на каждый день совершения сделок (покупка или продажа). База данных Shop (Магазин).

```
SELECT
    CONVERT(
        NVARCHAR(15)
        , P.DatePurchase
        , 104) AS [DatePurchase]
    , CONVERT(
        NVARCHAR(15)
        , S.DateBuy
        , 104) AS [DateBuy]
    , Pr.Name
    , ISNULL(
        S.PriceBuy,
        0) AS PriceBuy
    , ISNULL(
        P.PriceSell
        , 0) AS PriceSell
    , ISNULL(
        P.PriceSell
        , 0)
        - ISNULL(
            S.PriceBuy
            , 0) AS Profit
FROM Store S
    FULL OUTER JOIN Purchase P
        ON P.DatePurchase = S.DateBuy
    INNER JOIN Product Pr
        ON Pr.ID = S.ID_Product;
```

Результаты		Сообщения				
	DatePurchase	DateBuy	Name	PriceBuy	PriceSell	Profit
1	NULL	01.01.2021	Томаты	70,00	0,00	-70,00
2	NULL	01.01.2021	Бананы	40,00	0,00	-40,00
3	NULL	01.01.2021	Молоко	50,00	0,00	-50,00
4	NULL	01.01.2021	Манго	100,00	0,00	-100,00
5	NULL	01.01.2021	Киви	50,00	0,00	-50,00
6	02.01.2021	02.01.2021	Томаты	70,00	70,00	0,00
7	02.01.2021	02.01.2021	Томаты	70,00	110,00	40,00
8	02.01.2021	02.01.2021	Томаты	70,00	250,00	180,00
9	02.02.2021	02.02.2021	Бананы	40,00	100,00	60,00
10	02.02.2021	02.02.2021	Бананы	40,00	205,00	165,00
11	02.02.2021	02.02.2021	Бананы	40,00	100,00	60,00
12	02.03.2021	02.03.2021	Молоко	50,00	150,00	100,00
13	02.03.2021	02.03.2021	Молоко	50,00	120,00	70,00

Рис. 81. Результат выполнения скрипта (пример 63)

Как видно, текст скрипта отличается от предыдущего примера применением вместо левого внешнего соединения LEFT OUTER JOIN полного внешнего соединения FULL OUTER JOIN. В данном случае не было продаж 1 января 2021 г., поэтому в столбце «Дата продажи» (DatePurchase) нет значений (NULL), а цена продажи PriceBuy преобразована в нуль.

Пример 64

Рассчитать полную прибыль. База данных Shop (Магазин).

```
SELECT
    SUM(
        ISNULL(
            P.PriceSell
```

```

, 0)
    - ISNULL(
        S.PriceBuy
        , 0)) AS Total
FROM Store S
FULL OUTER JOIN Purchase P
ON P.DatePurchase = S.DateBuy;

```



	Total
1	305,00

Рис. 82. Результат выполнения скрипта (пример 64)

Самосоединение **SELF JOIN**

Иногда требуется соединить таблицу саму с собой. Такое соединение и получило название *самосоединение* **SELF JOIN**. При этом применяется оператор внутреннее соединение **INNER JOIN** по одному из полей с ограничением, что значения по другому полю не равны, с удалением дубликатов.

Пример 65

Какие товары продают несколько поставщиков? База данных Shop (Магазин).

```

SELECT DISTINCT
    Pr.Name
FROM Purchase P1

```

```

INNER JOIN Purchase P223
  ON P1.ID_Product = P2.ID_Product
  INNER JOIN Product Pr24
    ON Pr.ID = P1.ID_Product
WHERE P1.ID_Vendor <> P2.ID_Vendor;

```



	Name
1	Киви
2	Томаты

Рис. 83. Результат выполнения скрипта (пример 65)

Действительно, при заполнении таблиц мы указали, что оба поставщика (Иванов и Петров) продают товар киви и томаты.

Рассмотрим примеры на БД AdventureWorks.

Пример 66

Аналогичная примеру 65 задача. Показать вендоров, которые поставляют несколько товаров.

```

SELECT DISTINCT v1.*
FROM Purchasing.ProductVendor AS v1
  INNER JOIN Purchasing.ProductVendor AS v2
    ON v1.ProductID = v2.ProductID
WHERE v1.BusinessEntityID <> v2.BusinessEntityID;

```

²³ В данном случае применение псевдонима к таблице обязательно, так как таблица соединяется сама с собой.

²⁴ Второй INNER JOIN здесь присутствует, так как мы хотим видеть товары по имени, а не по идентификационному номеру.

	ProductID	BusinessEntityID	AverageLeadTime	StandardPrice	LastReceiptCost
1	317	1578	19	28,17	29,5785
2	317	1678	17	25,77	27,0585
3	318	1578	19	34,38	36,099
4	318	1678	17	31,98	33,579
5	319	1556	19	44,21	46,4205
6	319	1578	19	46,27	48,5835
7	319	1678	17	43,87	46,0635
8	320	1514	19	47,28	49,644
9	320	1602	17	45,21	47,4705
10	320	1604	17	43,21	45,3705
11	321	1514	19	42,21	44,3205
12	321	1602	17	40,76	42,798
13	321	1604	17	38,56	40,488
14	322	1514	19	27,33	28,6965

Рис. 84. Результат выполнения скрипта (пример 66)

Из рисунка 84 видно, что, например, продукт 317 продают поставщики 1578 и 1678.

Пример 67

Показать товары с такой же ценой, как у товара «Racing Socks, L».

```
SELECT DISTINCT
    p2.[Name]
    , p2.Color
    , p2.ListPrice
FROM Production.Product AS p1
    INNER JOIN Production.Product AS p2
        ON (p1.ListPrice = p2.ListPrice)
AND p1.[Name] = 'Racing Socks, L';
```

	Name	Color	ListPrice
1	AWC Logo Cap	Multi	8,99
2	Road Bottle Cage	NULL	8,99
3	Racing Socks, M	White	8,99
4	Racing Socks, L	White	8,99

Рис. 85. Результат выполнения скрипта (пример 67)

Пример 68

Показать все продукты с одинаковой ценой, отличной от нуля.

```
SELECT DISTINCT
    p2.[Name]
    , p2.Color
    , p2.ListPrice
FROM Production.Product AS p1
    INNER JOIN Production.Product AS p2
        ON (p1.ListPrice = p2.ListPrice)
AND p1.ProductID <> p2.ProductID
    AND p1.ListPrice > 0
ORDER BY p2.ListPrice;
```

	Name	Color	ListPrice
1	Water Bottle - 30 oz.	NULL	4,99
2	Mountain Tire Tube	NULL	4,99
3	Touring Tire Tube	NULL	4,99
4	Road Bottle Cage	NULL	8,99
5	Racing Socks, M	White	8,99
6	Racing Socks, L	White	8,99
7	AWC Logo Cap	Multi	8,99
8	Mountain Bike Socks, M	White	9,50
9	Mountain Bike Socks, L	White	9,50
10	Half-Finger Gloves, S	Black	24,49
11	Half-Finger Gloves, M	Black	24,49
12	Half-Finger Gloves, L	Black	24,49
13	Mountain Pump	NULL	24,99
14	LL Mountain Tire	NULL	24,99
15	ML Road Tire	NULL	24,99

Рис. 86. Результат выполнения скрипта (пример 68)

Пример 69

Показать города, в которых проживает несколько персоналий.

```
SELECT DISTINCT
    p.BusinessEntityID
  , p.FirstName
  , p.LastName
  , a1.City
FROM Person.[Address] a1
    INNER JOIN Person.[Address] a2
        ON a1.City = a2.City
        INNER JOIN Person.BusinessEntityAddress AS b
            ON b.AddressID = a1.AddressID
            INNER JOIN Person.Person AS p
                ON p.BusinessEntityID =
b.BusinessEntityID
WHERE a1.AddressID <> a2.AddressID
ORDER BY a1.City;
```

Results		Messages		
	BusinessEntityID	FirstName	LastName	City
1	2397	Isabella	Cox	Ballard
2	2573	Dustin	Nath	Ballard
3	3083	Stephanie	Cox	Ballard
4	3270	Ruth	Srini	Ballard
5	3942	Abigail	Peterson	Ballard
6	4003	Jeremiah	Martinez	Ballard
7	4268	Edward	Hernandez	Ballard
8	4498	Dalton	Hill	Ballard
9	4820	Bianca	Hu	Ballard
10	4959	Devin	Hernandez	Ballard
11	5124	Karl	Xie	Ballard

Рис. 87. Результат выполнения скрипта (пример 69)

Пример 70

Показать все продукты из одной подкатегории.

```
SELECT DISTINCT p1.ProductID
      , p1.[Name]
      , p1.ListPrice
      , s.[Name]
FROM Production.Product AS p1
      INNER JOIN Production.Product AS p2
            ON p1.ProductSubcategoryID =
p2.ProductSubcategoryID
            AND p1.ProductID <> p2.ProductID
      INNER JOIN Production.ProductSubcategory AS s
            ON p1.ProductSubcategoryID =
s.ProductSubcategoryID
ORDER BY s.[Name];
```

	ProductID	Name	ListPrice	Name
1	855	Men's Bib-Shorts, S	89,99	Bib-Shorts
2	856	Men's Bib-Shorts, M	89,99	Bib-Shorts
3	857	Men's Bib-Shorts, L	89,99	Bib-Shorts
4	870	Water Bottle - 30 oz.	4,99	Bottles and Cages
5	871	Mountain Bottle Cage	9,99	Bottles and Cages
6	872	Road Bottle Cage	8,99	Bottles and Cages
7	994	LL Bottom Bracket	53,99	Bottom Brackets
8	995	ML Bottom Bracket	101,24	Bottom Brackets
9	996	HL Bottom Bracket	121,49	Bottom Brackets
10	907	Rear Brakes	106,50	Brakes
11	948	Front Brakes	106,50	Brakes
12	949	LL Crankset	175,49	Cranksets
13	950	ML Crankset	256,49	Cranksets
14	951	HL Crankset	404,99	Cranksets
15	894	Rear Derailleur	121,46	Derailleurs
	...	...	...	...

Рис. 88. Результат выполнения скрипта (пример 70)

ОПЕРАТОР UNION

Оператор *Объединение* UNION применяется для «склейки» набора данных по вертикали. Объединение возможно только по полям, построенным на одних доменах (типах данных), а также должно совпадать их количество.

```
SELECT
    Table1.Field1
    , ...
    , Table1.FieldN
FROM Table1
UNION
SELECT
    Table2.Field1
    , ...
    , Table2.FieldN
FROM Table2
```

Пример 71

Объединить две таблицы Продажи (Purchase) и Покупки (Store). База данных Shop (Магазин).

```
SELECT
    ID
    , ID_Product
    , Store.DateBuy
    , Store.PriceBuy
FROM Store
UNION
SELECT
    ID
    , ID_Product
    , Purchase.DatePurchase
    , Purchase.PriceSell
FROM Purchase;
```

Результаты		Сообщения		
	ID	ID_Product	DateBuy	PriceBuy
1	1	1	2018-01-01 00:00:00.000	70,00
2	1	1	2018-02-01 00:00:00.000	70,00
3	2	2	2018-01-01 00:00:00.000	40,00
4	2	2	2018-02-02 00:00:00.000	100,00
5	3	3	2018-01-01 00:00:00.000	50,00
6	3	3	2018-02-03 00:00:00.000	150,00
7	4	4	2018-01-01 00:00:00.000	100,00
8	4	4	2018-02-01 00:00:00.000	110,00
9	5	5	2018-01-01 00:00:00.000	50,00
10	5	5	2018-02-03 00:00:00.000	120,00
11	6	1	2018-02-01 00:00:00.000	70,00

Рис. 89. Результат выполнения скрипта (пример 71)

Если требуется объединить два набора данных, количество полей у которых не совпадает, то в операторе `SELECT` той таблицы, в которой не хватает столбцов, через запятую необходимо записать числа 1, 2 и т. д., столько, сколько столбцов не хватает. В результирующем наборе в этих столбцах будут соответственно значения 1, 2 и т. д., как указано в операторе `SELECT`. Для типа данных дата в этих столбцах будет соответственно 01.01.1900, 02.01.1900 и т. д.

Пример 72

Выполнить предыдущий пример, но соединить только первые два столбца из таблицы Покупки (Store). База данных Shop (Магазин).

```
SELECT
    ID
```

```

        , ID_Product
        , 1 AS One
        , 2 AS Two
FROM Store
UNION
SELECT
    ID
    , ID_Product
    , Purchase.DatePurchase
    , Purchase.PriceSell
FROM Purchase;

```

Результаты Сообщения				
	ID	ID_Product	One	Two
1	1	1	1900-01-02 00:00:00.000	2,00
2	1	1	2018-02-01 00:00:00.000	70,00
3	2	2	1900-01-02 00:00:00.000	2,00
4	2	2	2018-02-02 00:00:00.000	100,00
5	3	3	1900-01-02 00:00:00.000	2,00
6	3	3	2018-02-03 00:00:00.000	150,00
7	4	4	1900-01-02 00:00:00.000	2,00
8	4	4	2018-02-01 00:00:00.000	110,00
9	5	5	1900-01-02 00:00:00.000	2,00
10	5	5	2018-02-03 00:00:00.000	120,00

Рис. 90. Результат выполнения скрипта (пример 72)

Как видно из рисунка 90, данные 1 и 2 преобразованы в соответствующий тип данных (дата и вещественный).

Оператор `UNION ALL` применяется, как и оператор `UNION`, для соединения данных из нескольких наборов. В отличие от оператора `UNION`, дубликаты строк в данном случае не удаляются.

Пример 73

Сравнение работы операторов UNION и UNION ALL на примере объединения данных из одной таблицы. База данных Shop (Магазин).

```
SELECT Name  
FROM Product  
      UNION  
SELECT Name  
FROM Product;
```



	Name
1	Бананы
2	Киви
3	Манго
4	Молоко
5	Томаты

Рис. 91. Результат выполнения оператора UNION

```
SELECT Name  
FROM Product  
      UNION ALL  
SELECT Name  
FROM Product;
```

Результаты	
	Name
1	Бананы
2	Киви
3	Манго
4	Молоко
5	Томаты
6	Бананы
7	Киви
8	Манго
9	Молоко
10	Томаты

Рис. 92. Результат выполнения оператора UNION ALL

Как видно из рисунка 92, дубликаты не были удалены, число строк увеличено вдвое.

ОПЕРАТОР EXCEPT

Для оператора *Исключение* EXCEPT, как и для оператора *Объединение* UNION, от наборов данных требуется, чтобы количество столбцов и домены, на которых они построены, совпадали. В результирующий набор данных попадают все данные из первого набора данных, которых нет во втором наборе данных.

```
SELECT
    Table1.Field1
    , ...
    , Table1.FieldN
FROM Table1
EXCEPT
SELECT
    Table2.Field1
    , ...
    , Table2.FieldN
FROM Table2
```

Пример 74

Какие продукты поставляет первый продавец, но не поставляет второй? База данных Shop (Магазин).

```
SELECT ID_Product
FROM Purchase
WHERE ID_Vendor = 1
EXCEPT
SELECT ID_Product
FROM Purchase
WHERE ID_Vendor = 2;
```

Результаты	
	ID_Product
1	2

Рис. 93. Результат выполнения оператора ЕХЕРТ

Действительно, поставщик с номером 1 поставляет продукты с номерами 1, 2, 5, а поставщик с номером 2 — продукты с номерами 1, 3, 4, 5. То есть продукт с номером 2 поставляется только первым продавцом.

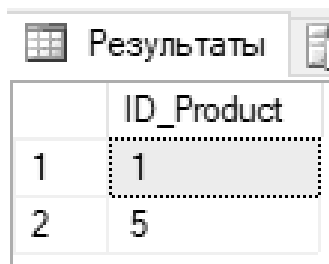
ОПЕРАТОР INTERSECT

Оператор *Пересечение* требует от наборов данных, чтобы количество столбцов и домены, на которых они построены, совпадали. В результирующий набор попадут данные, которые есть и в первом наборе, и во втором.

Пример 75

Какие продукты поставляются продавцом с номером 1 и продавцом с номером 2? База данных Shop (Магазин).

```
SELECT ID_Product
FROM Purchase
WHERE ID_Vendor = 2
      INTERSECT
SELECT ID_Product
FROM Purchase
WHERE ID_Vendor = 1;
```



	ID_Product
1	1
2	5

Рис. 94. Результат выполнения оператора INTERSECT

Действительно, продукты с номерами 1 и 5 поставляются обоими поставщиками.

ОГРАНИЧЕНИЕ РЕЗУЛЬТИРУЮЩЕГО НАБОРА ДАННЫХ.

ОПЕРАТОРЫ TOP И TABLESAMPLE

Чтобы ограничить результирующий набор данных, используются операторы TOP и TABLESAMPLE.

При использовании оператора TOP N выведутся первые N строк результирующего набора, при использовании оператора TOP N с оператором ORDER BY выведутся первые N строк в отсортированном наборе данных.

При использовании оператора TABLESAMPLE должны выводиться первые N% результирующего набора. Этот оператор работает корректно только на больших объемах данных, так как процент берется не от числа строк, а от числа страниц, на которых размещены первичные ключи (или кластеризованные индексы) данной таблицы.

```
SELECT TOP N Field1, ...FieldN  
FROM Table1
```

Пример 76

Показать первые 2 строки таблицы продуктов (Product). База данных Shop (Магазин).

```
SELECT TOP 2 *  
FROM Product;
```

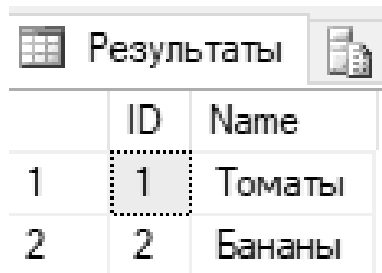
Результаты		
	ID	Name
1	2	Бананы
2	5	Киви

Рис. 95. Результат выполнения запроса с оператором TOP

Пример 77

Показать первые две строки таблицы продуктов (Product), таблицу отсортировать по идентификационному номеру (первому столбцу).

```
SELECT TOP 2 *  
FROM Product  
ORDER BY 1;
```



	ID	Name
1	1	Томаты
2	2	Бананы

**Рис. 96. Результат выполнения запроса
с операторами TOP и ORDER BY**

```
SELECT Field1  
      , ...  
      , FieldN  
FROM Table TABLESAMPLE (N PERCENT)
```

Пример 78

Показать все товары из таблицы Production.Product учебной БД AdventureWorks.

```
SELECT *  
FROM Production.Product;
```

В результате выполнения данного запроса выведено 504 строки.

Пример 79

Показать все товары из таблицы `Production.Product` учебной БД `AdventureWorks`. Вывести 10% всех строк.

```
SELECT *  
FROM Production.Product  
TABLESAMPLE (10 PERCENT);
```

В результате выполнения данного запроса совершенно неочевидно выводится 37 строк.

ПОДЗАПРОСЫ

Язык T-SQL поддерживает написание вложенных запросов. Запрос, который возвращает конечный результат, называют **внешним**. Внутри себя он может использовать результат выполнения других запросов. **Внутренние** запросы вызываются во время выполнения кода и являются аналогами выражений, основанных на переменных или константах. В отличие от выражений, их результат может изменяться в зависимости от содержимого таблицы. Использование вложенных запросов избавляет от необходимости хранить промежуточные результаты в отдельных переменных [15].

Автономные вложенные запросы

Вложенные запросы всегда вызываются извне. Автономный внутренний запрос не зависит от внешнего, его можно выполнять и отлаживать независимо.

Скалярные вложенные запросы

Скалярный вложенный запрос всегда возвращает единственное значение.

Пример 80

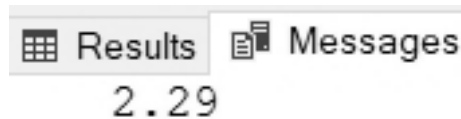
Показать продукты с минимальной ценой.

Решение 1. Решение разобьем на две части. Сначала рассчитаем минимальную цену для всех продуктов и результат запишем в переменную. Затем покажем все продукты с ценой, равной значению нашей переменной.

```
DECLARE @I MONEY;  
  
SET @I = (SELECT MIN(ListPrice)  
          FROM Production.Product  
          WHERE ListPrice > 0);  
  
PRINT @I;
```

```
SELECT [Name], ListPrice
FROM Production.Product
WHERE ListPrice = @I;
```

В результате выполнения кода мы получили результат в переменной @I (рис. 97) и нашли продукт с рассчитанной минимальной ценой (рис. 98).



2.29

Рис. 97. Минимальное значение цены (значение переменной @I)



	Name	ListPrice
1	Patch Kit/8 Patches	2,29

Рис. 98. Результат выполнения запроса (пример 80)

Решение 2. Решение с подзапросом создается следующим образом: запрос из прошлого решения будет внешним, а расчет минимального значения — внутренним подзапросом, взятым в скобки.

```
SELECT [Name], ListPrice
FROM Production.Product
WHERE ListPrice =
    (
        SELECT MIN(ListPrice)
        FROM Production.Product
        WHERE ListPrice > 0
    );
```

В результате мы получим такой же результат, как на рисунке 98.

Пример 81

Показать фамилию, имя и возраст самого молодого сотрудника.

Фамилия, имя и отчество сотрудников находятся в таблице `Person.Person`, сведения о дате рождения — в таблице `HumanResources.Employee`. Решение данной задачи также разделим на две части. Внутренним подзапросом найдем максимальную дату из таблицы `HumanResources.Employee`. Внешним подзапросом найдем идентификатор сотрудника с максимальной датой рождения. Внутренним соединением с таблицей `Person.Person` найдем требуемые фамилию и имя сотрудника и вычислим его возраст.

```
SELECT
    p.FirstName
  , p.MiddleName
  , p.LastName
  , DATEDIFF(YEAR,e.BirthDate,GETDATE()) AS Age
FROM Person.Person AS p
INNER JOIN HumanResources.Employee AS e
ON p.BusinessEntityID = e.BusinessEntityID
WHERE e.BirthDate =
    (
        SELECT MAX(BirthDate)
        FROM HumanResources.Employee
    );
```

	FirstName	MiddleName	LastName	Age
1	Angela	W	Barbariol	35

Рис. 99. Результат выполнения запроса (пример 81)

Вложенные запросы с предикатами

В предыдущих внутренних запросах результатом являлась скалярная величина. Существуют вложенные запросы, в которых в результате выполнения внутреннего запроса возвращается значение в виде столбца.

Предикат IN

Если в результате выполнения подзапроса возвращается множество значений (один столбец), то сравнивать его на строгое равенство нельзя. Для того чтобы отфильтровать данные во внешнем запросе на «равенство» или «неравенство» полученным значениям во внутреннем запросе, применяется оператор IN (NOT IN). Работа с оператором в подзапросах идентична его работе со множеством имеющихся значений [13].

Пример 82

Показать товары, которые продавали максимально часто (первые три места по продажам).

Внутренним подзапросом найдем идентификаторы товаров, продаваемых максимально часто.

```
SELECT TOP 3 ProductID  
FROM Sales.SalesOrderDetail  
GROUP BY ProductID  
ORDER BY SUM(OrderQty);
```

В результате выполнения данного запроса получим следующий результат (рис. 100).

	ProductID
1	897
2	942
3	943

**Рис. 100. Список идентификаторов товаров,
продаваемых максимально часто**

Отлаженный запрос поместим в предложение WHERE с предикатом IN для фильтрации товаров.

```

SELECT [Name]
      , Color
      , Style
      , ListPrice
FROM Production.Product
WHERE ProductID IN (
    SELECT TOP 3 ProductID
    FROM Sales.SalesOrderDetail
    GROUP BY ProductID
    ORDER BY SUM(OrderQty)
);

```

В результате выполнения запроса получим следующий результат (рис. 101).

	Name	Color	Style	ListPrice
1	LL Touring Frame - Blue, 58	Blue	U	333,42
2	ML Mountain Frame-W - Silver, 38	Silver	W	364,09
3	LL Mountain Frame - Black, 40	Black	U	249,79

Рис. 101. Результат выполнения запроса (пример 83)

Пример 83

Показать продавцов, получивших премию более \$5000.

```

SELECT FirstName
      , MiddleName
      , LastName
FROM Person.Person
WHERE BusinessEntityID IN
(
    SELECT BusinessEntityID
    FROM Sales.SalesPerson
    WHERE Bonus > 5000
);

```

Внутренний подзапрос вернет из таблицы Sales.SalesPerson идентификаторы продавцов, получивших премию более \$5000, а внешний подзапрос из таблицы Person.Person покажет фамилии и имена, соответствующие найденным идентификаторам.

Results Messages Client Statistics			
	FirstName	MiddleName	LastName
1	Tsvi	Michael	Reiter
2	Lynn	N	Tsoflias
3	Jae	B	Pak

Рис. 102. Результат выполнения запроса (пример 83)

Такой же результат, как на рисунке 102, можно получить с помощью применения оператора внутреннего соединения INNER JOIN:

```
SELECT FirstName
      , MiddleName
      , LastName
FROM Person.Person AS p
      INNER JOIN Sales.SalesPerson AS s
              ON p.BusinessEntityID = s.BusinessEntityID
WHERE Bonus > 5000;
```

Предикат ANY (SOME)

Предикат ANY (SOME) используется, когда необходимо, чтобы результат фильтрации внешнего запроса был меньше (или меньше либо равен)²⁵ любого значения из множества, возвращаемого внутренним запросом, т. е. меньше (меньше или равно) минимального значения из возвращаемого подзапроса.

Результат выполнения запроса:

²⁵ Также данный предикат можно использовать для отбора больше (или равно), чем любое.

```
SELECT
    Field1
    ,Field2
    ,...
FROM Table
WHERE Field1 <= ANY
    (1, 2, 3)
```

эквивалентен выполнению запроса

```
SELECT
    Field1
    , Field2
    ,...
FROM Table
WHERE Field1 <= 1
```

Пример 84

Показать товары, цена которых больше минимальной средней цены в каждом цвете.

Определим сначала список средних цен для каждого цвета:

```
SELECT AVG(ListPrice) AS AvgListPrice
FROM Production.Product
WHERE ListPrice > 0
AND Color IS NOT NULL
GROUP BY Color
ORDER BY 1;
```

Результат представлен на рисунке 103.

Results		Messages	
	AvgListPrice		
1	9,245		
2	59,865		
3	64,0185		
4	125,00		
5	757,7107		
6	923,6792		
7	959,0913		
8	1015,6425		
9	1401,95		

Рис. 103. Результат выполнения запроса (пример 84)

Как видно, минимальная средняя цена равна \$9,245.

Теперь запишем общий запрос:

```

SELECT [Name]
      , ListPrice
FROM Production.Product
WHERE ListPrice < ANY
(
  SELECT AVG(ListPrice)
  FROM Production.Product
  WHERE ListPrice > 0
  AND Color IS NOT NULL
  GROUP BY Color
)
ORDER BY ListPrice;

```

В результате выполнения данного запроса получим результат, представленный на рисунке 104.

Results			Messages	
	Name	ListPrice		
1	Mountain Bike Socks, M	9,50		
2	Mountain Bike Socks, L	9,50		
3	Mountain Bottle Cage	9,99		
4	Taillights - Battery-Powered	13,99		
5	Minipump	19,99		
6	Chain	20,24		
7	LL Road Tire	21,49		
8	Fender Set - Mountain	21,98		
9	Half-Finger Gloves, S	24,49		
10	Half-Finger Gloves, M	24,49		
11	Half-Finger Gloves, L	24,49		

Рис. 104. Результат выполнения запроса (пример 84)

Пример 85

Показать товары, цена которых меньше максимальной средней цены в каждом цвете.

```

SELECT [Name]
      , ListPrice
FROM Production.Product
WHERE ListPrice < ANY
(
    SELECT AVG(ListPrice)
    FROM Production.Product
    WHERE ListPrice > 0
    AND Color IS NOT NULL
    GROUP BY Color
)
ORDER BY ListPrice DESC;

```

Результат выполнения запроса показан на рисунке 105.

 Results  Messages		
	Name	ListPrice
1	HL Mountain Frame - Silver, 42	1364,50
2	HL Mountain Frame - Silver, 44	1364,50
3	HL Mountain Frame - Silver, 48	1364,50
4	HL Mountain Frame - Silver, 46	1364,50
5	HL Mountain Frame - Silver, 38	1364,50
6	HL Mountain Frame - Black, 42	1349,60
7	HL Mountain Frame - Black, 44	1349,60
8	HL Mountain Frame - Black, 48	1349,60
9	HL Mountain Frame - Black, 46	1349,60
10	HL Mountain Frame - Black, 38	1349,60
11	Touring-2000 Blue, 60	1214,85
12	Touring-2000 Blue, 46	1214,85

Рис. 105. Результат выполнения запроса (пример 84)

Из примера 84 видно, что максимальная средняя цена равна \$1401,95 (см. рис. 103). Из результирующего набора данных (рис. 105) видно, что все цены в результирующем наборе меньше этого значения.

Предикат ALL

Предикат ALL используется, когда в результирующем наборе необходимо провести фильтрацию больше (больше или равно)²⁶ всех значений из подзапроса, т. е. найти значения из внешнего запроса, которые больше или равны максимальному значению из внутреннего запроса.

²⁶ Также данный предикат можно использовать для отбора меньше (или равно), чем все.

Результат выполнения данного запроса:

```
SELECT
    Field1
    , Field2
    , ...
FROM Table
WHERE Field1 >= ALL
    (1, 2, 3)
```

эквивалентен выполнению следующего запроса:

```
SELECT
    Field1
    , Field2
    , ...
FROM Table
WHERE Field1 >= 3
```

Пример 86

Показать товары, цена которых больше любой средней цены в каждом цвете.

```
SELECT
    [Name]
    , ListPrice
FROM Production.Product
WHERE ListPrice > ALL
    (
        SELECT AVG(ListPrice)
        FROM Production.Product
        WHERE ListPrice > 0
            AND Color IS NOT NULL
        GROUP BY Color
    )
ORDER BY ListPrice;
```

Итоговый запрос вернет список товаров, цена которых больше или равна максимальной величине из внутреннего запроса, т. е. не меньше \$1401,95.

Results			Messages
	Name	ListPrice	
1	HL Road Frame - Black, 58	1431,50	
2	HL Road Frame - Red, 58	1431,50	
3	HL Road Frame - Red, 62	1431,50	
4	HL Road Frame - Red, 44	1431,50	
5	HL Road Frame - Red, 48	1431,50	
6	HL Road Frame - Red, 52	1431,50	
7	HL Road Frame - Red, 56	1431,50	
8	HL Road Frame - Black, 62	1431,50	
9	HL Road Frame - Black, 44	1431,50	
10	HL Road Frame - Black, 48	1431,50	
11	HL Road Frame - Black, 52	1431,50	
12	Road-450 Red, 58	1457,99	

Рис. 106. Результат выполнения запроса (пример 86)

Как видно из результирующего набора данных (рис. 106), все товары имеют цены выше, чем максимальная цена из подзапроса (\$1401,95).

Пример 87

Показать товары, цена которых меньше любой средней цены в каждом цвете.

```

SELECT
    [Name]
    ,ListPrice
FROM Production.Product
WHERE ListPrice > 0
    AND ListPrice > ALL
    (
        SELECT AVG(ListPrice)
        FROM Production.Product
        WHERE ListPrice > 0
    )

```

```

        AND Color IS NOT NULL
        GROUP BY Color
    )
ORDER BY ListPrice DESC;

```

Results			Messages
	Name	ListPrice	
1	AWC Logo Cap	8,99	
2	Road Bottle Cage	8,99	
3	Racing Socks, M	8,99	
4	Racing Socks, L	8,99	
5	Bike Wash - Dissolver	7,95	
6	Mountain Tire Tube	4,99	
7	Water Bottle - 30 oz.	4,99	
8	Touring Tire Tube	4,99	
9	Road Tire Tube	3,99	
10	Patch Kit/8 Patches	2,29	

Рис. 107. Результат выполнения запроса (пример 87)

Как видно из результирующего набора данных (рис. 107), все цены ниже минимальной средней цены (\$9,245).

Использование подзапросов в предложении FROM

Как известно, нельзя использовать агрегатную функцию от агрегатной функции, но можно написать запрос с подзапросом следующего вида:

```

SELECT f1(AliasField)
FROM
    (
        SELECT f2(Field1) AS AliasField
        FROM Table
        WHERE (Conditions1) – необязательная инструкция
        GROUP BY Field2 – необязательная инструкция
        HAVING (Conditions2) – необязательная инструкция
    ) AS AliasTable

```

В данном примере f_1 и f_2 — агрегатные функции²⁷.

Запишем запрос с использованием конструкции такого вида для одного из прошлых примеров.

Пример 88

Показать товары, цена которых меньше любой средней цены в каждом цвете. Решение с помощью двойного вложенного подзапроса.

Напишем сначала запрос, который возвращает максимальное значение средней цены для каждого цвета.

```
SELECT MAX(AvgListPrice)
FROM
(
  SELECT AVG(ListPrice) AS AvgListPrice
  FROM Production.Product
  WHERE ListPrice > 0
  AND Color IS NOT NULL
  GROUP BY Color
) AS T;
```

В результате выполнения данного запроса, получим следующий результат (рис. 108):

Results		Messages	
	(No column name)		
1	1401,95		

Рис. 108. Результат выполнения запроса (пример 88)

Значение максимальной средней цены, полученное с помощью применения подзапроса, совпадает с рассчитанным ранее значением (см. рис. 103).

²⁷ Необязательно функции должны быть агрегатные. Конструкцию такого вида можно использовать для любого вида подзапросов.

Применим отлаженный запрос с подзапросом в качестве вложенного внутреннего запроса для внешнего запроса, который будет фильтровать товары по цене — отбирать товары с меньшей ценой:

```
SELECT [Name], ListPrice
FROM Production.Product
WHERE ListPrice <
    (
        SELECT MAX(AvgListPrice)
        FROM
            (
                SELECT AVG(ListPrice) AS AvgListPrice
                FROM Production.Product
                WHERE ListPrice > 0
                    AND Color IS NOT NULL
                GROUP BY Color
            ) AS T
    )
ORDER BY ListPrice DESC;
```

Результат выполнения данного запроса совпадает с результатом (см. рис. 107), полученным для предыдущего примера 87. Аналогичным образом можно записать запросы для примеров 84–86 и получить такие же результаты.

Коррелированные вложенные запросы

Если результат выполнения внутреннего запроса зависит от внешнего, то такой запрос называется зависимым или коррелированным. Отладить отдельно внутренний и внешний запросы нельзя. Записываются такие запросы обычно с применением самосоединения таблицы (SELF JOIN).

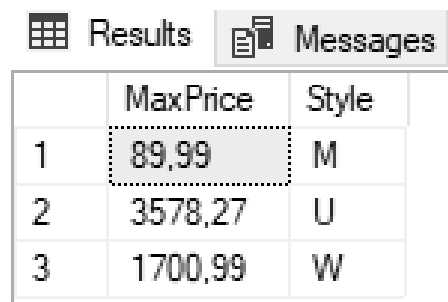
Пример 89

Для каждого стиля показать товары с максимальной ценой (стиль определен).

Несмотря на то что выполнить и отладить внутренний запрос для данного примера отдельно нельзя, запишем его в приближенном к окончательному виде: найдем максимальную цену товара для каждого стиля.

```
SELECT MAX(p2.ListPrice)
FROM Production.Product AS p2
WHERE p2.Style IS NOT NULL
GROUP BY p2.Style;
```

В результате получим набор данных, представленный на рисунке 109.



	MaxPrice	Style
1	89,99	M
2	3578,27	U
3	1700,99	W

Рис. 109. Результат выполнения запроса (пример 89)

По условию задачи необходимо показать товар в каждом стиле, цена которого равна полученной максимальной цене.

Напишем внешний запрос для первого стиля — М, зная, что максимальная цена в этом стиле равна \$89,99 (первая строка) (рис. 109).

```
SELECT p1.[Name]
      , p1.ListPrice
FROM Production.Product AS p1
WHERE p1.Style = 'M'
      AND p1.ListPrice = 89.99
```

В результате выполнения данного запроса получим результат, представленный на рисунке 110.

Results		Messages
	Name	ListPrice
1	Men's Bib-Shorts, S	89,99
2	Men's Bib-Shorts, M	89,99
3	Men's Bib-Shorts, L	89,99

Рис 110. Выборка товаров стиля М с заданной ценой (\$89,99)

Можно представить коррелированный запрос как цикл (на самом деле это не обязательно так), который для каждого стиля во внешнем подзапросе находит совпадающие строки во внутреннем подзапросе `p1.Style = p2.Style`. При выполнении такого подзапроса будет потрачено много машинного времени. Поэтому избегайте использования коррелированных запросов, где это возможно!

Итоговый запрос, показывающий товары для всех стилей, будет сформирован из двух отдельных запросов с некоторой модификацией:

```

SELECT p1.[Name]
      , p1.ListPrice
      , p1.Style -- добавим столбец стиль
FROM Production.Product AS p1
WHERE
    -- p1.Style = 'М' AND удалим фильтрацию для стиля
    p1.Style IS NOT NULL AND -- и добавляем фильтрацию для
стиля на то, что он заполнен
    p1.ListPrice = (
        SELECT MAX(p2.ListPrice)
        FROM Production.Product AS p2
        WHERE p1.Style = p2.Style -- добавим
условие
        -- p2.Style IS NOT NULL удалим условие
        --GROUP BY p2.Style -- удалим группировку
    )
ORDER BY p1.Style; --добавлено для удобства анализа

```

Итоговый запрос был дополнен строкой в предложении `SELECT` внешнего запроса, для того чтобы показать стиль, к которому относятся выбранные товары. В предложении `WHERE` внешнего запроса было убрано ограничение на конкретный вид стиля. Во внутреннем подзапросе были удалены строки на фильтрацию стиля, если тот не определен, потому что добавлено обязательное условие на равенство стиля внутреннего и внешнего запросов, а во внешнем запросе это условие было добавлено. Также во внутреннем запросе удалена строка на группировку по стилю, так как максимальная цена рассчитывается для одного конкретного стиля, пришедшего из внешнего запроса.

Запрос будет выполняться следующим образом. Сначала для внешнего запроса фильтрации будут подвержены строки, в которых стиль не определен. Затем для каждой строки из внешнего запроса будет выполнен внутренний запрос. В предложении выбора во внешнем запросе будут показаны товары, с ценой, равной цене, рассчитанной во внутреннем подзапросе (рис. 111).

Results		Messages	
	Name	ListPrice	Style
1	Men's Bib-Shorts, S	89,99	M
2	Men's Bib-Shorts, M	89,99	M
3	Men's Bib-Shorts, L	89,99	M
4	Road-150 Red, 62	3578,27	U
5	Road-150 Red, 44	3578,27	U
6	Road-150 Red, 48	3578,27	U
7	Road-150 Red, 52	3578,27	U
8	Road-150 Red, 56	3578,27	U
9	Road-350-W Yellow, 40	1700,99	W
10	Road-350-W Yellow, 42	1700,99	W
11	Road-350-W Yellow, 44	1700,99	W
12	Road-350-W Yellow, 48	1700,99	W

Рис. 111. Результат выполнения запроса (пример 89)

В результате получим набор, в котором для каждого стиля показаны товары с максимальной ценой (рис. 111).

Пример 90

Показать товары, в Линейке модели которых существует более 4 размеров

Определим сначала Линейки продуктов, в которых имеется более 4 размеров.

```
SELECT COUNT(DISTINCT p2.Size) AS SizeCount
      , p2.ProductLine
FROM Production.Product AS p2
WHERE p2.ProductLine IS NOT NULL
      AND p2.Size IS NOT NULL
GROUP BY p2.ProductLine
HAVING COUNT(DISTINCT p2.Size) > 4;
```

В результате получим следующий набор данных (рис. 112).

Results			Messages
	SizeCount	ProductLine	
1	10	M	
2	12	R	
3	5	S	
4	7	T	

**Рис. 112. Линейки товаров (ProductLine),
которые имеют более четырех размеров**

Запишем итоговый запрос, применяя измененный подзапрос.

```
SELECT p1.[Name]
      , p1.ProductLine
      , p1.Size
FROM Production.Product AS p1
WHERE p1.ProductLine IS NOT NULL
      AND p1.Size IS NOT NULL
```

```

AND
(
    SELECT COUNT(DISTINCT p2.Size) AS SizeCount
    FROM Production.Product AS p2
    WHERE p1.ProductLine = p2.ProductLine
    AND p2.Size IS NOT NULL
) > 4;

```

В результате выполнения данного запроса получим список товаров, для которого в представленных линиях продуктов имеется более четырех размеров (рис. 113).

Results Messages			
	Name	ProductLine	Size
1	HL Mountain Frame - Silver, 42	M	42
2	HL Mountain Frame - Silver, 44	M	44
3	HL Mountain Frame - Silver, 48	M	48
4	HL Mountain Frame - Silver, 46	M	46
5	HL Mountain Frame - Black, 42	M	42
6	HL Mountain Frame - Black, 44	M	44
7	HL Mountain Frame - Black, 48	M	48
8	HL Mountain Frame - Black, 46	M	46
9	HL Mountain Frame - Black, 38	M	38
10	HL Mountain Frame - Silver, 38	M	38
11	Mountain-100 Silver, 38	M	38
12	Mountain-100 Silver, 42	M	42

Рис. 113. Результат выполнения подзапроса (пример 90)

Предикат EXISTS

Иногда возникают ситуации, когда от подзапроса требуется лишь установить сам факт наличия или отсутствия строк в конкретной таблице, удовлетворяющих определенному условию, а непосредственные значения атрибутов в этих строках интереса не представляют. В подобных случаях используют предикат EXISTS (или NOT EXISTS). Поскольку от подзапроса в предикате EX-

ISTS требуется только установить факт наличия или отсутствия строк, соответствующих критерию отбора, то в документации рекомендуется делать так:

```
SELECT Field1  
      , ...  
FROM Table  
WHERE (NOT) EXISTS  
      (SELECT 1  
       FROM Table ...);
```

СПИСОК ЗАДАНИЙ [16]

БД «Аптека»

Описание предметной области. База данных (БД) создаётся для информационного обслуживания посетителей аптеки. В аптеку города поступает ассортимент лекарств со склада каждые семь дней. Аптека предлагает услуги по продаже лекарств и их бронированию. Срок бронирования лекарств — три дня. В справочной аптеки можно получить информацию о лекарствах, находящихся в аптеке: название, форма выпуска, срок годности, аннотация, цена, изготовитель.

Готовые запросы:

- выдавать данные о лекарствах.
- предоставлять покупателям возможность бронирования лекарств сроком на три дня;
- выдавать информацию о поступлении лекарства в данную аптеку исходя из ассортимента на складе;
- выдавать информацию о продажах за неделю (месяц, год) данного лекарства;
- выполнять поиск лекарства по названию, форме выпуска, изготовителю;
- выдавать список лекарств, применяемых для выбранной болезни (легких недугах).

БД «Поликлиника»

Описание предметной области. База данных создаётся для информационного обслуживания регистрационного отдела поликлиники. БД должна содержать информацию о врачах, ведущих прием, расписании приема и пациентах, проживающих на участке, закрепленном за данной поликлиникой.

Готовые запросы:

- выдавать сводную информацию обо всех врачах поликлиники;
- выдавать сводную информацию о пациентах;
- выдавать информацию о записи пациента к врачу;
- выдавать информацию о приеме врачей на указанную дату;
- выдавать информацию о пациентах, имеющих льготы на приобретение лекарств.

БД «ВУЗ»

Описание предметной области. База данных создаётся для информационного обслуживания работников деканата. В вузе учатся студенты, которые разбиты на группы. По каждой специальности обучается несколько групп. Каждая кафедра ведет занятия по нескольким дисциплинам. Занятия для студентов проводят преподаватели с разных кафедр. Лабораторные работы ведут два преподавателя.

Готовые запросы:

- выдавать информацию о студенте по № зачетной книжки, по Ф. И. О.;
- выдавать список предметов, читаемых данной кафедрой;
- выдавать список преподавателей, проводящих занятие в данной группе;
- выдавать список групп, обучающихся на данной специальности.

БД «Складской учет»

Описание предметной области. База данных создаётся для информационного обслуживания склада. Некоторая фирма имеет склад товаров. Эти товары фирма получает от производителя и расфасовывает их для магазинов. Также данная фирма имеет возможность оптового отпуска ассортимента товаров. Для постоянных клиентов предусмотрена система скидок.

Готовые запросы:

- выдавать ассортимент товара, находящегося на складе сейчас;
- выдавать ассортимент товара, заказанного данным магазином;
- показывать список продаж за указанный период времени;
- показывать список клиентов, имеющих скидку;

БД «Отдел кадров»

Описание предметной области. База данных создаётся для информационного обслуживания работников отдела кадров. В фирме работают сотрудники на разных должностях и разных ставках. Сотрудники могут иметь детей, быть пенсионерами или иметь инвалидность. Сотрудникам предоставляется очередной отпуск. Некоторые сотрудники могут быть в отпуске по уходу за ребенком.

Готовые запросы:

- показывать список сотрудников фирмы по Ф. И. О., по должности, ставке, имеющих детей, пенсионеров, находящихся в очередном отпуске, находящихся в отпуске по уходу за ребенком;
- выдавать информацию о должностях сотрудников;
- выдавать информацию о предыдущих местах работы сотрудников;
- находить сотрудников по Ф. И. О.

БД «Кафедра»

Описание предметной области. База данных создаётся для информационного обслуживания работников кафедры. БД должна содержать информацию о преподавателях и инженерах, работающих на кафедре, и их занятости (научно-исследовательских работах (НИР) и дисциплинах).

Готовые запросы:

- выдавать сводную информацию обо всех работниках кафедры;
- выдавать информацию о НИР;

- выдавать информацию о преподавателе, ведущем указанный вид занятий по указанной дисциплине;
- выдавать информацию о видах занятий, которые проводятся по выбранной дисциплине.

БД «Больница»

Описание предметной области. База данных создаётся для информационного обслуживания медицинских работников и посетителей больницы. БД должна содержать информацию о медицинских работниках больницы, лечащихся у них пациентах и нахождения пациентов по палатам.

Готовые запросы:

- выдавать сведения о медицинских работниках;
- выдавать сведения о медицинских работниках, наблюдающих за определенным пациентом;
- выдавать сведения о пациентах;
- выдавать сведения о пациентах, наблюдаемых у определенного медицинского работника;
- выдавать нахождение пациентов по палатам.

БД «Контора адвоката»

Описание предметной области. БД создается для информационного обслуживания сотрудников конторы. Контора оказывает юридические услуги (ведение дела в суде, консультация) по разным делам (гражданские, уголовные). Каждый адвокат специализируется в одной области (жилищные, семейные, убийства и т. д.). На каждое обращение в контору заводится Дело (№, содержание, адвокат, клиент).

Готовые запросы:

- показывать список предоставляемых услуг и их цену;
- выдавать список клиентов, обращавшихся за данной услугой;

- выдавать список свободных адвокатов по выбранной услуге;
- выдавать содержание дела по его номеру.

БД «Архив»

Описание предметной области. БД создается для информационного обслуживания сотрудников архива уголовного розыска. В архиве находятся Дела за разные годы.

Готовые запросы:

- выдавать список дел по Ф. И. О. осужденного;
- выдавать список дел, которые вел данный следователь;
- находить дела по содержанию;
- выдавать список дел по данной статье преступления (по характеру преступления).

БД «Продажа билетов»

Описание предметной области. БД создается для информационного обслуживания сотрудников кинотеатра (кассиров). В кинотеатре имеется несколько залов, в которых одновременно могут идти разные фильмы. Каждый фильм могут показывать несколько раз в день по сеансам (установленные часы). На фильм может налагаться ограничение по возрасту.

Готовые запросы:

- показывать сеансы на данное время, если имеются ограничения по возрасту, то сообщать об этом;
- показывать список фильмов, идущих сегодня в кинотеатре, по залам;
- выдавать информацию о цене билета на данный сеанс и данный фильм;
- выдавать информацию о кратком содержании, задействованных актерах и режиссерах данного фильма;
- продавать билет на выбранный сеанс данного фильма;
- отражать продажи по сеансам.

БД «Магазин обуви»

Описание предметной области. БД создается для информационного обслуживания сотрудников магазина. В магазине имеется несколько отделов обуви (мужская, женская, детская, спортивная и т. д.). Обувь имеет размер, цвет, фасон, цену, разделяется на зимнюю, летнюю, демисезонную и т. д., нумеруется артикулом. Для постоянных клиентов предусмотрена система скидок в виде карты.

Готовые запросы:

- выдавать информацию о количестве пар по заданному артикулу, размеру, цвету и т. д.;
- выдавать ассортимент обуви по отделам, сезону;
- выдавать информацию о данной обуви (цена, страна-изготовитель);
- продавать выбранный товар;
- показывать продажи за выбранный период времени.

БД «Библиотека»

Описание предметной области. БД создается для домашнего обслуживания библиотеки. Библиотека содержит книги разных авторов, изданий, тематики. БД предназначена для поиска выбранной книги на полке.

Готовые запросы:

- выдавать список книг по названию;
- выдавать список трудов данного автора (учитывать труды, выполненные в соавторстве);
- выдавать список книг по данной тематике;
- выдавать список книг данного издательства;
- выдавать местонахождение данной книги.

БД «Издательство»

Описание предметной области. БД создается для информационного обслуживания сотрудников издательства вуза. Издательство печатает книги, журналы, пособия и тому подобное студентов и преподавателей данного вуза. Каждый автор может написать работу в соавторстве.

Готовые запросы:

- выдавать список трудов данного автора (учитывать труды, выполненные в соавторстве);
- выдавать список трудов по выбранному разделу (книги, журналы, методические указания, пособия и т. д.);
- выдавать информацию о данном авторе;
- показывать количество напечатанных работ по кафедре, институту или университету в целом за отчетный период (квартал, год).

БД «Автосервис»

Описание предметной области. БД создается для информационного обслуживания сотрудников автосервиса. Автосервис предлагает различные услуги по ремонту, покраске, мойке и химчистке, тюнингу автомобилей и т. д. В автосервисе работают сотрудники, специализирующиеся на определенных видах работ. Каждый специалист может принять несколько машин. Каждая машина, находящаяся в сервисе, может иметь несколько проблем. У одного клиента может быть несколько машин. Для постоянных клиентов предусмотрена система скидок.

Готовые запросы:

- выдавать список услуг, предлагаемых автосервисом с ценой;
- выдавать список машин, находящихся в автосервисе;
- выдавать информацию о данной машине (оказываемые услуги);

- выдавать информацию о проделанной данным мастером работе за отчетный период времени (день, месяц, квартал, год);
- рассчитывать стоимость услуг для клиентов.

БД «Транспортная компания»

Описание предметной области. БД создается для информационного обслуживания сотрудников компании. Компания предоставляет услуги по доставке грузов воздушным, речным, морским путем, по железной дороге или автомобилем по различным маршрутам.

Готовые запросы:

- выдавать список маршрутов, обслуживаемых компанией, и цену на них;
 - выдавать список транспорта, занятого на данном маршруте;
 - выдавать список вариантов проезда по данному маршруту (морской и т. д.);
 - рассчитывать стоимость услуг, оказанных по перевозке данного груза.
- Постоянным клиентам предоставляется скидка;
- показывать грузооборот по данному маршруту.

БД «Туристическое бюро»

Описание предметной области. БД создается для информационного обслуживания сотрудников фирмы. Фирма оказывает услуги в туристическом бизнесе. Клиентам предлагаются путевки в разные страны, города. Путевки отличаются содержанием программы (отдых, экскурсии, туризм и т. д.), имеется возможность выбора путевки по цене (в зависимости от места проживания, все включено и т. д.). Для постоянных клиентов и детей до 12 лет имеются скидки.

Готовые запросы:

- выдавать список стран;
- выдавать список городов;

- покупать путевку в выбранное место с расчетом ее стоимости;
- показывать весь ассортимент путевок в данное место;
- выбирать путевку по содержанию, по цене и т. д.;
- показывать список самых популярных путевок (по месту пребывания, по содержанию, в целом).

БД «ГАИ»

Описание предметной области. БД создается для информационного обслуживания работников ГАИ. В БД находятся автомобили, зарегистрированные по данному адресу. Некоторые из них угнаны.

Готовые запросы:

- выдавать информацию об автомобиле по его регистрационному знаку (марка, цвет, модель и т. д.);
- выдавать информацию об автовладельце по регистрационному знаку данного автомобиля;
- выдавать информацию об автомобиле (прошлые автовладельцы, аварии и т. д.) по номеру двигателя;
- выдавать список угнанных автомобилей;
- выдавать список автомобилей, попавших в аварию в данный период времени;
- выдавать список наиболее угоняемых автомобилей по марке.

БД «Гостиница»

Описание предметной области. БД создается для информационного обслуживания сотрудников гостиницы и клиентов. В гостинице имеются номера, которые различаются по количеству мест, количеству комнат, удобствам и т. д., от всех этих параметров зависит цена номера (цена за номер будет выше). Посетители могут занимать номера, бронировать их заранее. Постоянным посетителям и детям до 12 лет, заселяющимся с родителями, предоставляется скидка.

Готовые запросы:

- предоставлять (и бронировать) посетителям номер в гостинице в зависимости от потребностей посетителя (количество мест и т. д.);
- показывать количество свободных (занятых) мест для данного типа номера (например, в виде диаграммы);
- показывать список постоянных посетителей и предоставляемую им скидку;
- показывать информацию по данному номеру (количество мест, комнат, холодильник, телевизор, бар и т. д.).

БД «Строительная компания»

Описание предметной области. БД создается для информационного обслуживания клиентов компании. Компания строит дачные домики, бани и т. д. Имеются готовые проекты с фиксированной ценой, проекты на заказ стоят дороже на $x\%$. На объект выезжает бригада, состоящая из рабочих и мастера.

Готовые запросы:

- показывать список предоставляемых услуг и цену на них;
- показывать количество свободных (занятых) бригад;
- показывать работу, проводимую данной бригадой;
- показывать список работ, проводимых у данного заказчика;
- рассчитывать стоимость выполненных услуг для данного заказчика.

БД «Компания по услугам связи»

Описание предметной области. БД создается для информационного обслуживания сотрудников и абонентов компании. Компания предоставляет услуги связи. Каждый абонент обслуживается по выбранному тарифу. Каждый тариф имеет свой набор услуг и стоимость за единицу услуги.

Готовые запросы:

- показывать список услуг;

- показывать список тарифов и цену за единицу услуги;
- показывать список абонентов по данному тарифу;
- показывать список самых распространенных тарифов;
- показывать процентное распределение тарифов среди абонентов.

БД «Школа»

Описание предметной области. БД создается для информационного обслуживания администрации школы. В школе учатся дети, разбитые на классы. У каждого класса свой классный руководитель. Каждый учитель имеет свое направление, вести он может шесть уроков день максимум, по данному направлению — один учитель.

Готовые запросы:

- показывать список учащихся данного класса;
- показывать классного руководителя данного класса;
- показывать занятость данного учителя;
- показывать успеваемость данного ученика;
- показывать список учеников, учащихся без троек.

БД «Детский сад»

Описание предметной области. БД создается для информационного обслуживания администрации детского сада. В сад ходят дети с двух до семи лет. Тем, кому меньше трех лет в сентябре, ходят в ясли. В каждой группе имеются два воспитателя (один — утром, другой — вечером) и нянечка. Детей в группе не больше 15 человек. С детьми проводятся занятия музыки, физкультуры и рисования в отдельных комнатах по два раза в неделю. Обязательно с детьми два раза в день гуляют.

Готовые запросы:

- показывать список детей данной группы;
- показывать список детей данного возраста;

- показывать занятость данного педагога;
- показывать занятость данной группы в данный день недели;
- показывать процентное отношение мальчиков и девочек в данной группе;
- находить название (№) группы по Ф. И. О. ребенка.

БД «Кондитерская фабрика»

Описание предметной области. БД создается для информационного обслуживания администрации фабрики. Фабрика изготавливает кондитерские изделия. Для изготовления товара требуются продукты, которые фабрика заказывает у поставщиков. Готовые товары расфасовываются для магазинов.

Готовые запросы:

- показывать список магазинов, заказывающих данный товар;
- показывать список продуктов, заказываемых у данного поставщика;
- показывать ассортимент данного товара и цену;
- выбирать наиболее популярный вид данного товара;
- показывать стоимость произведенного товара за отчетный период времени.

БД «Хлебопекарня»

Описание предметной области. БД создается для информационного обслуживания сотрудников хлебопекарни. Хлебопекарня выпекает изделия на заказ. Продукты заказывают у поставщиков, выпечку отправляют в магазины города.

Готовые запросы:

- показывать ассортимент товара и его цену;
- показывать сумму (количество) заказываемого продукта у данного поставщика за отчетный период;
- показывать список товара (общую сумму заказа, сумму заказа данного товара), заказанного данным магазином;

- показывать количество произведенного товара (за отчетный период, по названию).

БД «Компания по продаже недвижимости»

Описание предметной области. БД создается для информационного обслуживания сотрудников и клиентов фирмы. Компания покупает и продает дома и квартиры. С каждой сделки компания имеет процент ($x\%$). Клиенты также покупают и продают свои дома и квартиры.

Готовые запросы:

- показывать все предложения на данный вид недвижимости;
- показывать спрос на данный вид недвижимости;
- показывать сведения о клиенте по номеру его паспорта (Ф. И. О.);
- находить спрос (предложение) по выбранной цене, расположению, площади;
- показывать прибыль компании (без учета налогов и т. д.) за отчетный период;
- показывать список самых популярных запросов (предложений).

БД «Овощной магазин»

Описание предметной области. БД создается для информационного обслуживания сотрудников магазина. Магазин получает товар со склада и расфасовывает его для продажи. Некоторые овощи могут быть испорчены.

Готовые запросы:

- показывать ассортимент товара на выбранный день;
- показывать приход-расход выбранного товара;
- показывать прибыль магазина (без учета налогов и т. д.) по указанному товару (за отчетный период времени);
- показывать в процентном отношении количество хорошего и испорченного товара для выбранного товара;

- показывать количество проданного данного товара за отчетный период.

БД «Мебельный магазин»

Описание предметной области. БД создается для информационного обслуживания сотрудников магазина. Магазин продает товары, изготовленные на фабриках, принимает заказы на изготовление мебели по индивидуальному проекту (с увеличением стоимости на $x\%$).

Готовые запросы:

- показывать ассортимент предлагаемой мебели и цену товара;
- показывать количество проданной мебели выбранного образца за отчетный период времени;
- показывать список сделанных заказов за отчетный период времени;
- рассчитывать стоимость заказа клиента по индивидуальному проекту.

БД «Магазин музыкальных произведений»

Описание предметной области. БД создается для информационного обслуживания клиентов магазина. Покупатели самостоятельно выбирают и находят понравившийся им товар.

Готовые запросы:

- показать все произведения данного композитора (исполнителя), имеющиеся в магазине;
- показать местоположение выбранного произведения;
- показать список носителей для выбранного произведения;
- показать список произведений по жанру исполнения;
- находить произведение по названию, году выпуска, альбому и т. д.;
- показывать список произведений данного композитора (исполнителя) по выбранным годам творчества.

БД «Магазин „Детские товары“»

Описание предметной области. БД создается для информационного обслуживания сотрудников магазина. Магазин продает детские товары разного вида: одежда, обувь, питание, игрушки, письменные принадлежности, предметы по уходу за телом и т. д. По каждому виду имеется богатый ассортимент товаров. Постоянным клиентам и покупателям, сумма покупки которых превысила x руб., предоставляется система скидок.

Готовые запросы:

- находить товар по его названию, показывать отдел, где его можно приобрести;
- показывать ассортимент выбранного отдела;
- показывать товарооборот по выбранным отделам (сумму продаж);
- рассчитывать стоимость покупки, учитывая скидки;
- показывать наличие выбранного размера (цвета, фасона) для обуви и одежды.

БД «Фильмотека»

Описание предметной области. БД создается для информационного обслуживания сотрудников и посетителей магазина. Магазин продает различные виды фильмов (на носителях различного вида).

Готовые запросы:

- найти фильм по названию (по жанру, исполнителям, режиссеру);
- показывать список возможных носителей для выбранного фильма;
- показывать список фильмов по жанрам;
- показывать информацию по выбранному фильму (жанр, исполнители, режиссер, краткое содержание);
- показывать список наиболее продаваемых фильмов;

- показывать количество проданных фильмов за отчетный период (сумму продаж).

БД «Магазин по продаже оргтехники»

Описание предметной области. БД создается для информационного обслуживания сотрудников магазина. Магазин реализует товар. Постоянным клиентам и покупателям, которые приобрели товар на сумму x руб., предоставляется скидка.

Готовые запросы:

- показывать ассортимент выбранного товара и его цену;
- находить товар по названию (по фирме-изготовителю);
- рассчитывать стоимость покупки, учитывая скидку;
- показывать количество (стоимость) проданного товара (по выбранному товару, по магазину в целом) за отчетный период.

БД «Спортивная команда»

Описание предметной области. БД создается для информационного обслуживания администратора команды. Женский баскетбольный клуб, играющий в суперлиге в дивизионе А, состоит из четырех тренеров (главный и второй тренеры для основного и дублирующего состава), 24 игроков (12 — основной состав и 12 — дубль), врача, массажиста и администратора команды. На каждого члена команды имеется досье (Ф. И. О., возраст, рост, вес, звание (м/с, заслуженный тренер СССР или России и т. д.), амплуа). Результаты каждого матча фиксируются в таблицу индивидуально (фолы, подборы (свой/чужой щит), перехваты, потери, очки). Команда играет в чемпионате России и Евролиге.

Готовые запросы:

- выводить счет данного матча для основного состава (дубля);
- показывать список команд чемпионата России (Евролиги);

- выдавать сведения о данном игроке;
- выдавать список игроков для двух составов для данного амплуа (центральной, разыгрывающий, защитник);
- выдавать список игроков выбранной команды с тренерами;
- показывать результат данного матча по данному игроку (команде в целом).

ПРИМЕР ПРОЕКТИРОВАНИЯ РЕЛЯЦИОННОЙ БАЗЫ ДАННЫХ

Инфологическое проектирование

Анализ предметной области

В качестве примера возьмем базу данных компании, которая занимается продажей авиабилетов.

База данных создаётся для информационного обслуживания пассажиров и менеджеров авиакомпании. БД должна содержать данные о маршрутах, по которым летают самолеты данной авиакомпании, о рейсах, летающих по данному маршруту, сведения о пассажирах, купивших билет на рейс, сведения о наличии билетов на заданный рейс.

В соответствии с предметной областью система строится с учётом следующих особенностей.

1. По одному маршруту летает несколько рейсов.
2. Билет пассажир может купить, если на рейс остались места.
3. Пассажир может купить на один рейс только один билет.

С целью упрощения БД не вводится сущность самолет, а ответственность за продажу билетов ложится на сущность рейс.

Выделим базовые сущности этой предметной области:

- билет — номер, маршрут, рейс, дата и время отправления, дата продажи, паспорт и Ф. И. О. пассажира, ряд, место;
- пассажир — паспорт, Ф. И. О. и др.
- рейс — номер, маршрут, дата и время вылета, количество имеющихся билетов, максимальное количество билетов;
- маршрут — название маршрута, которое образуется соединением пунктов назначения (аэропортов);
- аэропорт (пункт назначения) — название аэропорта.

Разрабатываемая БД является учебной, поэтому некоторые сущности были упрощены.

ER-диаграмма БД, выполненная в графическом редакторе MS Visio, приведена на рисунке 114, внешние ключи не выделены.

На данной ER-диаграмме присутствуют связи «один-ко-многим». Такие связи легко реализуются средствами СУБД.

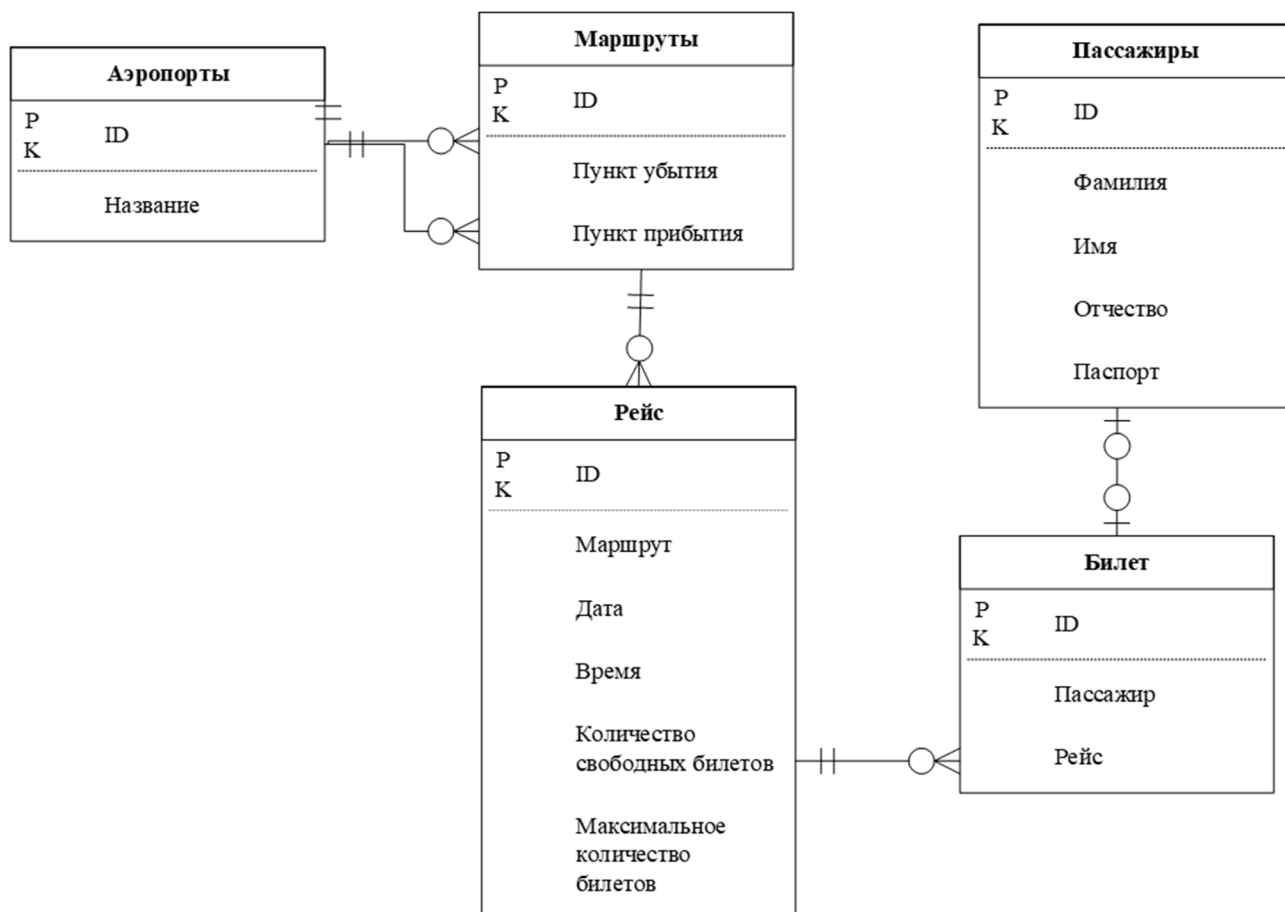


Рис. 114. ER-диаграмма

Если в полученной ER-диаграмме присутствует связь «многие-ко-многим», то вводится специальная вспомогательная сущность, которая является соединением первичных ключей соответствующих сущностей. Таким образом разбиваются все связи типа $n : m$. Обычно такая сущность именуется `НазваниеСущностиСлева_НазваниеСущностиСправа`. Затем строится уточненная ER-диаграмма.

Анализ информационных задач и круга пользователей системы

Система создаётся для обслуживания следующих групп пользователей:

- администрация (дирекция);
- менеджеры;

Определим границы информационной поддержки пользователей:

1) *функциональные возможности:*

- ведение БД (запись, чтение, модификация, перемещение в архив);
- обеспечение логической непротиворечивости БД;
- обеспечение защиты данных от несанкционированного или случайного доступа (определение прав доступа);
- реализация наиболее часто встречающихся запросов в готовом виде;

2) *готовые запросы:*

- ✓ выдавать сведения о рейсах, летающих по данному маршруту;
- ✓ выдавать список пассажиров, улетевших на данном рейсе;
- ✓ выдавать список билетов, которые купил данный пассажир;
- ✓ показывать количество проданных билетов для данного рейса.

Логическое проектирование реляционной БД

Преобразование ER-диаграммы в схему базы данных

База данных создаётся на основании схемы базы данных. Преобразование ER-диаграммы в схему БД выполняется путем сопоставления каждой сущности и каждой связи, имеющей атрибуты, отношения (таблицы БД).

На схеме, представленной на рисунке 115, введены обозначения.

Полученная схема реляционной базы данных (РБД) авиакомпании приведена ниже (рис. 116).

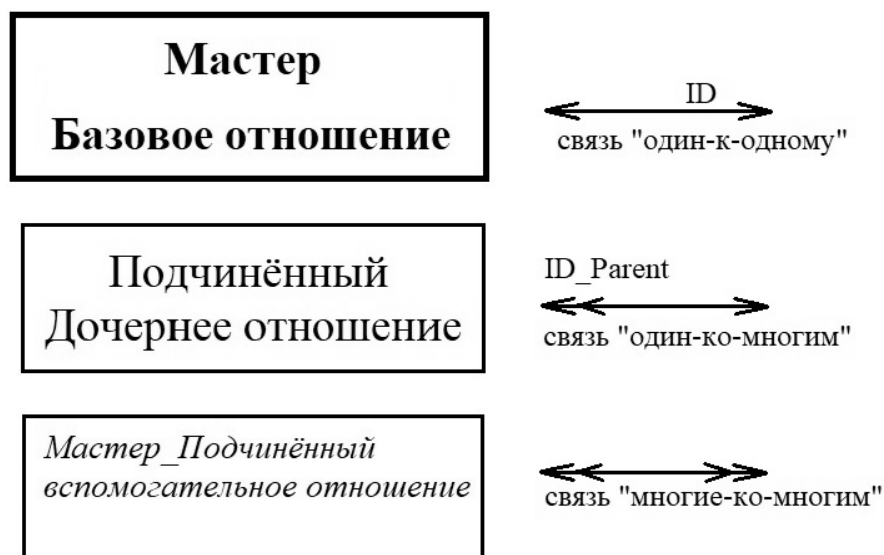


Рис. 115. Обозначения, используемые на схеме базы данных

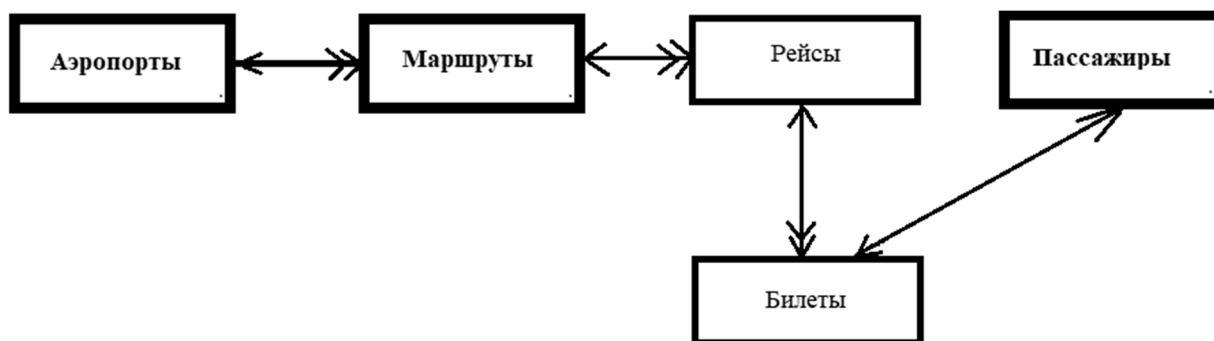


Рис. 116. Схема РБД, полученная из ER-диаграммы компании

Связь типа 1 : n (один-ко-многим) между отношениями реализуется через внешний ключ. Ключ вводится для того отношения, к которому осуществляется множественная связь.

На схеме присутствует связь 1 : 1 (один-ко-одному). Такая связь также реализуется через внешний ключ, но вводится ограничение на количество записей в дочернем отношении.

Составление реляционных отношений

Каждое реляционное отношение соответствует одной сущности (объекту ПрО), и в него вносятся все атрибуты сущности. Для каждого отношения необходимо определить первичный ключ и внешние ключи (если они есть). В том

случае, если базовое отношение не имеет потенциальных ключей, вводится *суррогатный первичный ключ*, который не несёт смысловой нагрузки и служит только для идентификации записей.

Примечание: суррогатный первичный ключ также может вводиться в тех случаях, когда потенциальный ключ имеет большой размер (например, длинная символьная строка) или является составным (не менее трёх атрибутов).

Для сущности **Аэропорты (Points)** потенциальным ключом является атрибут «Название» (Name), так как его значение является уникальным. Но значениями этого атрибута будут символьные строки. Введем для этой сущности суррогатный ключ ID.

Для сущности **Маршрут (Route)** нет потенциального ключа, так как совокупность атрибутов «Пункт убытия» (ID_PointDeparture) и «Пункт прибытия» (ID_PointArrival), из которых складывается маршрут, не является уникальной. Для единообразия именования для этой сущности в качестве ключевого атрибута введем суррогатный ключ ID.

Сущность **Рейс (Flight)**. Для нее уникальным атрибутом является «номер рейса» (ID), поэтому в качестве уникального атрибута выбран именно он.

Для сущности **Билет (Ticket)**, ключевым атрибутом выбран «номер билета» (ID), который, по мнению разработчика, не должен повторяться внутри БД (является уникальным).

Для сущности **Пассажир (Person)** уникальным атрибутом является паспорт. Однако серия и номер паспорта — составной ключ. Для единообразия именования для этой сущности в качестве ключевого атрибута введем суррогатный ключ ID.

Отношения приведены в таблицах 5–9. Для каждого отношения указаны атрибуты с их внутренним названием, типом и длиной. Для единообразия для ключевого атрибута возьмем тип данных «целое число».

Таблица 5. Схема отношения Аэропорты (Points)

<i>Содержание поля</i>	<i>Имя поля</i>	<i>Тип, длина</i>	<i>Примечания</i>
Первичный ключ	ID	Целое число	Первичный ключ
Название	Name	Строка (30)	Уникальное, обязательное поле

Таблица 6. Схема отношения Маршруты (Routes)

<i>Содержание поля</i>	<i>Имя поля</i>	<i>Тип, длина</i>	<i>Примечания</i>
Первичный ключ	ID	Целое число	Первичный ключ
Пункт убытия	ID_PointDeparture	Целое число	Обязательное поле, внешний ключ к отношению Аэропорты
Пункт прибытия	ID_PointArrival	Целое число	Обязательное поле, внешний ключ к отношению Аэропорты

Таблица 7. Схема отношения Рейсы (Flights)

<i>Содержание поля</i>	<i>Имя поля</i>	<i>Тип, длина</i>	<i>Примечания</i>
Номер рейса	ID	Целое число	первичный ключ
Маршрут следования	ID_Route	Целое число	Внешний ключ к отношению Маршруты
Дата вылета	[Date]	Дата, время	Обязательное поле
Время вылета	[Time]	Дата, время	Обязательное поле
Количество билетов, проданных на данный рейс	TicketsCount	Целое число	Обязательное поле, не меньше нуля, не больше максимального количества билетов
Максимальное количество билетов	MaxTicketsCount	Целое число	Обязательное поле, больше нуля

Таблица 8. Схема отношения Пассажиры (Persons)

<i>Содержание поля</i>	<i>Имя поля</i>	<i>Тип, длина</i>	<i>Примечания</i>
Первичный ключ	ID	Целое число	Первичный ключ
Паспортные данные	Passport	Строка (50)	Уникальное, обязательное поле
Фамилия	FamilyName	Строка (30)	Обязательное поле
Имя	FirstName	Строка (30)	Обязательное поле
Отчество	PatronymicName	Строка (30)	

Таблица 9. Схема отношения Билеты (Tickets)

<i>Содержание поля</i>	<i>Имя поля</i>	<i>Тип, длина</i>	<i>Примечания</i>
Номер билета	ID	Целое число	Первичный ключ
Номер рейса	ID_Flight	Целое число	Внешний ключ к отношению Рейсы
Паспортные данные	ID_Person	Целое число	Внешний ключ к отношению Пассажиры

Нормализация полученных отношений

1НФ. Для приведения таблиц к 1НФ требуется составить таблицы (один атрибут — один столбец) и разбить сложные атрибуты на простые, а многозначные атрибуты вынести в отдельные отношения.

Примечание. В реальных БД сложные атрибуты разбиваются на простые, если:

- а) этого требует внешнее представление данных;
- б) в запросах поиск может осуществляться по отдельной части атрибута.

Разделим атрибут Паспортные данные в отношении **Пассажиры** на атрибуты Номер и серия паспорта, Дата выдачи и Кем выдан.

2НФ. Все отношения находятся во 2НФ, так как для каждого из них введен уникальный ключ.

3НФ. В отношении **Пассажиры** атрибуты Дата выдачи и Кем выдан не зависят от первичного ключа. Отношение не находится в 3НФ. Чтобы отношение находилось в 3НФ, нужно данное отношение разбить на три: **Пассажир**, **Дата_Выдачи**, **Кем_Выдан**. Однако такое разбиение приведет к неоправданному усложнению работы с БД. Запросы по удалению и модификации данных в этих атрибутах в рамках данной предметной области не будут никогда проводиться.

В реальных базах данных после нормализации может проводиться денормализация. Она проводится с одной целью — повышение производительности БД.

Окончательная схема отношения **Пассажиры** базы данных с указанием ключей и других ограничений целостности приведена в таблице 10.

Таблица 10. Схема отношения Пассажиры (Persons)

<i>Содержание поля</i>	<i>Имя поля</i>	<i>Тип, длина</i>	<i>Примечания</i>
Первичный ключ	ID	Целое число	Первичный ключ
Номер и серия паспорта	PassportNumber	Строка (10)	Уникальное, обязательное поле
Дата выдачи	IssueDate	Дата, время	Обязательное поле
Кем выдан	Issue	Строка (50)	Обязательное поле
Фамилия	FamilyName	Строка (30)	Обязательное поле
Имя	FirstName	Строка (30)	Обязательное поле
Отчество	PatronymicName	Строка (30)	Необязательное поле

Определение дополнительных ограничений целостности

Перечислим ограничения целостности, которые не указаны в таблицах 5–10.

Вставка данных для таблицы Flights возможна при уникальности совокупности данных Маршрут — Дата — Время (может существовать только один рейс по выбранному маршруту в определенный день и время).

Покупка билета (вставка строки в таблицу Tickets) возможна на один рейс только для одного пассажира.

TicketsCount — должен обновляться автоматически при покупке билета.

Физическое проектирование БД

Разработка скриптов на создание базы данных и таблиц

БД «Система продажи авиабилетов» проектируется в среде Microsoft SQL Server Management Studio.

Создание базы данных «Авиакомпания»:

```
CREATE DATABASE Airline;
```

Создание таблицы Аэропорты:

```
USE Airline;
```

```
CREATE TABLE Points  
(  
    ID INT NOT NULL  
    , [Name] NVARCHAR(30) NOT NULL  
);
```

Создание ограничений для таблицы Аэропорты:

- добавление первичного ключа

```
ALTER TABLE Points  
ADD CONSTRAINT PK_Points  
PRIMARY KEY (ID);
```

- добавление ограничения уникальности аэропорта

```
ALTER TABLE Points
ADD CONSTRAINT UQ_Point_Name
UNIQUE ([Name]);
```

Создание таблицы Маршруты:

```
USE Airline;

CREATE TABLE [Routes]
(
    ID INT NOT NULL
    , ID_PointDeparture INT NOT NULL
    , ID_PointArrival INT NOT NULL
);
```

Создание ограничений для таблицы Маршруты:

- добавление первичного ключа

```
ALTER TABLE [Routes]
ADD CONSTRAINT PK_Routes
PRIMARY KEY (ID);
```

- добавление внешних ключей

```
ALTER TABLE [Routes]
ADD CONSTRAINT FK_Routes_PointArrival
FOREIGN KEY (ID_PointArrival)
REFERENCES Points (ID);
```

```
ALTER TABLE [Routes]
ADD CONSTRAINT FK_Routes_PointDeparture
FOREIGN KEY (ID_PointDeparture)
REFERENCES Points (ID);
```

Создание таблицы Рейсы:

```
USE Airline;

CREATE TABLE Flights
(
    ID INT NOT NULL
    , ID_Route INT NOT NULL
```

```
, [Date] DATE NOT NULL  
, [Time] TIME NOT NULL  
, TicketsCount INT NOT NULL  
, MaxTicketsCount INT NOT NULL  
);
```

Создание ограничений для таблицы Рейсы:

- *добавление первичного ключа*

```
ALTER TABLE Flights  
ADD CONSTRAINT PK_Flights  
PRIMARY KEY (ID);
```

- *добавление внешнего ключа*

```
ALTER TABLE Flights  
ADD CONSTRAINT FK_Flights  
FOREIGN KEY (ID_Route)  
REFERENCES [Routes] (ID);
```

- *максимальное количество билетов должно быть больше нуля*

```
ALTER TABLE Flights  
ADD CONSTRAINT CH_MaxTicketsCount  
CHECK (MaxTicketsCount > 0);
```

- *проданное количество билетов должно быть не больше максимального количества билетов*

```
ALTER TABLE Flights  
ADD CONSTRAINT CH_TicketsCount  
CHECK (TicketsCount >= 0  
AND TicketsCount <= MaxTicketsCount);
```

- *может существовать только один рейс по выбранному маршруту в определенный день и время*

```
ALTER TABLE Flights  
ADD CONSTRAINT UQ_Route_Date_Time  
UNIQUE (ID_Route, [Date], [Time]);
```

Создание таблицы Пассажиры:

```
USE Airline;

CREATE TABLE Persons
(
    ID INT NOT NULL
    , PassportNumber CHAR(10) NOT NULL
    , IssueDate DATETIME NOT NULL
    , Issue NVARCHAR(50) NOT NULL
    , FamilyName NVARCHAR(30) NOT NULL
    , FirstName NVARCHAR(30) NOT NULL
    , PatronymicName NVARCHAR(30)
);
```

Создание ограничений для таблицы Пассажиры:

- *добавление первичного ключа*

```
ALTER TABLE Persons
ADD CONSTRAINT PK_Persons
PRIMARY KEY (ID);
```

- *добавление ограничения уникальности номера паспорта*

```
ALTER TABLE Persons
ADD CONSTRAINT UQ_Person_PassportNumber
UNIQUE (PassportNumber);
```

Создание таблицы Билеты:

```
USE Airline;

CREATE TABLE Tickets
(
    ID INT NOT NULL
    , ID_Flight INT NOT NULL
    , ID_Person INT NOT NULL
);
```

Создание ограничений для таблицы Рейсы:

- *добавление первичного ключа*

```
ALTER TABLE Tickets
ADD CONSTRAINT PK_Tickets
PRIMARY KEY (ID);
```

- *добавление внешних ключей*

```
ALTER TABLE Tickets
ADD CONSTRAINT FK_Tickets_Flights
FOREIGN KEY (ID_Flight)
REFERENCES Flights (ID);
```

```
ALTER TABLE Tickets
ADD CONSTRAINT FK_Tickets_Persons
FOREIGN KEY (ID_Person)
REFERENCES Persons (ID);
```

- *на выбранный рейс пассажир может купить только 1 билет*

```
ALTER TABLE Tickets
ADD CONSTRAINT UQ_Flight_Person
UNIQUE(ID_Flight, ID_Person);
```

Диаграмма базы данных представлена на рисунке 117.

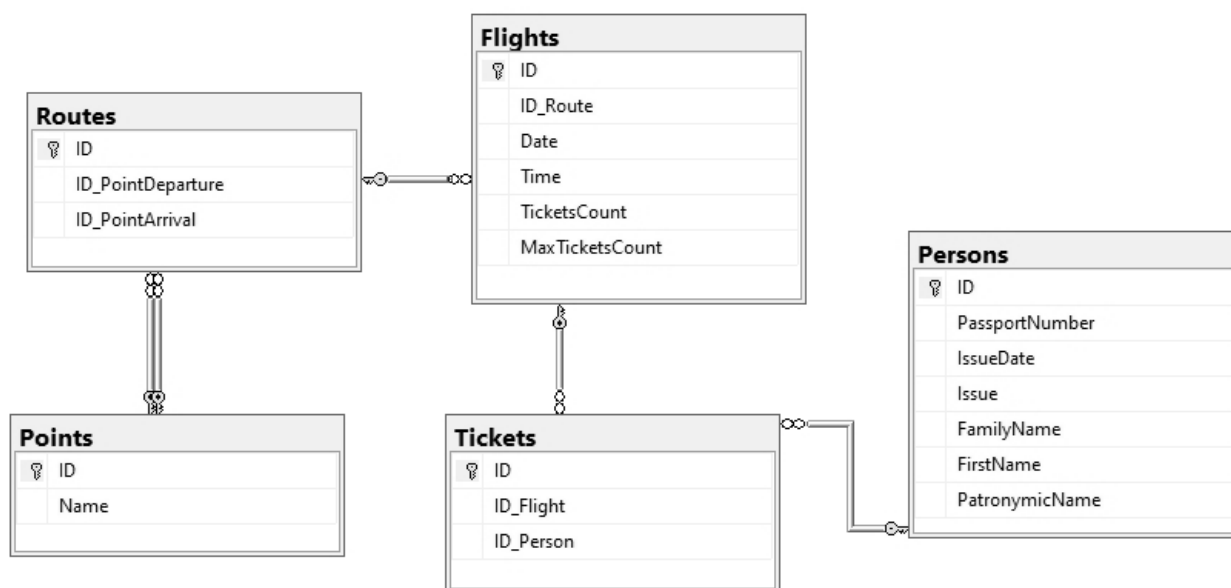


Рис. 117. Диаграмма базы данных

Разработка скриптов на добавление данных в таблицы

Заполнение таблицы Пункт назначения:

```
USE Airline;
```

```
INSERT INTO Points VALUES  
  (1, N'Москва')  
, (2, N'Санкт-Петербург')  
, (3, N'Новосибирск')  
, (4, N'Владивосток');
```

Заполнение таблицы Маршрут:

```
USE Airline;
```

```
INSERT INTO [Routes] VALUES  
  (1, 1, 2)  
, (2, 1, 3)  
, (3, 1, 4)  
, (4, 2, 1)  
, (5, 2, 3)  
, (6, 2, 4)  
, (7, 3, 1)  
, (8, 3, 2)  
, (9, 3, 4)  
, (10, 4, 1)  
, (11, 4, 2)  
, (12, 4, 3);
```

Заполнение таблицы Пассажир:

```
USE Airline;
```

```
INSERT INTO Persons VALUES  
  (1, 1111222221, '01.01.2000', N'Дмитровский отдел г. Москвы'  
, N'Иванов', N'Иван', N'Иванович')  
, (2, 1111222222, '02.01.2000', N'Дмитровский отдел г. Москвы'  
, N'Иванова', N'Светлана', N'Ивановна')  
, (3, 1111222223, '03.01.2000', N'Дмитровский отдел г. Москвы'  
, N'Петров', N'Петр', N'Петрович')  
, (4, 1111222224, '04.01.2000', N'Дмитровский отдел г. Москвы'  
, N'Петрова', N'Ольга', N'Петровна');
```

Заполнение таблицы Рейс:

```
INSERT INTO Flight VALUES
(1, 1, '20.12.2022', '10:00', 100, 100)
,(2, 2, '20.12.2022', '15:00', 100, 100)
,(3, 3, '20.12.2022', '22:00', 100, 100)
,(4, 4, '22.12.2022', '10:00', 100, 100)
,(5, 7, '22.12.2022', '15:00', 100, 100)
,(6, 10, '22.12.2022', '22:00', 100, 100);
```

При попытке вставить неправильное количество билетов возникает исключение:

```
USE Airline;
```

```
--Error on insert TicketsCount > MaxTicketsCount
```

```
INSERT INTO Flights VALUES
(8, 5, '2022-12-23', '10:00', 200, 100);
```

Messages

```
Msg 547, Level 16, State 0, Line 8
The INSERT statement conflicted with the CHECK constraint "CH_TicketsCount".
The conflict occurred in database "Airline", table "dbo.Flights".
The statement has been terminated.
```

```
Completion time: 2022-11-09T17:05:16.2410665+03:00
```

Рис. 118. Проверка ограничений

Заполнение таблицы Билет.

Заполнение данной таблицы организуем через **хранимую процедуру**, так как оно связано с выполнением ограничений.

Для выбранного рейса и пассажира необходимо вставлять строку при следующих условиях:

- 1) данный пассажир еще не покупал билет на данный рейс;
- 2) в наличии имеется билет (значение TicketsCount в таблице Flights больше нуля).

Также одновременно со вставкой строки необходимо уменьшать на единицу количество имеющихся в наличии билетов.

Разработка необходимых представлений (view)

Представление (view) *Маршрут* (просмотр всех маршрутов, по которым летают самолеты):

```
CREATE VIEW vRoutes
AS
SELECT
    r.ID AS ID
    , p1.[Name] + ' - ' + p2.[Name] AS [Route]
FROM dbo.[Routes] AS r
INNER JOIN dbo.Points AS p1
ON p1.ID = r.ID_PointDeparture
INNER JOIN Points AS p2
ON p2.ID = r.ID_PointArrival;
```

Результат выполнения следующего скрипта представлен на рисунке 119.

```
SELECT *
FROM vRoutes;
```

Результаты			Сообщения
	ID	Route	
1	1	Москва - Санкт-Петербург	
2	2	Москва - Новосибирск	
3	3	Москва - Владивосток	
4	4	Санкт-Петербург - Москва	
5	5	Санкт-Петербург - Новосибирск	
6	6	Санкт-Петербург - Владивосток	
7	7	Новосибирск - Москва	
8	8	Новосибирск - Санкт-Петербург	
9	9	Новосибирск - Владивосток	
10	10	Владивосток - Москва	
11	11	Владивосток - Санкт-Петербург	
12	12	Владивосток - Новосибирск	

Рис. 119. Выборка всех данных из представления *Маршрут*

Представление (view) Рейс (просмотр всех рейсов):

```
CREATE VIEW vFlights
AS
SELECT
    f.ID AS ID
    , p1.[Name] + ' - ' + p2.[Name] AS [Route]
    , CONVERT(VARCHAR(10),f.[Date], 104) AS [Date]
    , CONVERT(VARCHAR(8), f.[Time], 120) AS [Time]
    , f.TicketsCount
    , f.MaxTicketsCount
FROM [Routes] AS r
INNER JOIN Points AS p1
ON p1.ID = r.ID_PointDeparture
INNER JOIN Points AS p2
ON p2.ID = r.ID_PointArrival
INNER JOIN Flights AS f
ON f.ID_Route = r.ID;
```

Результат выполнения следующего скрипта представлен на рисунке 120.

```
SELECT *
FROM vFlights;
```

Results		Messages				
	ID	Route	Date	Time	TicketsCount	MaxTicketsCount
1	1	Москва - Санкт-Петербург	20.12.2022	10:00:00	100	100
2	2	Москва - Новосибирск	20.12.2022	15:00:00	100	100
3	3	Москва - Владивосток	20.12.2022	22:00:00	100	100
4	4	Санкт-Петербург - Москва	22.12.2022	10:00:00	100	100
5	5	Новосибирск - Москва	22.12.2022	15:00:00	100	100
6	6	Владивосток - Москва	22.12.2022	22:00:00	100	100
7	7	Владивосток - Москва	23.12.2022	22:00:00	0	100

Рис 120. Выборка всех данных из представления Рейс

Представление Билеты:

```
CREATE VIEW vTickets AS (
SELECT
    t.ID
    , p1.[Name] AS Departure
    , p2.[Name] AS Arrival
```

```

        , f.[Date]
        , f.[Time]
        , p.FirstName
        , p.PatronymicName
        , p.FamilyName
        , p.PassportNumber
        , f.ID AS [Flights]
FROM dbo.[Routes] AS r
INNER JOIN dbo.Points AS p1
ON p1.ID = r.ID_PointDeparture
INNER JOIN Points AS p2
ON p2.ID = r.ID_PointArrival
INNER JOIN Flights AS f
ON f.ID_Route = r.ID
INNER JOIN Tickets AS t
ON t.ID_Flight = f.ID
INNER JOIN Persons AS p
ON p.ID = t.ID_Person);

```

Так как таблица Билет еще не заполнена, то и выполнять выборку значений из представления Билеты сейчас не имеет смысла.

Для того чтобы написать процедуру покупки билета, напомним процедуры и функции, которые будут являться блоками будущей процедуры.

Разработка необходимых функций и процедур

Функция расчета количества доступных билетов на выбранный рейс:

```

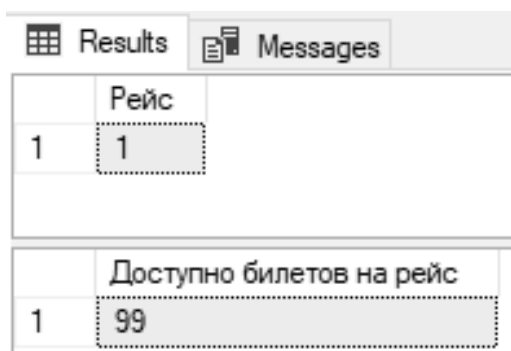
CREATE FUNCTION fCountAvailableTickets (@ID_Flight INT)
RETURNS INT
BEGIN
    DECLARE @Avaible INT
    SELECT @Avaible = TicketsCount
    FROM dbo.Flights
    WHERE ID = @ID_Flight
    RETURN @Avaible
END;

```

Выполним расчет для выбранного рейса:

```
--Доступные билеты на рейс
DECLARE @Flight INT = 1;
SELECT @Flight AS N'Рейс';
SELECT dbo.fCountAvailableTickets(@Flight)
       AS N'Доступно билетов на рейс ';
```

В результате выполнения расчета получили следующие результаты (рис. 121).



Results		Messages
	Рейс	
1	1	
	Доступно билетов на рейс	
1	99	

**Рис. 121. Результат выполнения функции
«Расчет количества доступных билетов на выбранный рейс»**

Для удобства использования написанной функции разработаем процедуру, выполняющую те же действия, но предоставляющую удобство использования.

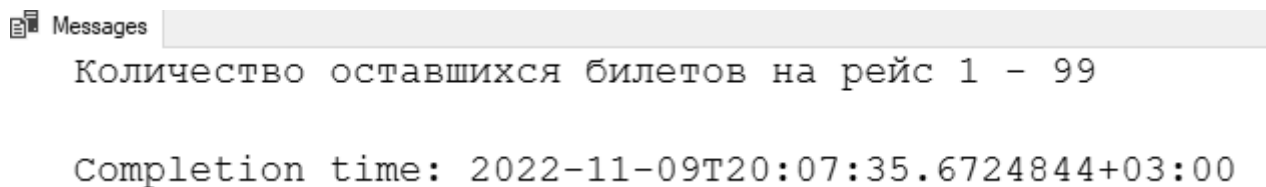
Процедура расчета количества доступных билетов.

```
CREATE PROC pPrintAvailableTicketsCount(@Flight INT) AS
BEGIN
    DECLARE @TicketsCount INT =
    dbo.fGetAvailableTicketsCount(@Flight);
    DECLARE @FlightChar NVARCHAR(10) = CAST(@Flight AS NVAR-
    CHAR(10));
    IF @TicketsCount = 0
        PRINT N'На рейс ' + @FlightChar + N' все билеты про-
        даны'
    ELSE PRINT N'Количество оставшихся билетов на рейс ' +
    @FlightChar + N' - ' + CAST(@TicketsCount AS NVARCHAR(10));
END;
```

Выполним процедуру pPrintAvailableTicketsCount:

```
--найдем текущее значение оставшихся билетов на рейс @Flight  
EXEC pPrintAvailableTicketsCount @Flight = 1;
```

Результат выполнения скрипта представлен на рисунке 122.

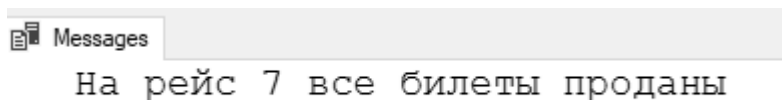


**Рис. 122. Результат выполнения процедуры
«Показать количество доступных билетов на выбранный рейс»
для рейса № 1**

Выполним процедуру для других входных параметров:

```
EXEC pPrintAvailableTicketsCount @Flight = 7;
```

Результат выполнения процедуры представлен на рисунке 123.



**Рис. 123. Результат выполнения процедуры
«Показать количество доступных билетов на выбранный рейс»
для рейса № 7**

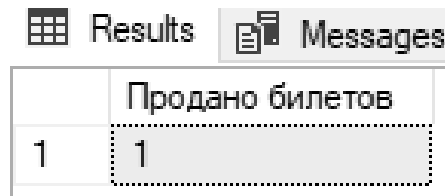
Функция расчета количества проданных билетов на выбранный рейс:

```
CREATE FUNCTION fCountSoldTickets (@ID_Flight INT)  
RETURNS INT  
BEGIN  
    DECLARE @Avaible INT  
    SELECT @Avaible = MaxTicketsCount - TicketsCount  
    FROM dbo.Flights  
    WHERE ID = @ID_Flight  
    RETURN @Avaible  
END;
```

Выполним разработанную функцию:

```
SELECT dbo.fCountSoldTickets(1) AS N'Продано билетов';
```

В результате получим следующий ответ (рис. 124).



Results		Messages	
	Продано билетов		
1	1		

**Рис. 124. Результат выполнения функции
«Расчет купленных билетов на выбранный рейс»**

Так как пассажир идентифицируется однозначно номером паспорта, а введенный суррогатный ключ ID не известен пассажиру, то напишем сначала функцию, которая по номеру паспорта выдает идентификационный номер.

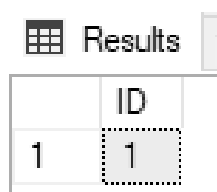
Функция расчета идентификационного номера по номеру паспорта.

```
--Получить ID человека по номеру паспорта
CREATE FUNCTION fGetIDPassagerFromPassportNumber
(@PassportNumber INT)
RETURNS INT AS
BEGIN
    DECLARE @ID INT = (
        SELECT ID
        FROM dbo.Persons
        WHERE PassportNumber = @PassportNumber)
    RETURN @ID
END;
```

Выполним написанную функцию для проверки ее функционирования:

```
DECLARE @PassportNumber INT = 111122221;
SELECT dbo.fGetIDPassagerFromPassportNumber (@PassportNumber)
AS ID;
```

Результат выполнения данной функции представлен на рисунке 125.



ID
1

**Рис. 125. Результат выполнения функции
«Показать ID пассажира по номеру его паспорта»**

Также для удобства использования напишем функцию, возвращающую полное имя пассажира по номеру его паспорта:

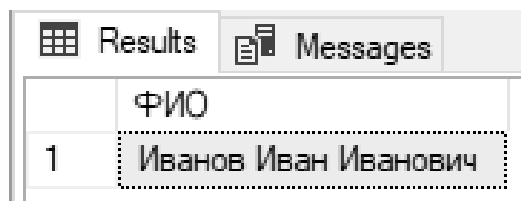
```
--Показать полное имя по паспорту
CREATE FUNCTION fGetFullNameByPassport(@PassportNumber INT)
RETURNS NVARCHAR(150)
AS
BEGIN
    DECLARE @ID INT =
    dbo.fGetIDPassagerFromPassportNumber(@PassportNumber);
    DECLARE @FullName NVARCHAR(150) = (
        SELECT FamilyName + ' ' + FirstName + ' ' + Patronym-
icName
        FROM Persons
        WHERE ID = @ID);

    RETURN @FullName;
END;
```

Выполним разработанную функцию:

```
DECLARE @PassportNumber INT = 1111222221;
DECLARE @FullName NVARCHAR(150) =
    dbo.fGetFullNameByPassport(@PassportNumber);
SELECT @FullName AS N'ФИО';
```

Результат выполнения данного скрипта представлен на рисунке 126.



Results		Messages
	ФИО	
1	Иванов Иван Иванович	

**Рис. 126. Результат выполнения функции
«Показать Ф. И. О. пассажира по номеру паспорта»**

Для удобства использования функции «Расчет купленных билетов на выбранный рейс» напишем скрипт для процедуры, выполняющей расчет количества проданных билетов для выбранного пассажира на выбранный рейс.

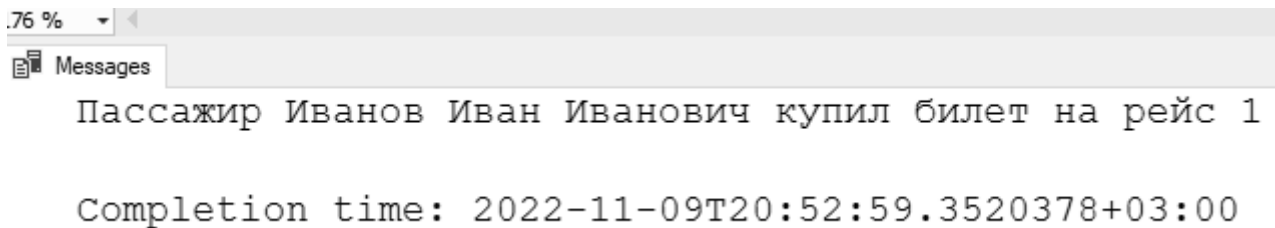
Процедура расчета количества проданных билетов для выбранного пассажира на выбранный рейс:

```
-- посчитаем количество проданных билетов
-- для пассажира @PassportNumber и рейса @Flight
CREATE PROC pGetSoldTicketsCount(
    @Flight INT
    , @PassportNumber INT) AS
BEGIN
    DECLARE @WasSold INT =
    dbo.fHowManyTicketsBoughtPassenger(@Flight, @PassportNumber);
    DECLARE @FlightChar NVARCHAR(10) = CAST(@Flight AS NVAR-
    CHAR(10));
    DECLARE @FullName NVARCHAR(150) =
    dbo.fGetFullNameByPassport(@PassportNumber);
    IF @WasSold > 0
        PRINT N'Пассажир ' + @FullName + N' купил билет на
    рейс ' + @FlightChar
    ELSE PRINT N'Пассажир ' + @FullName + N' не покупал билет
    на рейс ' + @FlightChar;
END;
```

Выполним разработанную процедуру.

```
EXEC dbo.pGetSoldTicketsCount
    @Flight = 1,
    , @PassportNumber = 1111222221;
```

Результат выполнения процедуры «Показать количество купленных билетов выбранным пассажиром на выбранный рейс» представлен на рисунке 127.

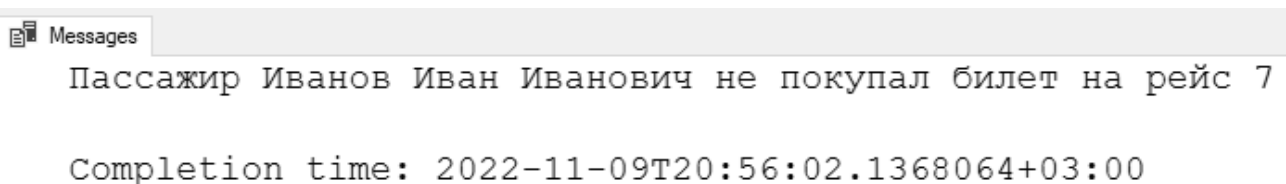


**Рис. 127. Результат выполнения процедуры
«Показать количество купленных билетов
выбранным пассажиром на выбранный рейс»
рейс № 1**

Выполним ту же процедуру для других входных параметров:

```
EXEC dbo.pGetSoldTicketsCount
    @Flight = 7
    , @PassportNumber = 1111222221;
```

Результат выполнения процедуры показан на рисунке 128.



**Рис. 128. Результат выполнения процедуры
«Показать количество купленных билетов
выбранным пассажиром на выбранный рейс»
рейс № 7**

Так как при покупке билета в таблицу «Билеты» нужно будет вставить запись, а значение идентификационного номера является уникальным (ПК), необходимо разработать функцию определения идентификационного номера последней записи.

Функция определения идентификационного номера последней записи:

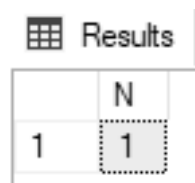
```
--найдем значение последнего ID в таблице Tickets
CREATE FUNCTION dbo.fGetMaxIDTickets()
RETURNS INT AS
BEGIN
    DECLARE @ID INT;--для автоматического заполнения ID
                        --в таблице Tickets
    SELECT @ID = MAX(ID)
    FROM Tickets;

    RETURN @ID;
END;
```

Проверим работу функции определения максимального идентификационного номера:

```
DECLARE @ID INT = dbo.fGetMaxIDTickets();
IF @ID IS NOT NULL
    SELECT @ID AS N
ELSE SELECT 0 AS N;
```

Результат выполнения данной функции представлен на рисунке 129.



N
1

**Рис. 129. Результат выполнения функции
«Показать максимальное значение идентификационного номера»**

Подготовив вышенаписанные процедуры и функции, разработаем скрипт процедуры покупки билета.

Процедура покупки билета:

```
CREATE PROC pBuyTicket(
    @Flight INT
    , @PassportNumber INT)
AS
BEGIN
```

```

BEGIN TRY
    BEGIN TRAN
        --Получаем ID человека по номеру паспорта
        --Если нет такого человека, то @PersonID =
NULL
        DECLARE @PersonID INT =
dbo.fGetIDPassagerFromPassportNumber(@PassportNumber);
        --для автоматического заполнения ID в таблице
Tickets
        DECLARE @ID INT = dbo.fGetMaxIDTickets();
        IF @ID IS NULL SET @ID = 1
        ELSE SET @ID = @ID + 1;

        --Вставляем строку в таблицу Tickets
        --Могут возникнуть ошибки:
        -- не нашли человека по паспорту (ID_Person IS
NOT NULL)
        -- пассажир уже купил билет на данный рейс
        -- ограничение UQ_Flight_Person)

        INSERT INTO Tickets VALUES (@ID, @Flight,
@PersonID);

        --найдем текущее значение оставшихся билетов
        --на рейс @Flight
        DECLARE @TicketsCount INT =
dbo.fGetAvailableTicketsCount(@Flight);
        --Уменьшаем на единицу количество оставшихся
билетов
        SET @TicketsCount = @TicketsCount - 1

        --Изменяем количество свободных билетов
        --Может возникнуть ошибка
        - Все билеты проданы (@TicketsCount < 0).
        UPDATE Flights
        SET TicketsCount = @TicketsCount
        WHERE ID = @Flight

        --не возникло ошибок - сохраняем результаты
транзакции
        COMMIT TRAN
    END TRY
    BEGIN CATCH

```

```

--возникли ошибки - откатываем транзакцию (не
купили билет)
        ROLLBACK
        RAISERROR (N'Не удалось купить билет.', 1, 1)
    END CATCH
END;

```

Выполним написанную процедуру:

```
EXEC pBuyTicket @Flight = 7, @PassportNumber = 1111222221;
```

Результат выполнения процедуры покупки билета представлен на рисунке 130.

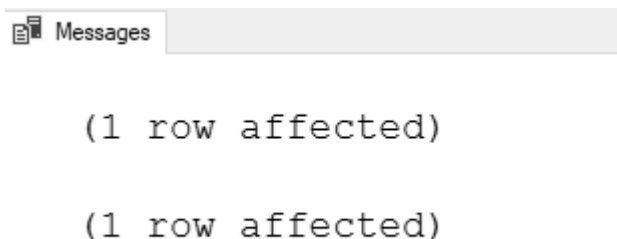


Рис. 130. Результат выполнения процедуры покупки билета

В результате выполнения процедуры должно было измениться содержимое таблиц `Flights` и `Tickets`.

Выполним следующие скрипты:

```

USE Airline;
SELECT *
FROM dbo.vFlights;

GO
USE Airline;
SELECT *
FROM dbo.vTickets;

```

Результат выполнения скриптов представлен на рисунке 131.

Results							Messages			
	ID	Route	Date	Time	TicketsCount	MaxTicketsCount				
1	1	Москва - Санкт-Петербург	20.12.2022	10:00:00	98	100				
2	2	Москва - Новосибирск	20.12.2022	15:00:00	100	100				
3	3	Москва - Владивосток	20.12.2022	22:00:00	100	100				
4	4	Санкт-Петербург - Москва	22.12.2022	10:00:00	100	100				
5	5	Новосибирск - Москва	22.12.2022	15:00:00	100	100				
6	6	Владивосток - Москва	22.12.2022	22:00:00	100	100				
7	7	Владивосток - Москва	23.12.2022	22:00:00	0	100				

	ID	Departure	Arrival	Date	Time	FirstName	PatronymicName	FamilyName	PassportNumber	Flights
1	1	Москва	Санкт-Петербург	2022-12-20	10:00:00.00000000	Иван	Иванович	Иванов	1111222221	1
2	2	Москва	Санкт-Петербург	2022-12-20	10:00:00.00000000	Светлана	Ивановна	Иванова	1111222222	1

Рис. 131. Выборка всех данных из представления Рейсы и Билеты после покупки билетов (выполнения процедуры Buy_Ticket)

Из рисунка 131 видно, что всего было куплено два билета на один рейс, при этом количество билетов также уменьшилось на два.

При повторном выполнении процедуры с теми же параметрами появляется надпись: «Не удалось купить билет» (рис. 132). При этом таблицы `Flights` и `Tickets` не изменяются (результат просмотра представлений соответствует вышеприведенному рисунку).

Messages	
(1 row affected)	
(0 rows affected)	
Не удалось купить билет.	
Msg 50000, Level 1, State 1	

Рис. 132. Результат выполнения процедуры с ошибкой «Не удалось купить билет»

Для проверки невозможности купить билет при отсутствии билетов на заданный рейс вставим строку в таблицу `Flights`, в которой значение поля `TicketsCount` будет равно нулю:

```
USE Airline;
INSERT INTO Flights VALUES
(7, 10, '2022-12-23', '22:00', 0, 100);
```

Теперь выполним процедуру покупки билета на только что вставленный рейс (ID = 7):

```
EXEC Buy_Ticket @Flight = 7, @Person = 2;
```

В результате получим ту же ситуацию (Рис.132).

Разработаем процедуру, которая покажет список пассажиров, купивших билет на заданный рейс.

Процедура «Показать фамилию, имя и отчество пассажиров, купивших билет на заданный рейс»:

```
CREATE PROC pShowPersonsForFlight (@ID_Flight INT)
AS
    SELECT *
    FROM vTickets
    WHERE Flights = @ID_Flight
RETURN;
```

Выполним разработанную процедуру:

```
EXEC pShowPersonsForFlight @ID_Flight = 1;
```

Результат выполнения скрипта представлен на рисунке 133.

Results		Messages							
	ID	Departure	Arrival	Date	Time	FirstName	PatronymicName	FamilyName	Flights
1	1	Москва	Санкт-Петербург	2022-12-20	10:00:00.0000000	Иван	Иванович	Иванов	1
2	2	Москва	Санкт-Петербург	2022-12-20	10:00:00.0000000	Светлана	Ивановна	Иванова	1

Рис. 133. Все пассажиры, улетевшие рейсом № 1

Функция «Сведения о рейсах, летающих по данному маршруту».

```
CREATE FUNCTION fFlightForRoute(@ID_Route INT)
RETURNS TABLE AS
RETURN (
```

```

SELECT
    CONVERT(VARCHAR(10), [Date], 104) AS [Date]
    , CONVERT(VARCHAR(8), [Time], 120) AS [Time]
FROM dbo.Flights
WHERE ID_Route = @ID_Route);

```

В результате выполнения скрипта:

```

DECLARE @Routes INT = 1;
SELECT @Routes AS N'Маршрут';
SELECT * FROM dbo.fFlightForRoute(@Routes);

```

получим следующий результат (рис. 134).

Results		Messages	
Маршрут			
1	1		
Date		Time	
1	20.12.2022	10:00:00	

**Рис. 134. Результат выполнения функции
«Рейсы по данному маршруту»**

Процедура «Билеты, которые купил данный пассажир»:

```

CREATE PROC pTicketsBoughtPassanger (@PassportNumber INT)
AS
    SELECT *
    FROM vTickets v
    WHERE v.PassportNumber = @PassportNumber;

```

В результате выполнения скрипта:

```
EXEC dbo.pTicketsBoughtPassanger @PassportNumber = 1111222221;
```

получим следующий результат (рис. 135).

Results Messages										
	ID	Departure	Arrival	Date	Time	FirstName	PatronymicName	FamilyName	PassportNumber	Flights
1	1	Москва	Санкт-Петербург	2022-12-20	10:00:00.0000000	Иван	Иванович	Иванов	1111222221	1

**Рис. 135. Результат выполнения функции
«Билеты, которые купил данный пассажир»**

Разработка необходимых триггеров

Триггер — это хранимая процедура, которая выполняется автоматически при выполнении соответствующего действия, описанного в этом триггере. Триггеры могут выполняться на удаление, добавление или модификацию данных в таблицах, однако такой механизм приводит к неоправданным затратам машинного времени при частой модификации данных.

В качестве примера напишем триггер, запрещающий удаление таблиц из нашей БД.

```
CREATE Trigger ddlt_PreventDDLDropTable
ON DATABASE FOR DROP_TABLE
AS
RAISERROR (N'Попытка удаления таблицы!', 10, 1);
ROLLBACK;
```

Выполним следующий скрипт:

```
DROP TABLE Tickets;
```

В результате мы получили сообщение (рис. 136), таблица Билеты не была удалена.

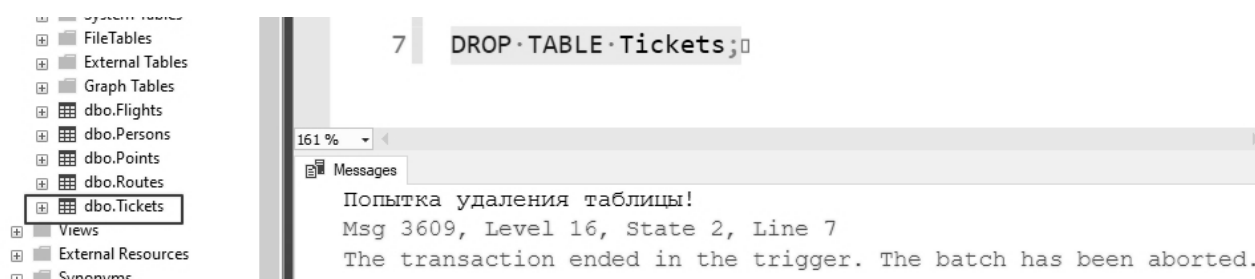


Рис. 136. Результат работы триггера

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Дейт, К. Д.* Введение в системы баз данных. — М. : Вильямс, 2019. — 1328 с.
2. *Кодд, Е. Ф.* Реляционная модель данных для больших совместно используемых банков данных / пер. М. Р. Когаловского // Системы управления базами данных. — 1995. — № 1.
3. *Codd, E. F.* A relational model of data for large shared data banks // Communications of the ACM. — 1970. — Vol. 13, № 6. — P. 377–387.
4. *Васильева, М. А.* Введение в базы данных : учебное пособие / М. А. Васильева, Е. П. Балакина. — М. : Рос. ун-т транспорта (МИИТ), 2007. — 80 с.
5. Отношение (реляционная модель). — URL: [https://ru.wikipedia.org/wiki/Отношение_\(реляционная модель\)](https://ru.wikipedia.org/wiki/Отношение_(реляционная_модель)) (дата обращения: 20.10.2022).
6. Реляционная алгебра. — URL: https://ru.wikipedia.org/wiki/Реляционная_алгебра (дата обращения: 20.10.2022).
7. Реляционное исчисление. — URL: https://ru.wikipedia.org/wiki/Реляционное_исчисление (дата обращения: 20.10.2022).
8. Википедия. Ссылочная целостность. — URL: https://ru.wikipedia.org/wiki/Ссылочная_целостность (дата обращения: 25.11.2022).
9. *Чен, П. П. III.* Модель «сущность — связь» — шаг к единому представлению о данных / пер. М. Р. Когаловского // ACM Transactions on Database Systems. — 1976. — Т. 1, № 1.
10. *Chen, P. P. S.* The entity-relationship model — toward a unified view of data // ACM Transactions on Database Systems. — 1976. — Vol. 1, № 1. — P. 9–36.
11. ER-модель. — URL: <https://ru.wikipedia.org/wiki/ER-модель> (дата обращения: 20.10.2022).
12. *Баркер, Р.* CASE Method — Моделирование взаимосвязей между сущностями. — Wokingham : Addison-Wesley, 1990.

13. *Васильева, М. А.* Фильтрация набора данных / М. А. Васильева, О. А. Тимофеева, М. К. Филипченко. — М. : РУТ (МИИТ), 2020. — 31 с.
14. © Microsoft 2020. SELECT-GROUP BY (Transact-SQL). — URL: <https://docs.microsoft.com/ru-ru/sql/t-sql/queries/select-group-by-transact-sql?view=sql-server-ver15> (дата обращения: 20.11.2020).
15. *Бен-Ган, И.* Microsoft SQL Server 2012. Основы T-SQL. — М. : Эксмо, 2015. — 400 с.
16. *Васильева М. А.* Информационное обеспечение систем управления : методические указания к курсовому проектированию / М. А. Васильева, Е. П. Балакина. — М. : Рос. ун-т транспорта (МИИТ), 2007. — 32 с.

ПРЕДМЕТНЫЙ УКАЗАТЕЛЬ

- Entity-Relationship diagrams
ER-диаграмма, 10
Атрибут, 8
Внешний ключ
Foreign key, 6, 96, 164, 165, 168, 169, 171
Вторичный ключ,
внешний ключ, 82, 96, 162
Кортеж, 5, 6, 8
Модель данных
 концептуальное описание
 предметной области, 7
 нормализация отношений, 11
 нормальная форма
 НФ, 12, 13
Отношение, 8
Relation, 8
Первичный ключ, 6, 9, 11, 82, 96-98, 162-166
ПК, 9, 12, 13, 182
Потенциальный ключ, 9, 163
Предметная область
 ПрО, 7, 10
 ПрО, domain, 5
 ПрО, domain, 8
 ПрО. domain, 7
Реляционная база данных
РБД, 3
Система управления базой данных
СУБД, 3
Система управления реляционной базой данных
СУРБД, 5
Ссылочная целостность, 5, 6, 7
Сущность, 8, 10, 12, 14, 159, 160, 163
Типы связей, 9, 11
Целостность сущности, 5

ОГЛАВЛЕНИЕ

Понятие реляционной модели данных	3
12 правил Кодда.....	3
Ограничения целостности	5
Модель данных	7
Краткие сведения об операторах SQL	15
Выбор полей	15
Фильтрация данных.....	19
Предикат LIKE	22
Предикат IN	28
Предикат BETWEEN.....	30
Определение переменных.....	31
Определение переменных некоторых типов	31
Преобразование типов данных.....	36
Работа со значениями NULL	39
Предложение DISTINCT.....	48
Сортировка результирующего набора данных	50
Агрегатные функции MS SQL Server	54
Использование агрегатных функций со значениями NULL	57
Итоги для сгруппированных данных	59
Использование предложения GROUP BY	59
Фильтрация сгруппированных данных, используя предложение HAVING.....	61
Изучение операторов ROLLUP и CUBE	62
Использование GROUPING SETS.....	69
GROUP BY ()	70
GROUPING.....	72
Ранжирование	74

ROW_NUMBER()	75
RANK()	77
DENSE_RANK()	79
NTILE()	81
Соединение данных из множества таблиц	82
Оператор INNER JOIN	82
Оператор OUTER JOIN	87
LEFT OUTER JOIN	87
RIGHT OUTER JOIN	91
FULL OUTER JOIN	93
Создание своей базы данных	95
Заполнение таблиц данными	97
Самосоединение SELF JOIN	104
Оператор UNION	110
Оператор EXCEPT	115
Оператор INTERSECT	117
Ограничение результирующего набора данных.	
Операторы TOP и TABLESAMPLE	118
Подзапросы	121
Автономные вложенные запросы	121
Скалярные вложенные запросы	121
Вложенные запросы с предикатами	123
Использование подзапросов в предложении FROM	133
Коррелированные вложенные запросы	135
Предикат EXISTS	140
Список заданий [16]	142
БД «Аптека»	142
БД «Поликлиника»	142

БД «Вуз»	143
БД «Складской учет»	143
БД «Отдел кадров»	144
БД «Кафедра».....	144
БД «Больница»	145
БД «Контора адвоката».....	145
БД «Архив»	146
БД «Продажа билетов»	146
БД «Магазин обуви»	147
БД «Библиотека»	147
БД «Издательство»	148
БД «Автосервис»	148
БД «Транспортная компания».....	149
БД «Туристическое бюро»	149
БД «ГАИ»	150
БД «Гостиница»	150
БД «Строительная компания»	151
БД «Компания по услугам связи».....	151
БД «Школа»	152
БД «Детский сад»	152
БД «Кондитерская фабрика»	153
БД «Хлебопекарня»	153
БД «Компания по продаже недвижимости»	154
БД «Овощной магазин»	154
БД «Мебельный магазин».....	155
БД «Магазин музыкальных произведений»	155
БД «Магазин „Детские товары“».....	156
БД «Фильмотека»	156

БД «Магазин по продаже оргтехники»	157
БД «Спортивная команда»	157
Пример проектирования реляционной базы данных	159
Инфологическое проектирование	159
Анализ предметной области.....	159
Анализ информационных задач и круга пользователей системы	161
Логическое проектирование реляционной БД	161
Преобразование ER-диаграммы в схему базы данных.....	161
Составление реляционных отношений	162
Нормализация полученных отношений	165
Определение дополнительных ограничений целостности.....	167
Физическое проектирование БД.....	167
Разработка скриптов на создание базы данных и таблиц	167
Разработка скриптов на добавление данных в таблицы.....	172
Разработка необходимых представлений (view).....	174
Разработка необходимых функций и процедур	176
Разработка необходимых триггеров.....	189
Список использованных источников	190
Предметный указатель	192